

Table of Contents

[HexVSDefTopic](#)

[hxabout.1.2](#)

[hxabout.1.3](#)

[hxabout.1.4](#)

[hxabout.1.5](#)

[hxabout.1.6](#)

[hxapps.5.01](#)

[hxapps.5.02](#)

[hxapps.5.03](#)

[hxapps.5.04](#)

[hxapps.5.05](#)

[hxapps.5.06](#)

[hxapps.5.07](#)

[hxapps.5.08](#)

[hxapps.5.09](#)

[hxapps.5.10](#)

[hxapps.5.11](#)

[hxapps.5.12](#)

[hxapps.5.13](#)

[hxapps.5.14](#)

[hxapps.5.15](#)

[hxapps.5.16](#)

[hxapps.5.17](#)

[hxapps.5.18](#)

[hxapps.5.19](#)

[hxapps.5.20](#)

[hxapps.5.21](#)

[hxapps.5.22](#)

[hxdebug.4.1](#)

[hxdebug.4.2](#)[hxdebug.4.3](#)[hxprog.3.01](#)[hxprog.3.02](#)[hxprog.3.03](#)[hxprog.3.04](#)[hxprog.3.05](#)[hxprog.3.06](#)[hxprog.3.07](#)[hxprog.3.08](#)[hxprog.3.09](#)[hxprog.3.10](#)[hxprog.3.11](#)[hxprog.3.12](#)[hxprog.3.13](#)[hxprog.3.14](#)[hxprog.3.15](#)[hxprog.3.16](#)[hxprog.3.17](#)[hxprog.3.18](#)[hxprog.3.19](#)[hxprog.3.20](#)[hxprog.3.21](#)[hxttheory.2.01](#)[hxttheory.2.02](#)[hxttheory.2.03](#)[hxttheory.2.04](#)[hxttheory.2.05](#)[hxttheory.2.06](#)[hxttheory.2.07](#)[hxttheory.2.08](#)[hxttheory.2.09](#)

- [hxttheory.2.10](#)
- [hxttheory.2.11](#)
- [hxttheory.2.12](#)
- [hxttheory.2.13](#)

HexVSDefTopic

<

>

About the HexVS

About the HexVS

The Hex Voltage Supply (HexVS) is a high-current power supply that allows you to test devices with high-current power requirements. This instrument board is made up of six 15A DC voltage supplies (DCVS) that can be merged to support a load current capacity of up to 90A. The HexVS can source voltage, measure voltage or current, and provide excellent force voltage accuracy. The HexVS High Accuracy includes all features of the HexVS and improves force voltage accuracy. In this description, the term “HexVS” refers to the HexVS and the HexVS High Accuracy unless otherwise noted.

This section includes the following topics:

- Features
- Safety Information
- Key Terms
- HexVS Calibration

- [HexVS Software Licensing Requirements](#)

HexVS information includes the following sections:

- [HexVS Theory of Operation](#)
- [HexVS DCVS Programming](#)
- [DCVS and HexVS Debug Displays](#)
- [HexVS Examples](#)

Related Topics

- [HexVS Specifications](#)
- [DCVS \(VBT syntax\)](#)
- [HexVS DIB Design Guidelines](#)

<

>

hxabout.1.2

<	>
-----------------	-----------------

About the HexVS : Features

Features

The HexVS instrument board provides a high-current power supply that allows you to test devices with high-current power supply requirements. Each instrument board is made up of six 15A DCVS instruments, each of which can supply 15A of power supply current. These six DCVSs can also be merged to support a load current capacity of up to 90A. Each DCVS can accurately force a programmed voltage level and measure power supply current to the DUT.

Each HexVS DCVS instrument has the following features:

- | | |
|---|---|
| • | Voltage output range of 0 to 5.5V |
| • | 15A maximum output current load capacity |
| • | Merging capability to provide up to 30A, 60A, or 90A of output current load capacity |
| • | Programmable multilevel overload and fold current limits |
| • | Programmable transient compensation bandwidth setting |
| • | Current measurement capability that includes measurement ranges of 10mA, 100mA, 1.0A, 15A, and an unfiltered High Speed 15A range |
| • | Pattern controlled measurement sampling, up to 256K samples |

- Real-time current monitor output at the DIB for active thermal control

The HexVS also offers on-board calibration resources, pattern-controlled voltage supply parameter settings for main and alternate voltage levels, and digital capture sampling rate and sample size. The following table shows several key specifications for the DCVS instrument.

DCVS Instrument Feature Highlights

Feature	Specification
DCVS channels	Six per board
Local voltage or current meters	Six (one per DCVS channel)
Voltage measurement ranges	4.0V and 8.0V
Force voltage range	0 to 5.5V
Filtered current measurement ranges	10mA, 100mA, 1.0A, 15A
Unfiltered (high speed) current measurement range	15A
Current range source	0 to 15A
Current range sink	2.0A
Maximum current output (single channel)	15A at up to 3.6V 10A at 5.0V 9.8A at 5.5V
Maximum merged current (six channels merged)	90A at 3.6V 60A at 5.0V 58.8A at 5.5V
Pattern control	Output levels, current measurement
Capture memory depth	256K of 12-bit current or voltage measurements at up to 25Msps
Capture memory sampling rates	24.4ksps to 25Msps, can be set to a value equal to 25M/n, where n = 1, 2, 3,..., 1024.

For more information, see HexVS Specifications.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxabout.1.3

<	>
-----------------	-----------------

About the HexVS : Safety Information

Safety Information

As defined by IEC standards, an energy hazard exists when power levels exceed 240W. This type of hazard can produce sparks and cause conductive parts to weld to electrically conductive surfaces if they come in contact with the high-current sources. By this definition, the HexVS board installed on the UltraFLEX test system represents an electrical hazard both to operators and equipment. Each of the six voltage sources on the HexVS can supply a maximum of 54W (15A at 3.6V). Merging these voltage sources together supplies up to 324W (90A at 3.6V). For complete safety information, see About Operator and Safety Information.

 Warning:

Because of the high current available from the HexVS instrument at the test head terminals, follow the safety precautions listed below to avoid injuries due to electric shock or burns and to prevent damage to the equipment such as melting or burning of components and boards. Observe the following safety precautions when using the HexVS.

- | | |
|--|---|
| | <ul style="list-style-type: none">Advise operators of the potential energy hazard and ask them to exercise caution during testing, especially if four or more high-current voltage sources are operating in parallel. |
| | <ul style="list-style-type: none">Do not touch the test head interface during testing or when the interface is powered on. |
| | <ul style="list-style-type: none">Avoid using conductive tools when using the test system and do not wear rings, watches, bracelets, necklaces, or other objects that are metallic or conductive. |

- Exercise caution when using conductive tools or probes. Make sure that these tools or probes do not come in contact with the high current sources or any other conductive part of the system.
- Exercise caution when probing on the test head to avoid inadvertent shorts between voltage or current terminals. Watch out especially for situations where a probe could become loose and touch two voltage or current terminals.
- Wear antistatic wrist straps with internal current limiting resistor because the HexVS is subject to static discharge.
- Wear safety glasses.
- Exercise caution when designing and running tests and when setting current limits for tests. You may see large current changes by simply changing voltage levels or turning the large current-consuming elements of a device on and off. Verify the ripple current specifications for the DIB capacitors to meet these rapid current changing conditions. This is true any time the voltage is changing rapidly during Vbump, shmoo, and other testing activities.
- Keep the high-current areas clear of any items that might alter their contacts. They must be clean and free of liquid or other conductors. Resistance from poor electrical contact may also result from physical damage, wear, or operation.
- Inspect the high-current area frequently and repair unsafe DIBs before use.
- Affix a "DANGER ELECTRICAL HAZARD" warning label to any DIB that uses the HexVS instrument, including customer-designed DIBs.
- Make sure that DIBs using the HexVS option have a protective cover with recessed or protected probe points, if present, to prevent operators from inadvertently touching the high-current components or dropping conductive tools or equipment onto them.
- Have an emergency response plan that all operators are familiar with. If an emergency occurs, immediately stop testing, get medical help if needed, and report the emergency to proper site

personnel. Do not touch the test head as items may be burning, hot, or still electrically charged.

- If you detect smoldering, sparks, component burning or melting, or other severe failure, power down the test system immediately by pressing an EMO button. Do not touch any of the damaged parts as they may still be very hot.



hxabout.1.4

<

>

About the HexVS : Key Terms

Key Terms

This section uses the following terms, abbreviations and acronyms:

capture	The process of storing individual voltage or current measurements taken in real time at a constant sample rate over a period of time.
capture memory	The HexVS hardware used to store sampled values. The HexVS can capture up to 256K samples at a sample rate of up to 25Msps.
current profile	The plot of sampled current measurements over time.
DCVS	Direct Current Voltage Supply. This term is used to refer to each channel of the HexVS board. Each DCVS on the HexVS board is an instrument with independently programmable voltage output, current limits, voltage or current meter, capture memory, and parameter set memories.
DGS	Device ground sense. See VSG.
force line	Wiring and DIB interposer that carries the voltage supply current to the DIB power planes and DUT.
HexVS	Hex Voltage Supply. This board has six DC voltage supply instruments per board. These supplies can be merged in pairs to increase the load current capacity to up to 90A.
PatGen	The HexVS pattern generator can be used to synchronize HexVS capabilities like capture, strobe, and Vbump to other instruments in the same PatGen domain. The PatGen controls the "real-time cadence of events" in a sequence.

sense line	An electrical connection used to measure the voltage at the Kelvin point. This connection is used to control the output of the force line to maintain the programmed output voltage at the desired level.
Vbump test	A test that changes the device power pin (VDD) level during operation to see if you can read from and write to the device correctly at different voltage levels. Vbump is also used to describe whenever the programmed voltage level for a device power supply is changed during a test pattern.
VSG	Voltage sense ground. VSG is a sense signal that tracks the voltage of digital ground on the DIB. This is equivalent to device ground sense (DGS) on other UltraFLEX boards.

< >	
	2015 Teradyne, Inc. All rights reserved.

hxabout.1.5

<	>
-----------------	-----------------

About the [HexVS](#) : HexVS Calibration

HexVS Calibration

HexVS calibration is handled internally by IG-XL and does not require the use of a calibration DIB.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxabout.1.6

<	>
-----------------	-----------------

[About the HexVS](#) : HexVS Software Licensing Requirements

HexVS Software Licensing Requirements
Operation of the HexVS does not require a software license.
For more information, see [IG-XL Licensing](#) in IG-XL Administration help.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxapps.5.01

<	>
--------------------------------------	--------------------------------------

HexVS Examples

HexVS Examples

This section contains two kinds of program examples for the HexVS:

- IG-XL workbook examples show all IG-XL data, VBT code, and patterns (if used) necessary for a complete running program.
- VBT function examples show one or more VBT functions that perform a programmed action when called as specified.

Workbook Examples

- HexVS Voltage Source Programming
- HexVS Static Icc Programming
- HexVS Dynamic Icc Programming Using VBT

VBT Function Examples

- Basic DCVS Setup Programming
- Vbump Testing
- Current Meter Setup, Strobe, and Readback Programming

- [Current Profiling](#)
- [HexVS Kelvin Alarm Threshold Programming](#)
- [DCVS Current Range and Limit Programming](#)

Related Topics

- [DCVS \(VBT Syntax Reference\)](#)
- [HexVS DCVS Programming](#)

<

>

2015 Teradyne, Inc. All rights reserved.

hxapps.5.02

<	>
-----------------	-----------------

HexVS Examples : HexVS Voltage Source Programming

HexVS Voltage Source Programming

This workbook example shows how to program the HexVS as a voltage source (FVMV) to test two SN74ALS245AN bidirectional digital transceiver devices on two sites. This example uses one HSD1000 board in slot 16 and a HexVS board in slot 19.

This workbook example includes the following program components:

- | | |
|---|-----------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | Patterns |
| • | VBT Code |

<	>
-----------------	-----------------

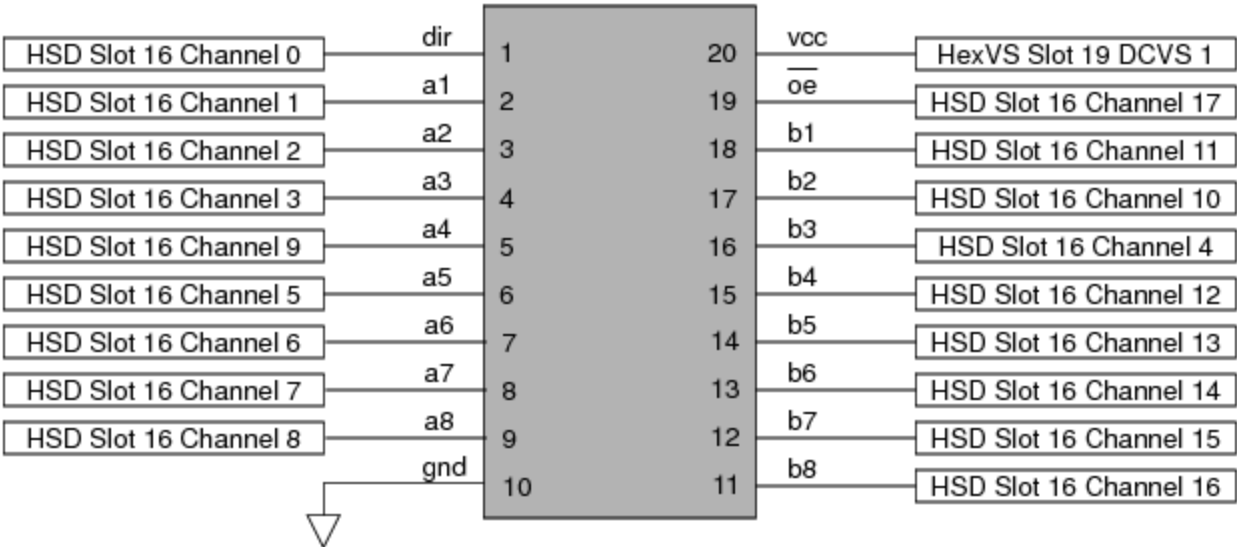
hxapps.5.03

<

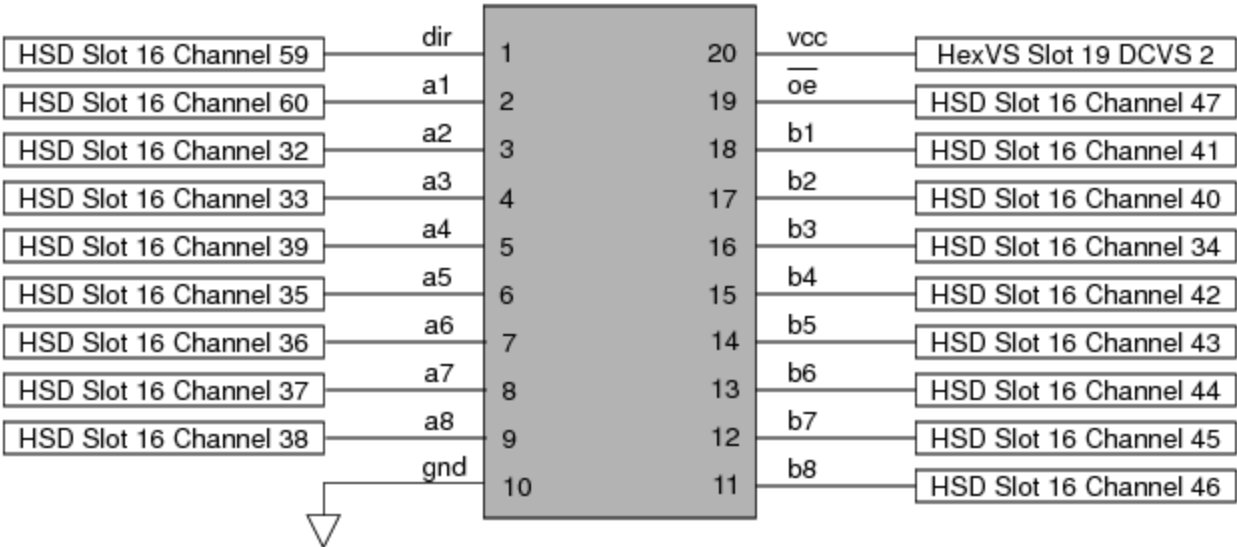
>

HexVS Examples : [HexVS Voltage Source Programming](#) : DUT Connections

DUT Connections



HSD1000 Connections for Digital Transceiver Device Site 0



HSD1000 Connections for Digital Transceiver Device Site 1

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxapps.5.04

<	>
-----------------	-----------------

HexVS Examples : [HexVS Voltage Source Programming](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|--------------------------------------|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Flow Table Sheet |
| • | DC Specs Sheet |
| • | Test Instances Sheet |

Pin Map Sheet

Defining the pins:

Pin Map

Group Name	Pin Name	Type
	dir	Input
	a1	I/O
	a2	I/O
	a3	I/O
	a4	I/O
	a5	I/O
	a6	I/O
	a7	I/O
	a8	I/O
	gnd	Gnd
	b8	I/O
	b7	I/O
	b6	I/O
	b5	I/O
	b4	I/O
	b3	I/O
	b2	I/O
	b1	I/O
	oe	Input
	vcc	Power

Defining the pin groups:

Pin Map

Group Name	Pin Name	Type
ctrls	oe	Input
ctrls	dir	
porta	a1	I/O
porta	a2	
porta	a3	
porta	a4	
porta	a5	
porta	a6	
porta	a7	
porta	a8	
portb	b1	I/O
portb	b2	
portb	b3	
portb	b4	
portb	b5	
portb	b6	
portb	b7	
portb	b8	
ports	porta	I/O
ports	portb	
digital_pins	ctrls	I/O
digital_pins	ports	

[Channel Map Sheet](#)

Channel Map

DIB ID:

View Mode: Signal

Device Under Test			Tester Channel	
Pin Name	Package Pin	Type	Site 0	Site 1
dir	Pin 1	I/O	16.ch0	16.ch59
a1	Pin 2	I/O	16.ch1	16.ch60
a2	Pin 3	I/O	16.ch2	16.ch32
a3	Pin 4	I/O	16.ch3	16.ch33
a4	Pin 5	I/O	16.ch9	16.ch39
a5	Pin 6	I/O	16.ch5	16.ch35
a6	Pin 7	I/O	16.ch6	16.ch36
a7	Pin 8	I/O	16.ch7	16.ch37
a8	Pin 9	I/O	16.ch8	16.ch38
gnd	Pin 10	N/C		
b1	Pin 18	I/O	16.ch11	16.ch41
b2	Pin 17	I/O	16.ch10	16.ch40
b3	Pin 16	I/O	16.ch4	16.ch34
b4	Pin 15	I/O	16.ch12	16.ch42
b5	Pin 14	I/O	16.ch13	16.ch43
b6	Pin 13	I/O	16.ch14	16.ch44
b7	Pin 12	I/O	16.ch15	16.ch45
b8	Pin 11	I/O	16.ch16	16.ch46
oe	Pin 19	I/O	16.ch17	16.ch47
vcc	Pin 20	DCVS	19.sense1	19.sense2

[Flow Table Sheet](#)

Flow Table

Label	Enable	Gate			Command		TName	TNum	Limits		Bin Number		Port Number		Result
		Job	Part	Env	Opcode	Parameter			LoLim	HiLim	Pass	Fail	Pass	Fail	
					set-error-bin							99		999	
					Test	FVMV									
					Use-Limit	FVMV	FVMV	10	1.985	2.015		1		1	Fail
					set-device						1		1		Pass
					stop										

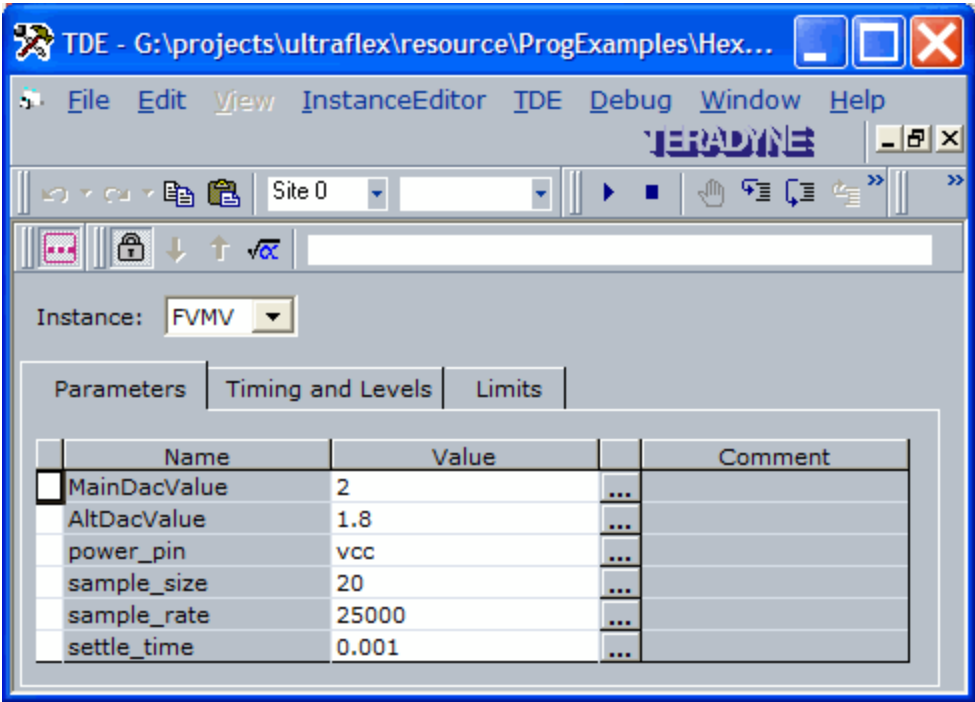
[DC Specs Sheet](#)

DC Specs		TERADYNE			
		Selector		Commercial	Zero Spec
Symbol	Value	Name	Val	Typ	Typ
vcc_value		vcc_min	Typ	5	0
vcc_value	0	vcc_max	Typ		
vcc_value	0	vcc_nom	Typ		

Test Instances Sheet

Test Instances				TERADYNE								
Test Procedure				DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
FVMV	VBT	FVMV		Zero_Spec	vcc_nom							

The FVMV test instance has the following test-specific parameters and values as shown in the Instance Editor:



hxapps.5.05

<	>
-----------------	-----------------

[HexVS Examples](#) : [HexVS Voltage Source Programming](#) : Patterns

Patterns

This program example does not require any pattern files.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxapps.5.06

	
---	---

[HexVS Examples](#) : [HexVS Voltage Source Programming](#) : VBT Code

VBT Code

Option Explicit

Private Sub HexVS_Notes()

```
' HexVS notes:
' Force Voltage (0 to 5.5V)
' Measure Voltage Ranges (4V, 8V)
' Max Power (54W/supply, 324W/brd)
' Source OverLoadLimit (1.05A-> 16.5A)
' Source FoldLimit (50mA-> 15.5A)
' Source OverLoadLimit_TimeOut (102.4us-> 300ms)
' Source FoldLimit_TimeOut (102.4us-> 5s)
' Iranges (10mA, 100mA, 1A, 15A, HS15A)
' Measure Capture/Sample Rate 25M/N, N=1,2,... 1024
' Program current clamps in order of time 1, 2, 3, -> t
' or use the function to turn on at same time
```

End Sub

Public Function FVMV(MainDacValue As Double, _
 AltDacValue As Double, _
 power_pin As PinList, _
 sample_size As Double, _
 sample_rate As Double, _
 settle_time As Double)

Dim Result As New PinListData

Dim Result2 As New PinListData

Dim Foldlev As Double

Dim OvLevel As Double

Dim Site As Variant

' - - - Power Supply and Current Limit Setup - - -

```
With TheHdw.DCVS.Pins("vcc")      ' VCC (HexVS channel 1)
    .Voltage.Main.Value = MainDacValue      ' Main Voltage (2V)
    .Voltage.Alt.Value = AltDacValue        ' Alternative Voltage Source (1.8V)
    .CurrentRange.Value = 1                ' Current Range (1A)
    .Gate = False
        ' a) Enable or Disable the Overload Limiting
        ' OR
        ' b) Change Fold or Overload Limit Timeout values
        ' otherwise a run-time error will occur
```

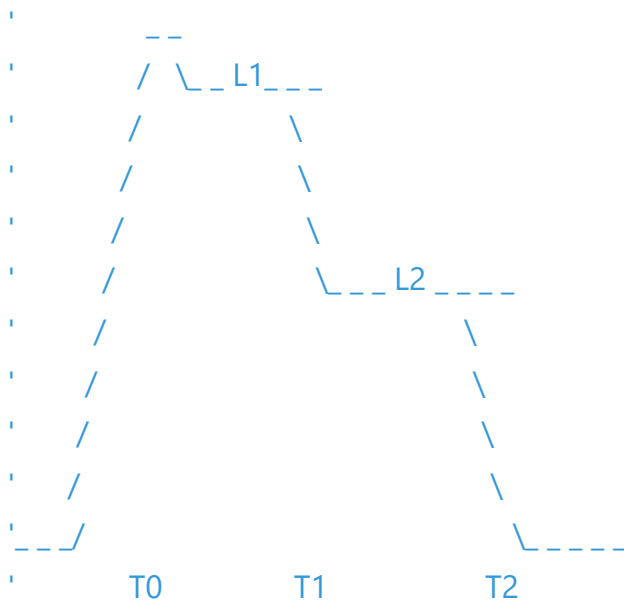
' Single-level source current limit

' With .CurrentLimit.Source

```
' .OverloadLimit.Enabled = False
' .FoldLimit.Level.Value = (0.65 * .FoldLimit.Level.Max)
' .FoldLimit.TimeOut.Value = 0.01
' .FoldLimit.Behavior = tlDCVSCurrentLimitBehaviorGateOff
' End With
```

' - - - Dual-Level Clamp Setup - - -

' If not sure what to expect, then start high or with max values



```
' .CurrentLimit.Source.SetLimitLevels(Foldlev, Overldlev)
' .CurrentLimit.Source.SetLimitTimeouts(Foldtime, Overldtime)
```

With .CurrentLimit.Source

```
.FoldLimit.Behavior = tlDCVSCurrentLimitBehaviorGateOff ' Gated off if L2 til T2
.FoldLimit.Level.Value = 4      ' Fold Limit = 4A
```

```

.FoldLimit.TimeOut.Value = 0.008      ' Fold TO (T2) must be > T1+92.16us
.OverloadLimit.Enabled = True          ' Enable Overload limits
.OverloadLimit.Level.Value = 10        ' Overload Limit=10A, (L1) must be >=L2+1A
.OverloadLimit.TimeOut.Value = 0.005   ' Overload Timeout(T1) = 500ms
End With

' - - - Setup Continued - - -

.Alarm(tIDCVSAlarmAll) = tlAlarmDefault ' All Alarms On
.BandwidthSetting.Value = 0             ' Min BW (0 - 7)
.BleederResistor = tIDCVSAuto           ' Auto is default
.LocalCapacitor = tIDCVSAuto           ' Auto is default
.Mode = tIDCVSModeVoltage               ' Force Voltage Mode
.Meter.Mode = tIDCVSMeterVoltage        ' Meter in Voltage Mode
.Meter.VoltageRange.Value = 4           ' Meter Vrange = 4

' - - - Turn On - - -

.Voltage.Output = tIDCVSVoltageMain     ' Output setup
.Connect (tIDCVSConnectDefault)        ' Connect Force and Sense
.Gate = True                            ' Gate Supply on
' TheHdw.Wait (2 * settle_time)          ' Wait 2ms

' - - - Take Measurement - - -
' Meter strobes rate=25K samples/sec. and reads back an average of 20 samples.
Result = TheHdw.DCVS.Pins("vcc").Meter.Read(tlStrobe, sample_size, _
' TheHdw.Wait (2 * settle_time)          ' Wait 2ms

' - - - Verification - DATALOG RESULT(s) - - -
' Set up OFFLINE Simulation by stuffing
' the PinListData variable with simulation data
If TheExec.TesterMode = testModeOffline Then
    For Each Site In TheExec.Sites
        Result.Value = 2# + CDBl(Site) * 0.005
    Next Site
End If

TheExec.Flow.TestLimit ResultVal:=Result, _
    unit:=unitVolt, _
    pinName:=power_pin, _
    forceVal:=MainDacValue, _
    forceUnit:=unitVolt, _
    ForceResults:=tlForceFlow

Exit Function
errHandler:

```

TheExec.AddOutput "Error in the FVMV "

' - - - Power Down and Disconnect - - -

TheHdw.DCVS.Pins("vcc").Gate = False

.Disconnect (tlDCVSConnectNone)

End With

End Function

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxapps.5.07

<	>
-----------------	-----------------

HexVS Examples : HexVS Static Icc Programming

HexVS Static Icc Programming

This workbook example shows how to program a HexVS static Icc test that forces voltage and measures current (FVMI) on two SN74ALS245AN bidirectional digital transceiver devices on two sites. This example connects one HexVS channel to the Vcc pin, applies 5.1V to the pin, and measures the resulting current while the digital pins are set to a known state. This example uses one HSD1000 board in slot 16 and a HexVS board in slot 19.

This workbook example includes the following program components:

- | | |
|---|-----------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | Patterns |
| • | VBT Code |

<	>
-----------------	-----------------

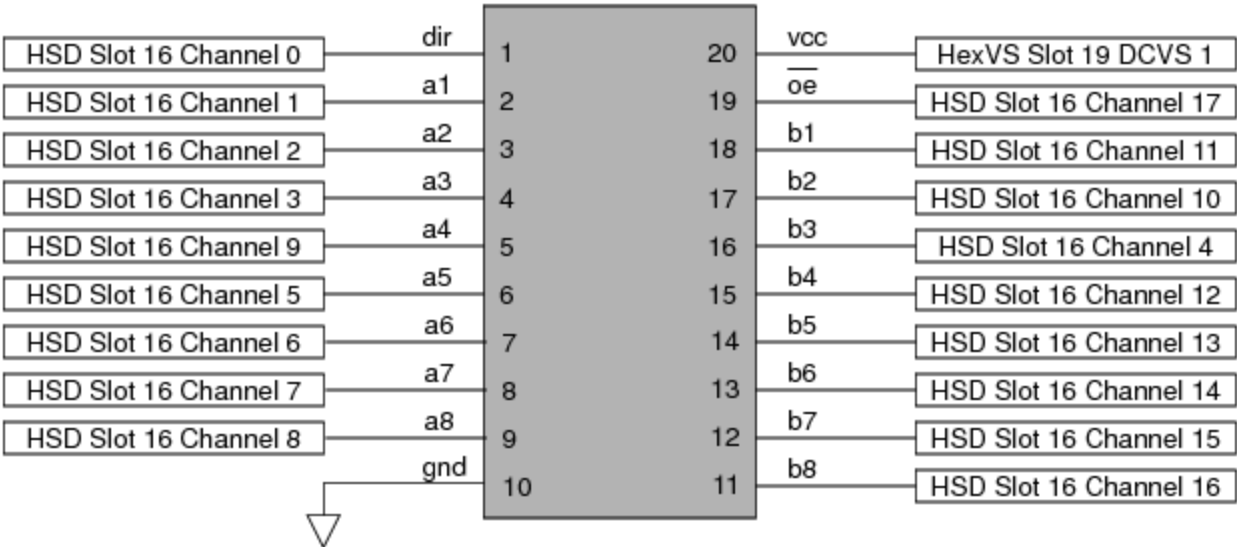
hxapps.5.08

<

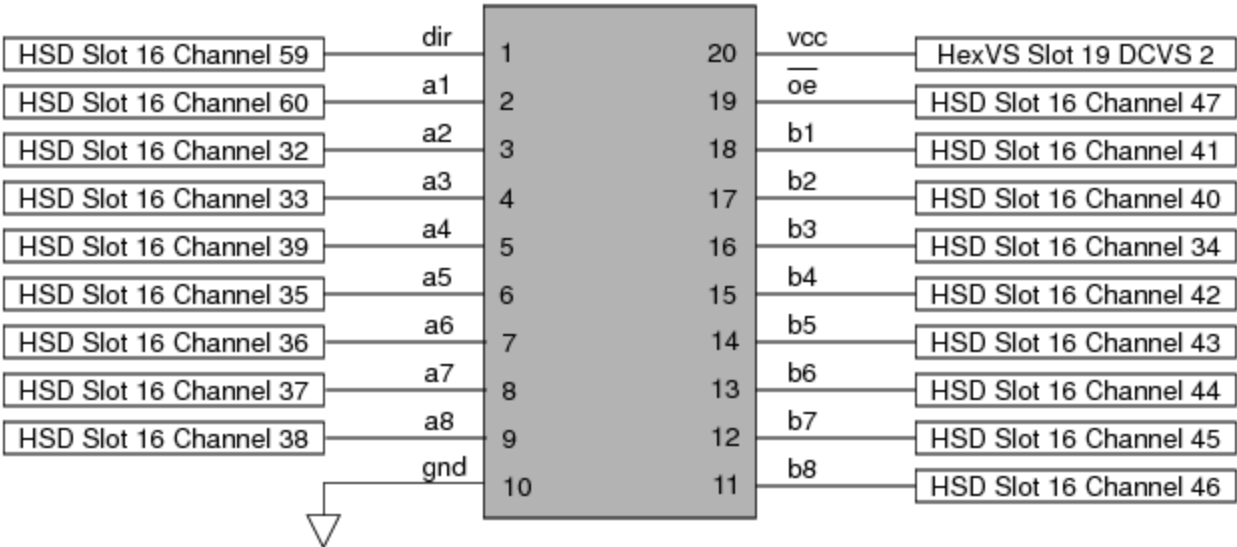
>

HexVS Examples : HexVS Static Icc Programming : DUT Connections

DUT Connections



HSD1000 Connections for Digital Transceiver Device Site 0



HSD1000 Connections for Digital Transceiver Device Site 1

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxapps.5.09

<	>
-----------------	-----------------

HexVS Examples : [HexVS Static Icc Programming](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|--------------------------------------|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Flow Table Sheet |
| • | Pin Levels Sheet |
| • | Global Specs Sheet |
| • | DC Specs Sheet |
| • | Test Instances Sheet |

Pin Map Sheet

Defining the pins:

Pin Map

Group Name	Pin Name	Type
	dir	Input
	a1	I/O
	a2	I/O
	a3	I/O
	a4	I/O
	a5	I/O
	a6	I/O
	a7	I/O
	a8	I/O
	gnd	Gnd
	b8	I/O
	b7	I/O
	b6	I/O
	b5	I/O
	b4	I/O
	b3	I/O
	b2	I/O
	b1	I/O
	oe	Input
	vcc	Power

Defining the pin groups:

Pin Map

Group Name	Pin Name	Type
ctrls	oe	Input
ctrls	dir	
porta	a1	I/O
porta	a2	
porta	a3	
porta	a4	
porta	a5	
porta	a6	
porta	a7	
porta	a8	
portb	b1	I/O
portb	b2	
portb	b3	
portb	b4	
portb	b5	
portb	b6	
portb	b7	
portb	b8	
ports	porta	I/O
ports	portb	
digital_pins	ctrls	I/O
digital_pins	ports	

[Channel Map Sheet](#)

Channel Map

DIB ID:

View Mode: Signal

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Site 1
dir	Pin 1	I/O	16.ch0	16.ch59
a1	Pin 2	I/O	16.ch1	16.ch60
a2	Pin 3	I/O	16.ch2	16.ch32
a3	Pin 4	I/O	16.ch3	16.ch33
a4	Pin 5	I/O	16.ch9	16.ch39
a5	Pin 6	I/O	16.ch5	16.ch35
a6	Pin 7	I/O	16.ch6	16.ch36
a7	Pin 8	I/O	16.ch7	16.ch37
a8	Pin 9	I/O	16.ch8	16.ch38
gnd	Pin 10	N/C		
b1	Pin 18	I/O	16.ch11	16.ch41
b2	Pin 17	I/O	16.ch10	16.ch40
b3	Pin 16	I/O	16.ch4	16.ch34
b4	Pin 15	I/O	16.ch12	16.ch42
b5	Pin 14	I/O	16.ch13	16.ch43
b6	Pin 13	I/O	16.ch14	16.ch44
b7	Pin 12	I/O	16.ch15	16.ch45
b8	Pin 11	I/O	16.ch16	16.ch46
oe	Pin 19	I/O	16.ch17	16.ch47
vcc	Pin 20	DCVS	19.sense1	19.sense2

Flow Table Sheet

Flow Table



Label	Enable	Gate			Command		TName	TNum	Limits		Bin Number		Sort Number		Result
		Job	Part	Env	Opcode	Parameter			LoLim	HiLim	Pass	Fail	Pass	Fail	
					set-error-bin							99		999	
					Test	FVMI									
					Use-Limit	FVMI	FVMI	12	0.05	0.1		2		2	Fail
					Use-Limit	FVMI	FVMI	13	0.3	0.5		3		3	Fail
					set-device						1		1		Pass
					stop										

Pin Levels Sheet

Defined using formulas:

Pin Levels

Pin/Group	Seq.	Parameter	Value
vcc		VMain	5.1
vcc		VAlt	4.65
vcc		Tdelay	0.02
ctrls		Vil	0
ctrls		Vih	5
ctrls		Vt	$=1/2*(E7+E8)$
ctrls		Vcl	=_Vcl_default
ctrls		Vch	=_Vch_default
ctrls		DriverMode	Largeswing-HiZ
ports		Vil	0
ports		Vih	5
ports		Vol	0.8
ports		Voh	2.4
ports		Vt	=1.6
ports		Vcl	=_Vcl_default
ports		Vch	=_Vch_default
ports		VoutLoTyp	0.1
ports		VoutHiTyp	4.9
ports		DriverMode	Largeswing-VT1K

Formulas evaluated:

Pin Levels

Pin/Group	Seq.	Parameter	Value
vcc		VMain	5.1
vcc		VAlt	4.65
vcc		Tdelay	0.02
ctrls		Vil	0
ctrls		Vih	5
ctrls		Vt	2.5
ctrls		Vcl	-1
ctrls		Vch	6
ctrls		DriverMode	Largeswing-HiZ
ports		Vil	0
ports		Vih	5
ports		Vol	0.8
ports		Voh	2.4
ports		Vt	1.6
ports		Vcl	-1
ports		Vch	6
ports		VoutLoTyp	0.1
ports		VoutHiTyp	4.9
ports		DriverMode	Largeswing-VT1K

Global Specs Sheet

Global Specs

Symbol	Job	Value	Comment
Vcl_default		-1	Detector clamp voltage low
Vch_default		6	Detector clamp voltage high

DC Specs Sheet

DC Specs



		Selector		Commercial			Zero Spec		
Symbol	Value	Name	Val	Typ	Min	Max	Typ	Min	Max
vcc_value	5	vcc_min	Typ	5	4.5	0	0	0	0
vcc_value	5	vcc_max	Typ	5	4.5	0			
vcc_value	5	vcc_nom	Typ	5	4.5	0			

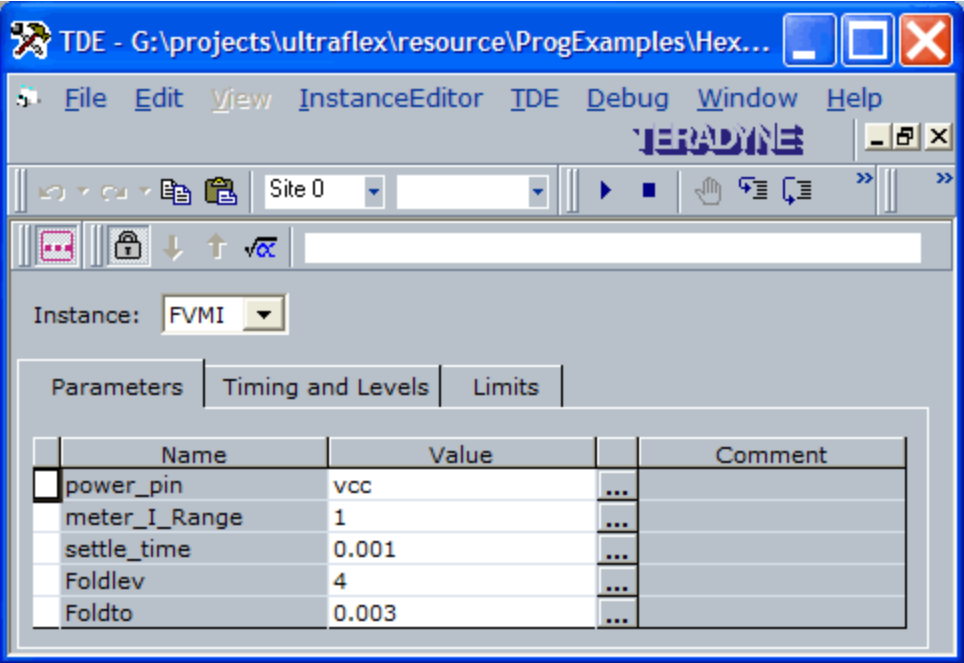
Test Instances Sheet

Test Instances

TERADYNE

Test Procedure				DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
FVMI	VBT	FVMI		Zero_Spec	vcc_nom					Levels		

The FVMI test instance has the following test-specific parameters and values as shown in the Instance Editor:



hxapps.5.10

	
---	---

[HexVS Examples](#) : [HexVS Static lcc Programming](#) : Patterns

Patterns

This program example does not require any pattern files.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxapps.5.11



[HexVS Examples](#) : [HexVS Static Icc Programming](#) : VBT Code

VBT Code

```
Public Function FVMI(power_pin As PinList, _  
    meter_I_Range As Double, _  
    settle_time As Double, _  
    Foldlev As Double, _  
    Foldto As Double)  
  
Dim HexMeasure As New PinListData  
Dim HexMeasure2 As New PinListData  
Dim Site As Variant  
On Error GoTo errHandler  
' Apply HSD levels, Init States, and Powersupply pin values  
' Connect all pins, load levels, do not load timing (not needed)  
Call TheHdw.Digital.ApplyLevelsTiming(True, True, False, tlPowered)  
' Supply Setup  
With TheHdw.DCVS.Pins(power_pin)  
    .Gate = False  
    ' Single Clamp setup  
    With .CurrentLimit.Source  
        .OverloadLimit.Enabled = False  
        .FoldLimit.Behavior = tlDCVSCurrentLimitBehaviorGateOff  
        .FoldLimit.Level.Value = Foldlev  
        .FoldLimit.TimeOut.Value = Foldto  
    End With  
    .Meter.Mode = tlDCVSMeterCurrent  
    .Meter.CurrentRange.Value = meter_I_Range
```

.Gate = True

' Strobe the meter on the VCC pin and

' store it in an PinListData variable defined.

' Settle time may be required for valid measurement,

' especially after down-ranging meter range

TheHdw.Wait (settle_time)

HexMeasure = TheHdw.DCVS.Pins(power_pin).Meter.Read(tlStrobe)

' Change static state, then measure again.

TheHdw.Digital.disconnectPins ("portB")

HexMeasure2 = TheHdw.DCVS.Pins(power_pin).Meter.Read(tlStrobe)

End With

' Set up OFFLINE Simulation by stuffing the

' PinListData variable with simulation data.

If TheExec.TesterMode = testModeOffline Then

For Each Site In TheExec.Sites

HexMeasure.Value = 0.078 + Cdbl(Site) * 0.005

HexMeasure2.Value = 0.358 + Cdbl(Site) * 0.005

Next Site

End If

' - - - DATALOG RESULTS - - -

TheExec.Flow.TestLimit ResultVal:=HexMeasure, unit:=unitAmp, pinName:=power_pin, _

ForceResults:=tlForceFlow

Call TheExec.Datalog.WriteComment("<FVMI> Static State Change")

TheExec.Flow.TestLimit ResultVal:=HexMeasure2, unit:=unitAmp, pinName:="vcc", _

ForceResults:=tlForceFlow

Exit Function

errHandler:

TheExec.AddOutput "Error in the Static (FVMI) Icc Test"

End Function

<

>

hxapps.5.12

<	>
-----------------	-----------------

HexVS Examples : HexVS Dynamic lcc Programming Using VBT

HexVS Dynamic lcc Programming Using VBT

This workbook example shows how to program a HexVS dynamic lcc test that forces voltage and measures current (FVMI) on the SN74ALS245AN bidirectional digital transceiver device in multisite mode (in this case, on two sites). This example uses one HSD1000 board in slot 16 and a HexVS board in slot 19. Test program control is handled through VBT code.

This example implements a dynamic lcc test by using the DCVS instrument to make multiple current measurements at three different Vcc levels while the HSD1000 PatGen is looping through a functional pattern. By implementing a handshaking technique, the DCVS and HSD1000 can communicate with each other using the four CPU flags. This example uses only one flag, cpuA. This CPU flag allows the test program to monitor the PatGen flag condition through the three pattern loops. The test program requires this flag because both controllers are running at different speeds (with the PatGen running much faster than the CPU).

Each pattern loop has the Vcc force condition programmed to a different Vcc level, with 5.5V for the first loop, 5.0V for the second loop, and 4.5V for the third loop. With this setup, the test program follows this sequence:

- | | |
|----|--|
| 1. | The PatGen begins the pattern burst with the DCVS main DAC level set to 5.5V. |
| 2. | The CPU reads the DCVS meter and makes a measurement based on the programmed sample size and sample rate. |
| 3. | Once the DCVS strobing takes place, the CPU flag allows the PatGen to test the branch condition and continue through to the next pattern loop at Vcc Nom with the DCVS main DAC level set to 5.0V. |

4.

The CPU reads the DCVS meter and makes a measurement again, this time with the force condition at 5.0V.
5.

Once this strobe has occurred, the PatGen tests the branch condition one more time allowing the pattern flow to move on the Vcc Min loop with the DCVS main DAC level set to 4.5V.
6.

The CPU reads the DCVS meter and makes a final measurement, this time with the force condition at 4.5V.
7.

After the last measurement is made, the CPU flag allows the PatGen to fall out of the Vcc Min loop, and the PatGen halt opcode forces the Patten Generator to stop.
8.

The CPU continues to take readings (an average of measurements at the three levels) and compares the result values at 5.5V, 5.0V and 4.5V to the specified device limits.

This workbook example includes the following program components:

- DUT Connections
- DataTool Sheets
- Patterns
- VBT Code

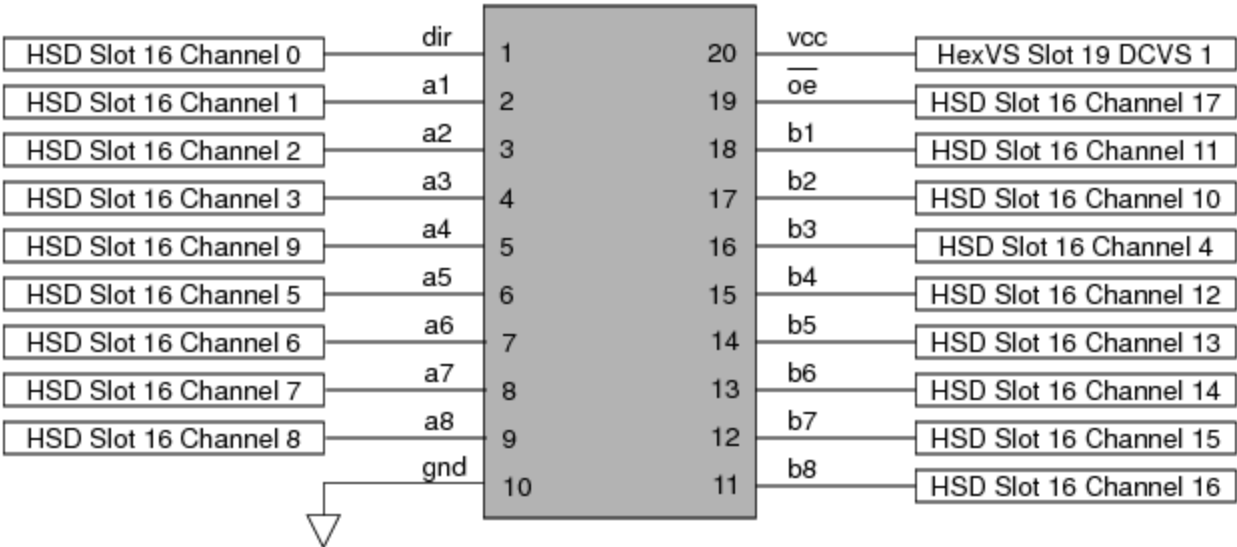
hxapps.5.13

<

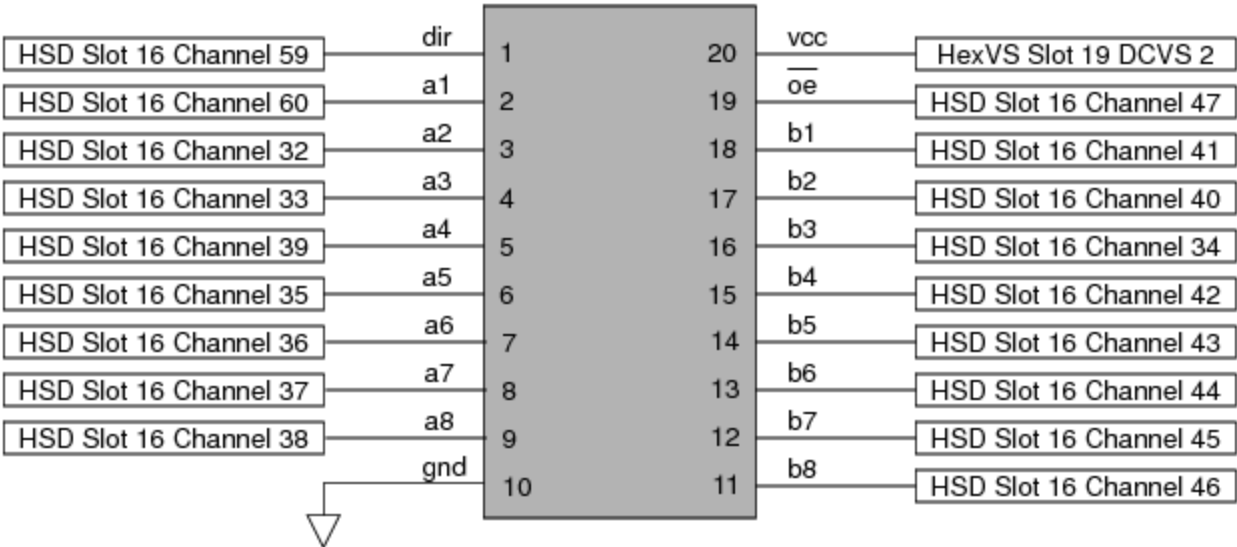
>

HexVS Examples : HexVS Dynamic Icc Programming Using VBT : DUT Connections

DUT Connections



HSD1000 Connections for Digital Transceiver Device Site 0



HSD1000 Connections for Digital Transceiver Device Site 1

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxapps.5.14

<	>
-----------------	-----------------

HexVS Examples : [HexVS Dynamic Icc Programming Using VBT](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|--------------------------------------|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Flow Table Sheet |
| • | Pin Levels Sheet |
| • | Global Specs Sheet |
| • | DC Specs Sheet |
| • | AC Specs Sheet |
| • | Test Instances Sheet |

[Pin Map Sheet](#)

Defining the pins:

Pin Map

Group Name	Pin Name	Type
	dir	Input
	a1	I/O
	a2	I/O
	a3	I/O
	a4	I/O
	a5	I/O
	a6	I/O
	a7	I/O
	a8	I/O
	gnd	Gnd
	b8	I/O
	b7	I/O
	b6	I/O
	b5	I/O
	b4	I/O
	b3	I/O
	b2	I/O
	b1	I/O
	oe	Input
	vcc	Power

Defining the pin groups:

Pin Map

Group Name	Pin Name	Type
ctrls	oe	Input
ctrls	dir	
porta	a1	I/O
porta	a2	
porta	a3	
porta	a4	
porta	a5	
porta	a6	
porta	a7	
porta	a8	
portb	b1	I/O
portb	b2	
portb	b3	
portb	b4	
portb	b5	
portb	b6	
portb	b7	
portb	b8	
ports	porta	I/O
ports	portb	
digital_pins	ctrls	I/O
digital_pins	ports	

Channel Map Sheet

Channel Map

DIB ID: View Mode:

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Site 1
dir	Pin 1	I/O	16.ch0	16.ch59
a1	Pin 2	I/O	16.ch1	16.ch60
a2	Pin 3	I/O	16.ch2	16.ch32
a3	Pin 4	I/O	16.ch3	16.ch33
a4	Pin 5	I/O	16.ch9	16.ch39
a5	Pin 6	I/O	16.ch5	16.ch35
a6	Pin 7	I/O	16.ch6	16.ch36
a7	Pin 8	I/O	16.ch7	16.ch37
a8	Pin 9	I/O	16.ch8	16.ch38
gnd	Pin 10	N/C		
b1	Pin 18	I/O	16.ch11	16.ch41
b2	Pin 17	I/O	16.ch10	16.ch40
b3	Pin 16	I/O	16.ch4	16.ch34
b4	Pin 15	I/O	16.ch12	16.ch42
b5	Pin 14	I/O	16.ch13	16.ch43
b6	Pin 13	I/O	16.ch14	16.ch44
b7	Pin 12	I/O	16.ch15	16.ch45
b8	Pin 11	I/O	16.ch16	16.ch46
oe	Pin 19	I/O	16.ch17	16.ch47
vcc	Pin 20	DCVS	19.sense1	19.sense2

Flow Table Sheet

Flow Table



		Gate			Command				Limits		Datalog Display Results			Bin Number		Sort Number		
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim	Scale	Units	Format	Pass	Fail	Pass	Fail	Result
					Test	Dyn_lcc_vb												
					Use-Limit	Dyn_lcc_vb	Dynlcc_55	10	0.02	0.08					8		855	Fail
					Use-Limit	Dyn_lcc_vb	Dynlcc_50	15	0.02	0.08					8		850	Fail
					Use-Limit	Dyn_lcc_vb	Dynlcc_45	20	0.02	0.08					8		845	Fail
					set-device										1		1	Pass
					stop													

Pin Levels Sheet

Defined using formulas:

Pin Levels

Pin/Group	Seq.	Parameter	Value
vcc		VMain	=_vcc_value-0.5
vcc		VAlt	=_vcc_value
vcc		IFoldLevel	=_gl_lsc
vcc		Tdelay	=_gl_Tdelay
ctrls		Vil	=_vil
ctrls		Vih	=_vih
ctrls		Vt	=_Vt
ctrls		Vcl	=_Vcl_default
ctrls		Vch	=_Vch_default
ctrls		DriverMode	Largeswing-HiZ
ports		Vil	=_vil
ports		Vih	=_vih
ports		Vol	=_vol
ports		Voh	=_voh
ports		Vt	=_Vt
ports		Vcl	=_Vcl_default
ports		Vch	=_Vch_default
ports		DriverMode	Largeswing-VT1K

Formulas evaluated:

Pin Levels

Pin/Group	Seq.	Parameter	Value
vcc		VMain	4.5
vcc		VAlt	5
vcc		IFoldLevel	0.5
vcc		Tdelay	0.02
ctrls		Vil	0.8
ctrls		Vih	5
ctrls		Vt	2
ctrls		Vcl	-1
ctrls		Vch	6
ctrls		DriverMode	Largeswing-HiZ
ports		Vil	0.8
ports		Vih	5
ports		Vol	0.4
ports		Voh	2.4
ports		Vt	2
ports		Vcl	-1
ports		Vch	6
ports		DriverMode	Largeswing-VT1K

Global Specs Sheet

Global Specs

Symbol	Job	Value	Comment
Vcl_default		-1	Detector clamp voltage low
Vch_default		6	Detector clamp voltage high
gl_lsc		0.5	
gl_Tdelay		=20*ms	
lcc_sample_rate		=37*KHz	
lcc_sample_size		=19000	

DC Specs Sheet

DC Specs



Symbol	Value	Selector		Commercial			Zero Spec		
		Name	Val	Typ	Min	Max	Typ	Min	Max
vil	0.8	vcc_nom	Typ	0.8	0.8	0.8	0	0	0
voh	2.4	vcc_nom	Typ	2.4	2.4	2.4	0	0	0
vol	0.4	vcc_nom	Typ	0.4	0.4	0.4	0	0	0
vcc_value	5	vcc_nom	Typ	5	4.5	5.5	0	0	0
vcc_for_DynIcc	5	vcc_nom	Typ	5	5	5	0	0	0
Vt	2	vcc_nom	Typ	2	2	2	0	0	0
vih	5	vcc_nom	Typ	5	5	5	0	0	0

AC Specs Sheet

AC Specs



Symbol	Value	Selector		Commercial		
		Name	Val	Typ	Min	Max
tplk	0.00000004	max_AC	Min	0.00000005	0.00000004	0.00000006
Period	0.0000005	max_AC	Typ	0.0000005	0.0000005	0.0000005

Test Instances Sheet

Test Instances



Test Procedure				DC Specs		AC Specs		Sheet Parameters				
Test Name	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing	Overlay
Dyn_Icc_vb	VBT	DynIcc_HexVB		Commercial	vcc_nom	Commercial	max_AC	TimeSetsBasic		Levels		

The Dyn_Icc_vb test instance has the following test-specific parameters and values as shown in the Instance Editor:

Instance: Dyn_Icc_vb		
Parameters	Timing and Levels	Limits
Name	Value	
☐ DynIccPat	.\patterns\dynamic_Icc_pattern.PAT	...
power_pin	vcc	...
ForceVccValue	5	...
sample_size	19000	...
sample_rate	37000	...
meter_I_Range	1	...
settle_time	0.01	...
InitHi	portb	
InitLo	oe,dir	
InitHiZ	porta	



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxapps.5.15

	
---	---

HexVS Examples : HexVS Dynamic lcc Programming Using VBT : Patterns

Patterns

dynamic_lcc_pattern.atp Pattern File

This pattern file has been shortened in several places. The pattern contains 954 vectors.

digital_inst = hsdm;

opcode_mode = single;

import tset time_func;

```
srm_vector dyn_lcc      ( $tset  , oe, dir, porta  , portb  )
{
global dyn_lcc_start_glbv:
    > time_func 0 1 00000000 LLLLLLLL ;
branch_expr = (cpuA_cond) > time_func 0 1 00000001 LLLLLLLH ;
    > time_func 0 1 00000010 LLLLLLHL ;
set_cpu_cond (cpuA_cond) > time_func 0 1 00000011 LLLLLLHH ;
    > time_func 0 1 00000100 LLLLLHLL ;
vcc_max:
    > time_func 0 1 00000101 LLLLLHLH ;
    > time_func 0 1 00000110 LLLLLHHL ;
    > time_func 0 1 00000111 LLLLLHHH ;
... // More vectors
if(branch_expr) jump vcc_max > time_func 0 0 HLLLLLLL 11000000 ;
    set_cpu_cond (cpuA_cond) > time_func 0 1 00000011 LLLLLLHH ;
    > time_func 0 1 00000100 LLLLLHLL ;
vcc_nom:
    > time_func 0 1 00000101 LLLLLHLH ;
```

```
> time_func 0 1 00000110 LLLLLHHL ;
> time_func 0 1 00000111 LLLLLHHH ;
```

... // Even more vectors

```
if(branch_expr) jump vcc_nom > time_func 0 0 HHLLLLLL 11000000 ;
  set_cpu_cond (cpuA_cond) > time_func 0 1 00000011 LLLLLLHH ;
    > time_func 0 1 00000100 LLLLLHLL ;
```

vcc_min:

```
> time_func 0 1 00000101 LLLLLHLH ;
> time_func 0 1 00000110 LLLLLHHL ;
> time_func 0 1 00000111 LLLLLHHH ;
```

... // Yes, still more vectors

```
if(branch_expr) jump vcc_min > time_func 0 0 HHLLLLLL 11000000 ;
  halt > time_func 0 0 HHHLLLLL 11100000 ;
}
```

The following shows the first 10 vectors of this pattern file in PatternTool.

dynamic_1cc_pattern.PAT (dyn_1cc - SRM)						
TSet	Command	Label	Vector/Cycle	o d i e r	+porta aaaaaaaa 12345678	+portb bbbbbbbb 12345678
time_func		global dyn_1cc_start_glbv:	0	0 1	00000000	LLLLLLLL
time_func	branch_expr = (cpuA_cond)		1	0 1	00000001	LLLLLLLLH
time_func			2	0 1	00000010	LLLLLLHL
time_func	set_cpu_cond (cpuA_cond)		3	0 1	00000011	LLLLLLHH
time_func			4	0 1	00000100	LLLLLHLL
time_func		vcc_min:	5	0 1	00000101	LLLLLHLH
time_func			6	0 1	00000110	LLLLLHHL
time_func			7	0 1	00000111	LLLLLHHH
time_func			8	0 1	00001000	LLLLHLLL
time_func			9	0 1	00001001	LLLLHLLH
time_func			10	0 1	00001010	LLLLHLHL

hxapps.5.16

	
---	---

HexVS Examples : [HexVS Dynamic Icc Programming Using VBT](#) : VBT Code

VBT Code

Option Explicit

Public Function Dynlcc_HexVB(DynlccPat As Pattern, power_pin As PinList, _
ForceVccValue As Double, sample_size As Long, _
sample_rate As Double, meter_I_Range As Double, _
settle_time As Double, InitHi As String, _
InitLo As String, InitHiZ As String)

Dim HexMeasure55 As New PinListData

Dim HexMeasure50 As New PinListData

Dim HexMeasure45 As New PinListData

On Error **GoTo** errHandler

' Apply HSD levels, Init States, and PowerSupply pin values.

' Connect all pins and load levels. Do not load timing (not needed).

' Set up initial state of pins as high, low, and Hi-Z.

Call TheHdw.Digital.ApplyLevelsTiming(True, True, False, tIPowered, _
InitHi, InitLo, InitHiZ)

If TheHdw.DCVS.Pins(power_pin).Connected = tIDCVSConnectForce **Then**

 Debug.Print "Force Connected"

End If

' Wait 10ms.

TheHdw.Wait (settle_time)

' Set sample size and sample rate.

TheHdw.DCVS.Pins(power_pin).Capture.SampleSize.Value = sample_size

TheHdw.DCVS.Pins(power_pin).Capture.SampleRate.Value = sample_rate

' Load the dynamic lcc pattern.

Call TheHdw.Patterns(DynlccPat).Load

' Start the dynamic lcc pattern burst.

Call TheHdw.Patterns(DynlccPat).Start

' Set up the DCVS conditions for the dynamic lcc test at a force value of 4.5V.

' PatGen is running, looping with branch_expr = cpuA_cond with DCVS main DAC = 4.5V.

With TheHdw.DCVS.Pins(power_pin)

.Gate = True

TheHdw.DCVS.Pins(power_pin).Meter.Mode = tlDCVSMeterCurrent

.Meter.CurrentRange.Value = meter_I_Range

.Voltage.Main.Value = (ForceVccValue + 0.5 * v)

End With

' Strobe the meter on the VCC pin and store it in defined PinListData variable.

HexMeasure55 = TheHdw.DCVS.Pins(power_pin).Meter.Read(tlStrobe, sample_size, _
sample_rate)

Call TheHdw.Digital.Patgen.Continue(None, cpuA)

' Set up the DCVS conditions for the dynamic lcc test at a force value of 5.0V.

' PatGen is running, looping with branch_expr = cpuA_cond with DCVS main DAC = 5.0V.

With TheHdw.DCVS.Pins(power_pin)

.Meter.Mode = tlDCVSMeterCurrent

.Meter.CurrentRange.Value = meter_I_Range

.Voltage.Main.Value = ForceVccValue

End With

' Strobe the meter on the VCC pin and store it in defined PinListData variable.

HexMeasure50 = TheHdw.DCVS.Pins(power_pin).Meter.Read(tlStrobe, sample_size, _
sample_rate)

Call TheHdw.Digital.Patgen.Continue(None, cpuA)

' Set up the DCVS conditions for the dynamic lcc test at a force value of 5.5V.

' PatGen is running, looping with branch_expr = cpuA_cond with DCVS main DAC = 5.5V.

With TheHdw.DCVS.Pins(power_pin)

.Meter.Mode = tlDCVSMeterCurrent

.Meter.CurrentRange.Value = meter_I_Range

.Voltage.Main.Value = (ForceVccValue - 0.5 * v)

End With

' Strobe the meter on the VCC pin and store it in defined PinListData variable.

HexMeasure45 = TheHdw.DCVS.Pins(power_pin).Meter.Read(tlStrobe, sample_size, _
sample_rate)

Call TheHdw.Digital.Patgen.Continue(None, cpuA)

' - - - DATALOG RESULTS - - -

TheExec.Flow.TestLimit ResultVal:=HexMeasure55, _
unit:=unitAmp, _
pinName:=power_pin, _
forceVal:=ForceVccValue + 0.5, _
forceUnit:=unitVolt, _
ForceResults:=tlForceFlow

TheExec.Flow.TestLimit ResultVal:=HexMeasure50, _
unit:=unitAmp, _
pinName:=power_pin, _
forceVal:=ForceVccValue, _
forceUnit:=unitVolt, _
ForceResults:=tlForceFlow

TheExec.Flow.TestLimit ResultVal:=HexMeasure45, _
unit:=unitAmp, _
pinName:=power_pin, _
forceVal:=ForceVccValue - 0.5, _
forceUnit:=unitVolt, _
ForceResults:=tlForceFlow

Exit Function

errHandler:
 If AbortTest Then Exit Function Else Resume Next
End Function

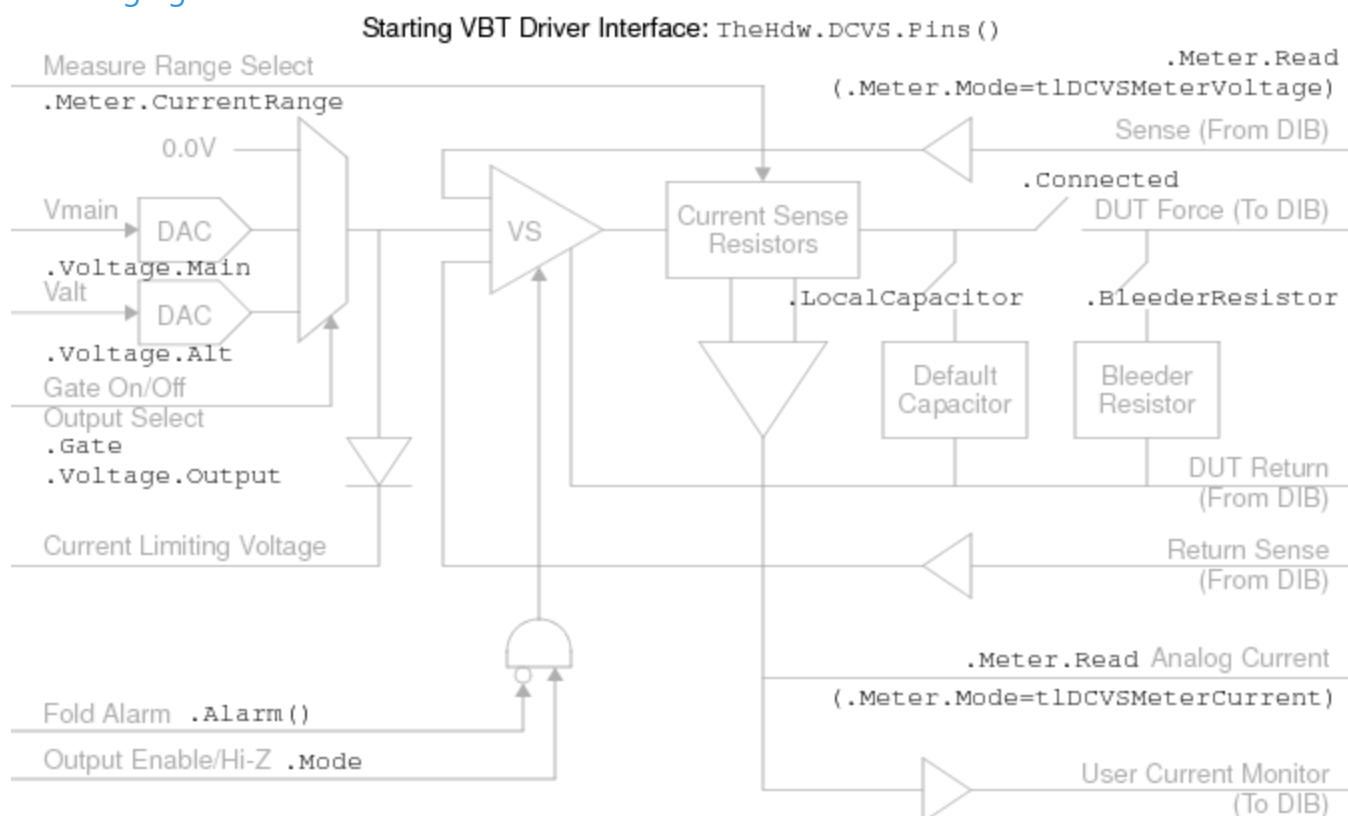
hxapps.5.17



HexVS Examples : Basic DCVS Setup Programming

Basic DCVS Setup Programming

The following example describes how to program the main output subsystem as shown in the following figure.



DCVS Instrument: Programming the Main Output Subsystem

Sub DCVS_BasicSetup()

With TheHdw.DCVS.Pins("VDD")

' Set the DCVS bandwidth to optimize for desired transient response.

```
.BandwidthSetting = 3
```

```
' Auto is the default setting for the bleeder resistor and local capacitor,
' but it may be turned on or off for some applications.
.BleederResistor = tIDCVSAuto
.LocalCapacitor = tIDCVSAuto
' You can force merged DCVS into a lower current output mode. The lower
' current output mode is necessary for tighter current limiting
' specifications and low current metering.
.CurrentRange.Value = .CurrentRange.Min
' Set the main and alt voltages
.Voltage.Main.Value = 2.5
.Voltage.Alt.Value = 2.1
' Set the output to Main DAC value. You can select Alt later using VBT or a
' pattern opcode
.Voltage.Output = tIDCVSVoltageMain
' Set the DCVS output mode to Voltage (not High Impedance)
.Mode = tIDCVSModeVoltage
' Connect the DCVS output (This connects both force and sense)
.Connect (tIDCVSConnectDefault)
' Gate on the DCVS
.Gate = True
End With
```

End Sub

You can use the following code to verify your force and sense connections.

With TheHdw.DCVS.Pins(...)

```
If .Connected = tIDCVSConnectForce Then
    debug.print "Force Connected"
End If
If .Connected = tIDCVSConnectSense Then
    debug.print "Sense Connected"
End If
If .Connected = tIDCVSConnectSense + tIDCVSConnectForce Then
    debug.print "Force and Sense Connected"
End If
If .Connected = tIDCVSConnectNone
    debug.print "No Connection"
End If
End With
```

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxapps.5.18

<	>
----------------------	----------------------

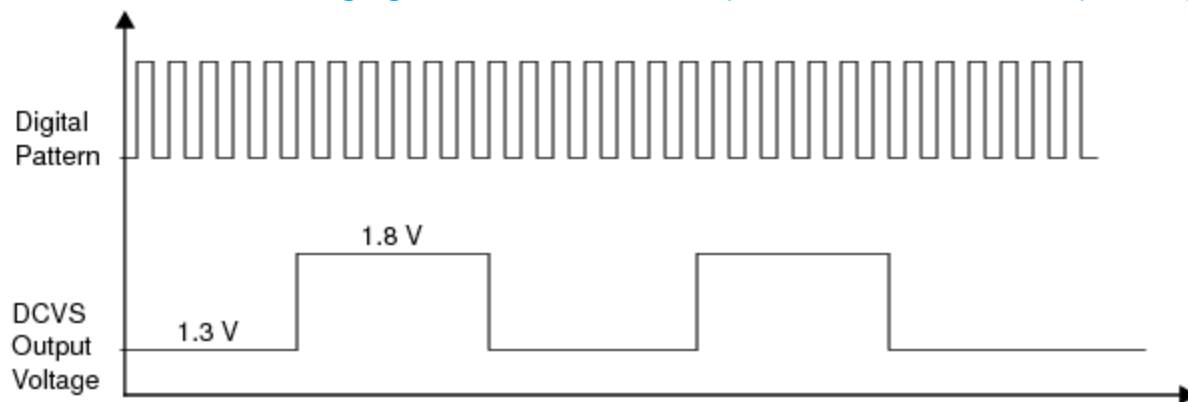
HexVS Examples : Vbump Testing

Vbump Testing

A Vbump test changes the device power pin (VDD) level during a test pattern burst. You may need to conduct a Vbump test to verify that a device is making smooth power transitions as it moves between different power states. For example, you could check a device's operation at two different voltage levels (1.3V and 1.8V).

To set up a Vbump test, the HexVS can change the voltage during a burst with Vmain, Valt, and parameter set programming. Note that the HexVS also uses the high-speed current meter to collect the Vbump test data during pattern bursts.

To give you pattern control for Vbump testing, the HexVS allows you to select one of two voltage levels during pattern bursts. Within a pattern burst, you can start and stop the current meter only as needed. The following figure models the HexVS pattern control for Vbump testing.



DCVS Voltage Pattern Control for Vbump Testing

Vbump Example Using VBT PSets and a Pattern File

The following example uses PSets and a pattern file with the VS_Alt_Sel and VS_Main_Sel microcodes to “bump” the output voltage on a pin named VDD between four different values in the following order:

1. [Start at 3.0V.](#)
2. [Vbump to 3.5V.](#)
3. [Vbump to 3.0V.](#)
4. [Vbump to 1.5V.](#)
5. [Vbump to 1.75V.](#)
6. [Vbump to 1.5V.](#)
7. [End at 1.5V.](#)

VBT PSet Programming Function

This example calls the [AddPset](#) function to create a PSet named "my_pset_1". This function is described in the [DCVS PSet Example Function](#). For more information on programming with PSets, see [PSets](#).

Function VBumpSetup(argc As Long, argv() As String) As Long

' Set voltage levels and define the PSet before pattern burst

' (For example, PrePat interpose function)

Dim power_supply_pin As String

' Set the power supply pin to VDD

power_supply_pin = "VDD"

TheHdw.DCVS.Pins(power_supply_pin).Voltage.Main.Value = 3

TheHdw.DCVS.Pins(power_supply_pin).Voltage.Alt.Value = 3.5

' Create "my_pset_1" using the AddPSet() function.

Call AddPSet(power_supply_pin, _

 PSetName:="my_pset_1", _

 BandwidthSetting:=0, _

 CurrentRange:=15, _

 SampleRate:=1000000, _

 SampleSize:=100000, _

 ConditionBitCaptureDone:=False, _

 ConditionBitLevelChanged:=False, _

 ConditionBitSourceTimeout:=False, _

```

ConditionBitSinkTimeout:=False, _
SourceFoldLimit:=16.5, _
MeterCurrentRange:=15, _
MeterVoltageRange:=4, _
MeterMode:=tlDCVSMeterCurrent, _
MainVoltage:=1.5, _
AltVoltage:=1.75)

```

```
VBumpSetup = TL_SUCCESS
```

End Function

Pattern File Example

The embedded comments in the following pattern file show where Vbump changes occur. For more information on programming pattern files, see [DCVS Pattern Control](#). For settle time guidelines, see [DCVS Pattern File Example](#).

```

import tset ts1;

instruments = {
    VDD : DCVS;
}
srm_vector ($tset, dig1:S)
{
    // __ THIS PATTERN STARTS WITH VDD VOLTAGE at 3.0V __
    > ts1 1;
    > ts1 1;
    //... many more vectors
    > ts1 1;
    > ts1 1;
    // __ VBUMP to 3.5 V __
    (vdd : DCVS = VS_Alt_Sel)
    > ts1 1;
    // settle time repeat. See guidelines in "DCVS Pattern File Example."
    repeat 5000
    > ts1 1;
    > ts1 1;
    //... many more vectors
    > ts1 1;
    > ts1 1;
    // __ VBUMP TO 3.0 V __
    (vdd : DCVS = VS_Main_Sel)

```

```
> ts1 1;
// settle time repeat. See guidelines in "DCVS Pattern File Example."
repeat 5000
> ts1 1;
> ts1 1;
//... many more vectors
> ts1 1;
// __ APPLY A PSET TO LOAD NEW VALUES OF MAIN AND ALT __
// __ THIS PSET CHANGES VDD TO 1.5 V __
(vdd : DCVS = PSet my_pset_1)
> ts1 1;
> ts1 1;
//... many more vectors
> ts1 1;
> ts1 1;
//... many more vectors
> ts1 1;
> ts1 1;
// __ VBUMP TO 1.75 V __
(vdd : DCVS = VS_Alt_Sel)
> ts1 1;
// settle time repeat. See guidelines in "DCVS Pattern File Example."
repeat 5000
> ts1 1;
> ts1 1;
//... many more vectors
> ts1 1;
> ts1 1;
// __ VBUMP TO 1.5 V __
(vdd : DCVS = VS_Main_Sel)
> ts1 1;
// settle time repeat. See guidelines in "DCVS Pattern File Example."
repeat 5000
> ts1 1;
> ts1 1;
> ts1 1;
//... many more vectors
```

```
> ts1 1;  
}
```

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxapps.5.19

	
---	---

HexVS Examples : Current Meter Setup, Strobe, and Readback Programming

Current Meter Setup, Strobe, and Readback Programming

The following example shows how to program the DCVS current meter to strobe the value of the current at a given time and store these current values in capture memory. It also shows how to read back data from the capture memory.

Dim PLD As New PinListData

Dim Pin As Variant

Dim Site As Variant

Dim PinList As String

PinList = "PinA,PinB"

With TheHdw.DCVS.Pins(PinList)

.CurrentRange.Value = 15

.Meter.Mode = tlDCVSMeterCurrent

.Meter.CurrentRange.Value = 1

End With

' To use High Speed 15A meter range, you must bypass the filter.

' Your code would look like this.

' With TheHdw.DCVS.Pins(PinList)

' .CurrentRange.Value = 15

' .Meter.Mode = tlDCVSMeterCurrent

' .Meter.CurrentRange.Value = 15

' .Meter.Filter.Bypass = True

' End With

' Strobe and read back 10 samples

PLD = TheHdw.DCVS.Pins(PinList).Meter.Read(tlStrobe, 10)

For Each Site In TheExec.Sites

For Each Pin In PLD.Pins

```
Debug.Print "The measured value on Site(" + CStr(Site) + _  
    ") of Pin (" + Pin + ") is " + CStr(PLD.Pins(Pin))
```

Next Pin

Next Site

		
	2015 Teradyne, Inc. All rights reserved.	

hxapps.5.20

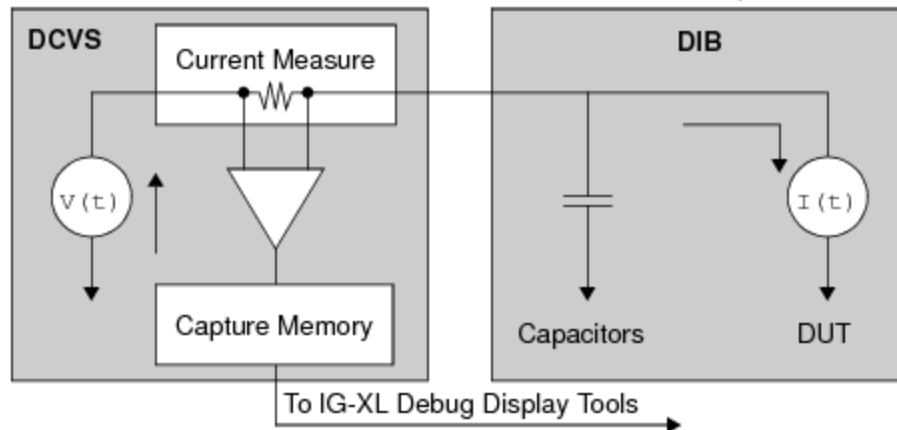


HexVS Examples : Current Profiling

Current Profiling

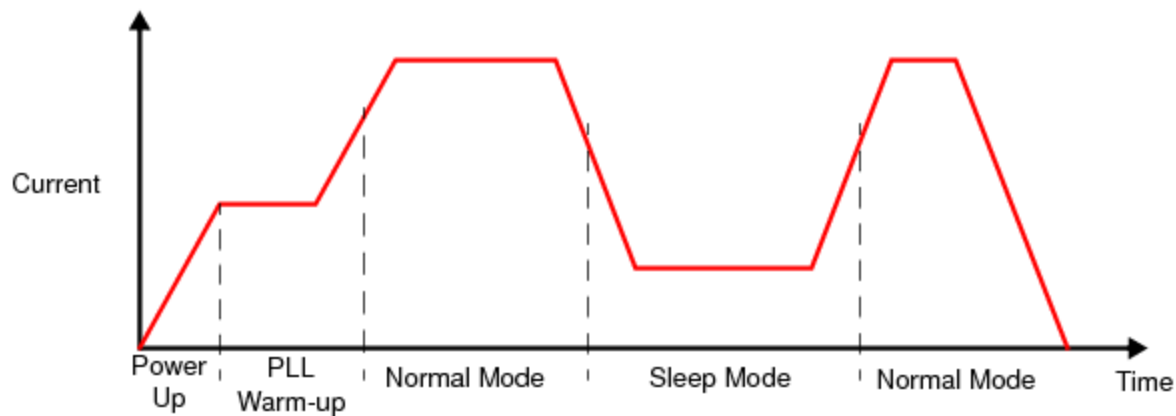
A current profile is a collection of data points of a device's current draw over time. Generally, you conduct a current profile to see what a tested part is doing as you cause it to pull more or less current. This allows you to monitor the current profile during the transition between device power states. For instance, you may want to verify that the device is drawing the expected current and is not experiencing current spikes as it moves between inactive and active modes.

The HexVS also allows you to collect a current profile with a single, continuous, full-speed PatGen burst. The data is sampled at the specified frequency and stored in the on-board capture memory. Note that each HexVS DCVS instrument has an independent 256K capture memory.



Current Profile Circuit Diagram

The figure below shows a hypothetical current profile generated from multiple test patterns run at speed to test a device's current requirements as it moves through different power states. The current changes as the device goes through the power-up sequence, the phase-locked loop (PLL) warm-up, and its normal and sleep modes.



Current Profile Sample

Capture Setup, Trigger, and Plot Programming

The following example shows how to capture a current or voltage waveform and plot the result.

```
Sub subProfile_Start(PlotName As String, _
    PinName As String, _
    Optional Vorl As tIDCVSMeterMode = tIDCVSMeterCurrent, _
    Optional SampleRate As Double = 25000, _
    Optional SampleSize As Long = 5000)
    Dim TestCaptureLabel As String
    If SampleRate < TheHdw.DCVS.Pins(PinName).Meter.SampleRate.Min Then
        SampleRate = TheHdw.DCVS.Pins(PinName).Meter.SampleRate.Min
    ElseIf SampleRate > TheHdw.DCVS.Pins(PinName).Meter.SampleRate.Max Then
        SampleRate = TheHdw.DCVS.Pins(PinName).Meter.SampleRate.Max
    End If
    If SampleSize < TheHdw.DCVS.Pins(PinName).Meter.SampleSize.Min Then
        SampleSize = TheHdw.DCVS.Pins(PinName).Meter.SampleSize.Min
    ElseIf SampleSize > TheHdw.DCVS.Pins(PinName).Meter.SampleSize.Max Then
        SampleSize = TheHdw.DCVS.Pins(PinName).Meter.SampleSize.Max
    End If
    TheHdw.DCVS.Pins(PinName).Meter.Mode = Vorl
    TestCaptureLabel = PlotName
    ' Wait if a capture is already running.
    Do While TheHdw.DCVS.Pins(PinName).Capture.IsRunning = True
        Loop
    TheHdw.DCVS.Pins(PinName).Capture.Signals.Add TestCaptureLabel
    TheHdw.DCVS.Pins(PinName).Capture.Signals.DefaultSignal = TestCaptureLabel
    With TheHdw.DCVS.Pins(PinName).Capture.Signals.Item(TestCaptureLabel)
        .Reinitialize
        .Mode.Mode = tSignalModeUseLoadedValue
    ' When calling .LoadSettings, do not override the .Meter.Mode.
```

```

' ____ NOTE on Signals.Item(<label>).<signal_parameter>.Mode ____
' The signal parameter mode is a LoadSettings Mode for the signal.
' It determines if the parameter is set to the Signal-defined value
' or remains untouched when the .LoadSettings method is called.
' This example uses the .Mode.Mode syntax, which determines the
' behavior of the .LoadSettings function for the parameter. In the
' example syntax above, .Mode.Mode, the first .Mode is the
' DCVS.Meter.Mode (voltage or current). The second .Mode is the signal
' LoadSettings mode. All signal parameters have a LoadSettings mode.
' For instance you can also program:
' .SampleRate.Mode = tlSignalModeUseLoadedValue ' Use value already ' loaded to
hardware. Do not override hardware.
' .SampleSize.Mode = tlSignalModeUseLoadedValue ' Use value already ' loaded to
hardware. Do not override hardware.
' The default value is tlSignalModeUseValue (Use the value defined for ' the signal; that is,
override current hardware.)
    .SampleRate = SampleRate
    .SampleSize = SampleSize
    .LoadSettings
    .Trigger
End With
End Sub
Sub subProfile_Plot(PlotName As String, _
    PinName As String, _
    Optional DumpToFile As Boolean = False)
    Dim TestCaptureLabel As String
    Dim DSPW As New DSPWave
    ' Wait if capture is still running.
    Do While TheHdw.DCVS.Pins(PinName).Capture.IsRunning = True
    Loop
    TestCaptureLabel = PlotName
    Dim FileName As String
    Dim strMsg As String
    ' Single-pin PinListData can be read directly into DSPWave
    DSPW = TheHdw.DCVS.Pins(PinName).Capture.Signals(TestCaptureLabel).DSPWave
    If DSPW.SampleSize > 0 Then
        DSPW.Plot TestCaptureLabel
        If DumpToFile Then

```

```
    FileName = "PlotData_" & TestCaptureLabel & ".txt"
    Call PlotToFile(FileName, DSPW)
End If
Else
    strMsg = "subProfile_Plot() Warning: No Samples Captured (CaptureLabel=" &
TestCaptureLabel & ")"
    TheExec.DataLog.WriteComment (strMsg)
    Beep
End If
Set DSPW = Nothing
End Sub
Sub PlotToFile(PlotFilePathAndName As String, PlotDSPW As DSPWave)
    Dim SamplePeriod As Double
    Dim SampleSize As Long
    Dim s As Long
    Dim FileNo As Integer
    Dim TempArray() As Double
    FileNo = 7
    Open PlotFilePathAndName For Output As #FileNo
    SampleSize = PlotDSPW.SampleSize
    SamplePeriod = (1 / PlotDSPW.SampleRate)
    TempArray = PlotDSPW.data
    For s = 0 To (SampleSize - 1)
        Print #FileNo, s * SamplePeriod & ", " & TempArray(s)
    Next s
    Erase TempArray
    Close #FileNo
End Sub
```

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hxapps.5.21

	
---	---

HexVS Examples : HexVS Kelvin Alarm Threshold Programming

HexVS Kelvin Alarm Threshold Programming

The HexVS board features variable Kelvin programming. You can program Kelvin thresholds to the DIB NVRAM (EEPROM). The specified values are loaded to HexVS boards during IG-XL initialization. If a user-specified threshold is not available, a default value of 250mV is used. The following example shows how to program the Kelvin sensing and alarm subsystem.

Sub ProgramDibEeprom_SetHexVsKelvinThresholds(TestHeadSlot As Long)

' Write HexVS Kelvin threshold settings to the DIB EEPROM to override the default
' threshold settings for the specified HexVS board.

Dim MyEEPROM As ExecDIBEEPROM

Dim EepromLabelPrefix As String

Set MyEEPROM = TheHdw.DIB.EEPROM

MyEEPROM.program

EepromLabelPrefix = "HexVS_" & TestHeadSlot & "_"

' Set up the Kelvin values

MyEEPROM.Record.Item(EepromLabelPrefix & "DibSenseKelvinAlarmThreshold")= 0.3

MyEEPROM.Record.Item(EepromLabelPrefix & "DutSenseKelvinAlarmThreshold")= 0.35

MyEEPROM.Record.WriteToHW

End Sub

Sub ProgramDibEeprom_DeleteHexVsKelvinThresholdSettings(TestHeadSlot As Long)

' When HexVS Kelvin threshold settings are deleted from the DIB EEPROM, default
' values will be used.

Dim MyEEPROM As ExecDIBEEPROM

Dim rec() As IDIB_EEPROM_RecordObj

Dim EepromLabelPrefix As String

Set MyEEPROM = TheHdw.DIB.EEPROM

```
EepromLabelPrefix = "HexVS_" & TestHeadSlot & "_"  
' Delete/Reset the Kelvin values  
MyEEPROM.Record.Delete EepromLabelPrefix & "DibSenseKelvinAlarmThreshold", rec  
MyEEPROM.Record.Delete EepromLabelPrefix & "DutSenseKelvinAlarmThreshold", rec  
MyEEPROM.Record.WriteToHW  
End Sub
```

hxapps.5.22

<	>
-----------------	-----------------

HexVS Examples : DCVS Current Range and Limit Programming

DCVS Current Range and Limit Programming

The following example shows how to set up current limiting programming using the following DCVS “shorthand” functions.

- `TheHdw.DCVS.Pins(PinList).CurrentLimit.Source.SetLimitLevels (SourceFoldLevel, SourceOverloadLevel)`
- `TheHdw.DCVS.Pins(PinList).CurrentLimit.Sink.SetLimitLevels (SinkFoldLevel, SinkOverloadLevel)`
- `TheHdw.DCVS.Pins(PinList).CurrentLimit.Source.SetLimitTimeouts (SourceFoldTimeout, SourceOverloadTimeout)`
- `TheHdw.DCVS.Pins(PinList).CurrentLimit.Sink.SetLimitTimeouts (SinkFoldTimeout, SinkOverloadTimeout)`
- `TheHdw.DCVS.Pins(PinList).SetCurrentRanges(OutputCurrentRange, MeterCurrentRange)`

These functions help make DCVS programming more efficient. Each of these functions takes the place of several lines of code that would otherwise be needed to check that dependent settings (like `CurrentRange` and `MeterCurrentRange`) are compatible with the target instrument hardware. In the case of current limit programming, these functions help replace about 12 lines of VBT code for each current limit (source and sink).

`Public Const UseExistingValue = -99`

```

Function DCVS_SetCurrentLimits(PinList As String, _
    Optional SourceFoldLevel As Double = UseExistingValue, _
    Optional SourceFoldTimeout As Double = UseExistingValue, _
    Optional SourceOverloadLevel As Double = UseExistingValue, _
    Optional SourceOverloadTimeout As Double = UseExistingValue, _
    Optional SourceOverloadEnabled As Boolean = False, _
    -
    Optional SinkFoldLevel As Double = UseExistingValue, _
    Optional SinkFoldTimeout As Double = UseExistingValue, _
    Optional SinkOverloadLevel As Double = UseExistingValue, _
    Optional SinkOverloadTimeout As Double = UseExistingValue, _
    Optional SinkOverloadEnabled As Boolean = False _
) As Long

```

' For VS Source Current Limits

```
With TheHdw.DCVS.Pins(PinList).CurrentLimit.Source
```

```

    If SourceFoldLevel = UseExistingValue Then SourceFoldLevel = _
        .FoldLimit.Level.Value
    If SourceFoldTimeout = UseExistingValue Then SourceFoldTimeout = _
        .FoldLimit.TimeOut.Value
    If SourceOverloadLevel = UseExistingValue Then SourceOverloadLevel = _
        .OverloadLimit.Level.Value
    If SourceOverloadTimeout = UseExistingValue Then SourceOverloadTimeout = _
        .OverloadLimit.TimeOut.Value

```

```
End With
```

```
TheHdw.DCVS.Pins(PinList).CurrentLimit.Source.SetLimitLevels SourceFoldLevel, _
    SourceOverloadLevel
```

```

If TheHdw.DCVS.Pins(PinList).Gate = False Then
    TheHdw.DCVS.Pins(PinList).CurrentLimit.Source.SetLimitTimeouts _
        SourceFoldTimeout, SourceOverloadTimeout
    TheHdw.DCVS.Pins(PinList).CurrentLimit.Source.OverloadLimit.Enabled = _
        SourceOverloadEnabled

```

```
End If
```

' For VS Sink CurrentLimits

```
With TheHdw.DCVS.Pins(PinList).CurrentLimit.Sink
```

```

    If SinkFoldLevel = UseExistingValue Then SinkFoldLevel = _
        .FoldLimit.Level.Value
    If SinkFoldTimeout = UseExistingValue Then SinkFoldTimeout = _
        .FoldLimit.TimeOut.Value

```

```

    If SinkOverloadLevel = UseExistingValue Then SinkOverloadLevel = _
        .OverloadLimit.Level.Value
    If SinkOverloadTimeout = UseExistingValue Then SinkOverloadTimeout = _
        .OverloadLimit.TimeOut.Value
End With
TheHdw.DCVS.Pins(PinList).CurrentLimit.Sink.SetLimitLevels SinkFoldLevel, _
    SinkOverloadLevel
If TheHdw.DCVS.Pins(PinList).Gate = False Then
    TheHdw.DCVS.Pins(PinList).CurrentLimit.Sink.SetLimitTimeouts _
        SinkFoldTimeout, SinkOverloadTimeout
    TheHdw.DCVS.Pins(PinList).CurrentLimit.Sink.OverloadLimit.Enabled = _
        SinkOverloadEnabled
End If
End Function
Function DCVS_SetCurrentRanges(PinList As String, _
    Optional OutputCurrentRange As Double = UseExistingValue, _
    Optional MeterCurrentRange As Double = UseExistingValue _
    ) As Long
If OutputCurrentRange = UseExistingValue Then OutputCurrentRange = _
    TheHdw.DCVS.Pins(PinList).CurrentRange.Value
If MeterCurrentRange = UseExistingValue Then MeterCurrentRange = _
    TheHdw.DCVS.Pins(PinList).Meter.CurrentRange.Value
    TheHdw.DCVS.Pins(PinList).SetCurrentRanges OutputCurrentRange, MeterCurrentRange
End Function
```

hxdebug.4.1

<	>
-----------------	-----------------

DCVS and HexVS Debug Displays

DCVS and HexVS Debug Displays

This section describes both the DCVS instrument and the HexVS board debug displays:

- [DCVS Debug Display](#)
- [HexVS Board Debug Display](#)

For more information on using debug displays, see the following:

- [Using Debug Displays in TDE](#)
- [Producing Code from Debug Displays](#)

<	>
-----------------	-----------------

hxdebug.4.2

<	>
-----------------	-----------------

DCVS and HexVS Debug Displays : DCVS Debug Display

DCVS Debug Display

The interactive DCVS debug display allows you to troubleshoot DCVS settings. You can change any values on the display and run your test program to see the effects of the change. This section includes the following topics:

- Starting the DCVS Debug Display
- DCVS Debug Display Interface
- Setting the Output Current Range and Meter Ranges

Starting the DCVS Debug Display

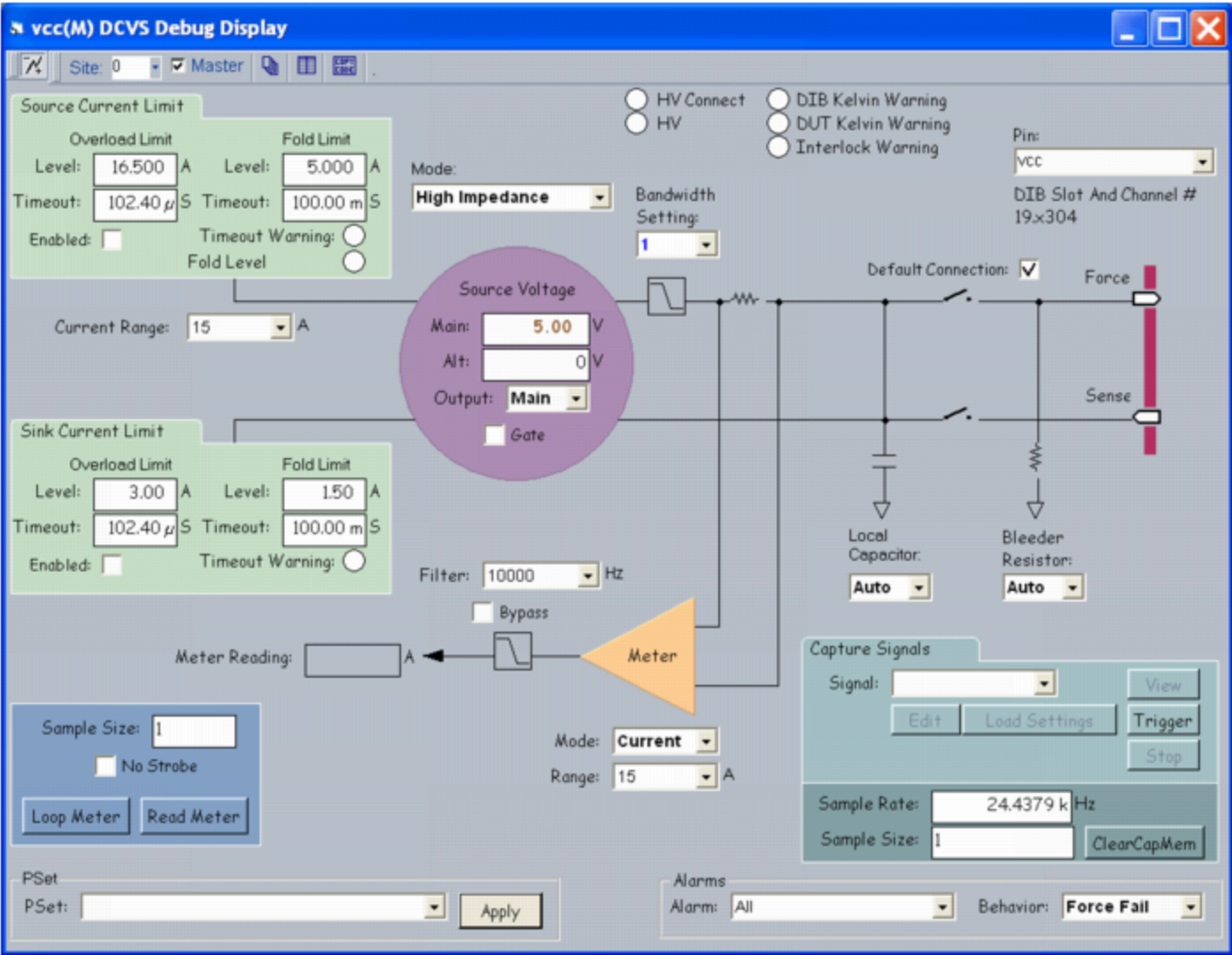
To start the DCVS debug display for HexVS, validate a HexVS test program. Then, on the IG-XL Tools tab, in the Launch group, select Debug Displays>DCVS. This opens the display for active sites only (see figure below).

If you start a debug display before the job has been validated, an error appears in the display saying that the program has not been validated. If you open a display for an instrument that is in the configuration file but not in the channel map, the display shows a message that no instruments have been found.

Once the job is validated, the display shows the current hardware settings. When you run to a breakpoint in the flow or in the VBT code, or as you step through the flow or code, IG-XL updates the display to show any changes to the settings.

For information on other methods for starting debug displays, see Starting Debug Displays in the Test Debug Environment (TDE) section.

For information on the fields used in the DCVS debug display, see DCVS Debug Display Interface.



DCVS Debug Display with HexVS Values

DCVS Debug Display Interface

This section describes the parameters, buttons, and fields used in the DCVS debug display. To change values in the debug display, either type the new values in the text box or select the new value from the menu.

Note:

To view the ranges or available options for fields that do not have drop-down menus, place your pointer over the field and the minimum and maximum values appear in a pop-up window. These ranges vary depending on which VS board you have assigned to the DCVS channel and how you have programmed it.

The table lists DCVS Debug Display parameters, buttons, and fields used by the HexVS.

DCVS Debug Display Fields for HexVS

Menu, Button, or Field	Action
------------------------	--------

Source Current Limit

Use the following fields to view or set your source current limiting hardware:

- **Overload Limit:** The overload current level limit in the Level field and the timeout period in the Timeout field.
- **Fold Limit:** The fold current level limit in the Level field and the timeout period in the Timeout field.
- **Enabled:** Checkbox enables the source overload current limiting.
- **Timeout Warning:** Red circle indicates the fold timeout has expired.
- **Fold Level Changed:** Yellow circle indicates the hardware has changed the source fold current limit. This happens when the HexVS power management hardware enforces power compliance and shared tracking supply characteristics. See [Output Power](#).

[See Setting the Current Limit.](#)

Sink Current Limit

Use the following fields to view or set your source current limiting hardware:

- **Overload Limit:** The overload current level limit in the Level field and the timeout period in the Timeout field.
- **Fold Limit:** The fold current level limit in the Level field and the timeout period in the Timeout field.
- **Enabled:** Checkbox enables the sink overload current limiting.
- **Timeout Warning:** Red circle indicates the fold timeout has expired.

See [Setting the Current Limit](#).

Source Voltage

Use the following fields to set up your output voltage:

- **Main:** The main source voltage value in volts.
- **Alt:** The alternate source voltage value in volts.
- **Output:** The output for the source voltage, either Main or Alt.
- **Gate:** The DCVS is gated on (checked) or off (empty).

Mode

The DCVS operation mode. Options:

- **Force Voltage** (On)
- **High Impedance** (Off)

Pin

The pin you are observing. The DIB slot and channel number you are currently viewing are shown below this field.

Bandwidth Setting

The compensation mode. Options: 0 through 7. See [Setting the Output Compensation Bandwidth](#).

Current Range	The output current range you are forcing. See Setting the Output Current Range and Meter Ranges .
Default Connection	Checkbox determines how the force and sense connections are opened and closed. When checked, these connections are opened and closed together. When unchecked, force and sense connections are controlled independently.
Local Capacitor	The connection state of the local output capacitor. When the DCVS is disconnected from the DIB, the default capacitor keeps the DCVS voltage output from oscillating. Options: • On • Off • Auto See Local Output Capacitor and Bleeder Resistor.
Bleeder Resistor	The connection state of the bleeder resistor. When the DCVS is disconnected from the DIB, and the bleeder resistor helps quickly discharge DIB capacitance. Options: • On • Off • Auto See Local Output Capacitor and Bleeder Resistor.
Meter	The DCVS meter settings, including: • Mode: The meter mode, either Current or Voltage. • Range: The meter range. See Setting the Output Current Range and Meter Ranges.

- [Filter](#): The meter bandwidth (for viewing only).
- [Bypass](#): Checking this box disables filtering, allowing you to view a higher frequency signal with more stability.
- [Meter Reading](#): The measured voltage or current value.
- [Sample Size](#): The number of samples strobed.
- [No Strobe](#): Checking this box disables the strobe function.
- [Loop Meter](#): Clicking this button opens the Meter Loop Tool and causes the meter to loop through multiple strobe sets. See [Meter Loop Tool](#).
- [Read Meter](#): Clicking this button prints the average of the strobed samples.

PSet

A list of defined PSets for selected pin site. When no PSet is defined, the list is empty. When a PSet is selected, the PSet is applied to instrument hardware immediately. The box is updated to display any PSet that is applied from VBT.

Alarms

The alarm type and behavior for the selected alarm.

- [Alarm](#): The alarm type.
- [Behavior](#): The action that the alarm takes when triggered.

See [Kelvin Sensing and Alarms Subsystem](#) and [Setting Alarms](#).

Capture Signals

The capture memory signal settings, including:

- [Signal](#): A capture signal tag listed in the drop-down menu.

- [Load Settings](#): Clicking this button loads the capture signal tag's settings.
- [Edit](#): Brings up the PSet and Signals Editor (PSSE) with the selected signal loaded. See [PSet and Signals Editor](#).
- [View](#): Clicking this button shows the selected capture signal tag's waveform.
- [Trigger](#): Clicking this button triggers new capture signal with the current hardware setup.
- [Stop](#): Clicking this button stops the current capture, if one is running.
- [Sample Rate](#): The capture rate currently set in hardware.
- [Sample Size](#): The capture sample size currently set in hardware.
- [ClearCapMem](#): Clicking this button clears both the signal (capture) and meter memories for channels behind the specified pin or pins.

DIB Kelvin Warning	A red circle indicates that the DIB Kelvin threshold has been exceeded.
DUT Kelvin Warning	A red circle indicates that the DUT Kelvin threshold has been exceeded.
Interlock Warning	A red circle indicates that either the DIB is not seated correctly on the test head or a connection problem exists in the DCVS interlock circuit.
HV Gate	A red circle indicates that the high voltage gate (HV_GATE) safety signal from the support board is not connected properly to DIB ground. This usually means that the DIB is either not properly seated or designed incorrectly.

HV Connect

A red circle indicates that the high voltage connect (HV_CONNECT) safety signal from the support board is not connected properly to DIB ground. This usually means that the DIB is either not properly seated or designed incorrectly.

Setting the Output Current Range and Meter Ranges

The available output current ranges and corresponding meter ranges depend on whether you have merged your DCVS channels or left them independent. For nonmerged DCVS channel, the only valid option is 15A. However, for merged DCVS channels, you have the choice between using the 15A option or the merged range option (30A, 60A, or 90A). See the following table for complete description of the valid options.

Valid Output Current and Meter Ranges

Output Current Range	Valid Meter Ranges
15A	15A, 1A, 100mA, 10mA
30A	30A
60A	60A
90A	90A

<

>

2015 Teradyne, Inc. All rights reserved.

hxdebug.4.3

<

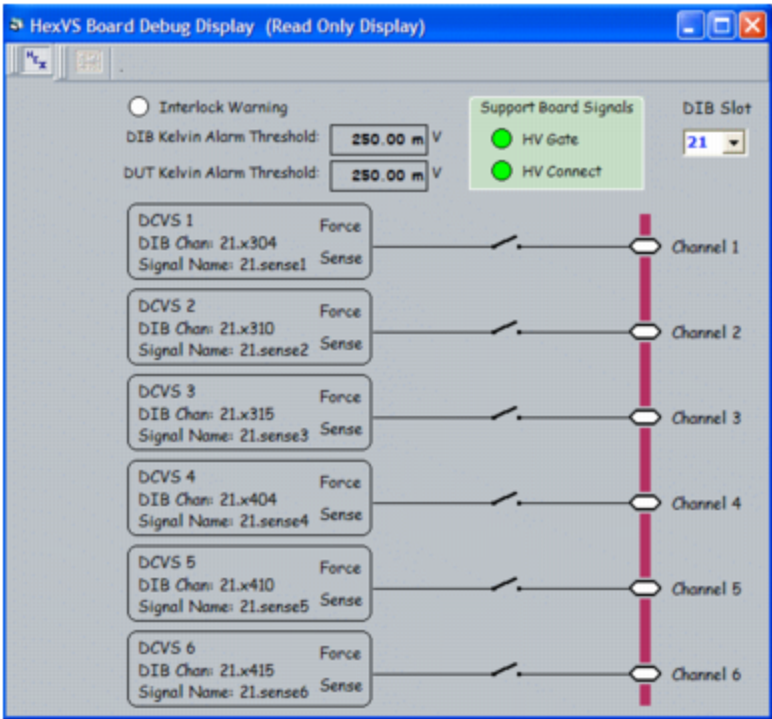
>

DCVS and HexVS Debug Displays : HexVS Board Debug Display

HexVS Board Debug Display

This read-only debug display allows you to check how the six DCVS instruments are connected to the DIB. It also shows merged DCVS instruments.

To bring up the HexVS Board Debug Display, on the IG-XL Tools tab, in the Launch group, select Subsystems>HexVS Board. This brings up the HexVS debug display shown in the following figure.



HexVS Board Debug Display

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxprog.3.01

<

>

HexVS DCVS Programming

HexVS DCVS Programming

This section describes programming of a HexVS board’s DCVS instruments.
The following topics are available:

- [Simulated Configuration Files](#)
- [Programming Procedure Overview](#)
- [Before You Begin](#)
- [DCVS DataTool Programming](#)
- [DCVS VBT Programming](#)
- [DCVS Pattern Control](#)
- [Using HexVS in Concurrent Testing](#)

For more information, see:

- [DCVS \(VBT Syntax Reference\)](#)
- [HexVS Examples](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxprog.3.02

<

>

HexVS DCVS Programming : Simulated Configuration Files

Simulated Configuration Files

You can use the HexVS offline by forcing IG-XL to map the HexVS. Do this by creating a SimulatedConfig.txt file and placing the file in any of the following locations:

- Folder containing the IG-XL workbook
- \tester folder: C:\Program Files (x86)Teradyne\IG-XL\<version>\tester
- \bin folder: C:\Program Files (x86)\Teradyne\IG-XL\<version>\bin

The following SimulatedConfig.txt file maps the HexVS board in slot 12:

<Version>

1

</Version>

<TEST_HEAD TH 1>

#slot[.subslot]	Type	idprom (type serial rev company)
-----------------	------	----------------------------------

4.0	HSD-M	979-219-00 0000000 0000-A 0000
	HSD-M	956-361-00 0000000 0000-A 0000

12.0	HexVS	974-294-00 0000000 0000-A 0000
	HexVS	956-388-00 0000000 0000-A 0000

24.0	SupportBoard	974-220-12 0000000 0000-A 0000
------	--------------	--------------------------------

SupportBoard 956-354-00 0000000 0000-A 0000

62.0 DistributionBoard 956-355-00 0000000 0000-A 0000
DistributionBoard 956-355-00 0000000 0000-A 0000

</TEST_HEAD TH 1>

To use the [HexVS offline](#), the SimulatedConfig.txt file must include every board needed in the IG-XL workbook. The SimulatedConfig.txt file represents the state of the offline tester. To use the HexVS online, you need not edit any files. IG-XL automatically maps all boards in the tester.
[For more information, see Tester Configuration.](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxprog.3.03

<	>
-----------------	-----------------

HexVS DCVS Programming : Programming Procedure Overview

Programming Procedure Overview

To develop a test program that uses the DCVS instrument, follow this basic procedure:

1. Determine device power supply requirements (Device Power Supply Requirements).

- a. Number of power pins
- b. Output voltage and current ranges for each power pin
- c. Maximum current limiting requirements

2. Consider the following features specific to the DCVS instrument (DCVS Instrument Considerations):

- a. Instrument board-specific DIB design considerations such as thermal control unit connections for output current monitoring.
- b. Output power compliance
- c. Programming shared tracking supply pairs
- d. Merging DCVS instrument supplies

3. [Create a Pin Map sheet \(Programming the Pin Map\).](#)
4. [Create a Channel Map sheet \(Programming the Channel Map\).](#)
5. [Create a Pin Levels sheet \(Programming Pin Levels\).](#)
6. [Set the maximum current limits using VBT code \(Setting the Current Limit\).](#)
7. [Set the compensation mode using VBT code \(Setting the Output Compensation Bandwidth\).](#)
8. [Set the alarms using VBT code \(Setting Alarms\).](#)
9. [Set the DCVS meter to measure current using VBT code \(Setting the DCVS Current Meter\).](#)
10. [Create DCVS test patterns to use in your test instances \(DCVS Pattern Control\).](#)
11. [Create test instances in the Test Instances sheet \(DCVS Test Instance Programming\).](#)
12. [Add test instances to the Flow Table sheet.](#)
13. [Run the test program, debug run-time errors or alarms, and optimize for run-time performance.](#)

For more information, see [DCVS](#) in the VBT Syntax Reference.



hxprog.3.04

<	>
-----------------	-----------------

HexVS DCVS Programming : Before You Begin

Before You Begin

Before writing a DCVS test program, you must determine the power requirements for your device application, including the number of power pins, the output voltage and current ranges for each power pin, and the maximum current limiting requirements. Moreover, you must understand the features of the DCVS instrument and how they affect your test program development. This section covers both of these topics:

- | | |
|---|----------------------------------|
| • | Device Power Supply Requirements |
| • | DCVS Instrument Considerations |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxprog.3.05

<	>
-----------------	-----------------

HexVS DCVS Programming : Before You Begin : Device Power Supply Requirements

Device Power Supply Requirements

Review the device datasheet to determine the device’s power pin requirements. Note, however, that during testing, the device power requirements are likely to exceed the standard functional values listed in the datasheet. When possible, therefore, consult the device product engineer to get an idea of the predicted device power requirements during testing. In general, new generation digital VLSI devices tend to draw more power supply current during testing than previous generation devices.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxprog.3.06

	
---	---

[HexVS DCVS Programming : Before You Begin](#) : DCVS Instrument Considerations

DCVS Instrument Considerations

Consider the following before creating a DCVS test program.

DCVS Instrument DIB Design Guidelines

For information on DIBs, including DIB signals and connections and wiring guidelines, see [HexVS DIB Design Guidelines](#).

Output Power Compliance

Make sure you understand the output power specifications of the HexVS board and the DCVS instrument before setting up your test program.

For more information, see [Output Power Compliance](#).

Programming Shared Tracking Supply Pairs

Be careful when assigning supplies to device pins that you will program to different voltages during testing. This is not a concern for pins that are connected to merged supplies.

Note:

Due to the HexVS shared tracking supply characteristics, test programmers must help assign HexVS channels before the DIB is designed and fabricated.

For more information on the programming implications of shared tracking supply pairs, see [Shared Tracking Supply Pair Considerations](#) and [Assigning Channels to Nonmerged Adjacent DCVS Pairs](#) in the DIB design wiring guidelines section.

Merging DCVS Instrument Supplies

For the most part, programming merged DCVS instruments is similar to programming a single DCVS. The primary difference is that the test program channel map indicates that the DCVS instrument is in a DCVSMerged2, DCVSMerged4, or DCVSMerged6 configuration. Beyond that, the only other difference is that the maximum source and sink current ranges and limit settings have increased by a factor proportional to the number of merged DCVS instruments. Voltage ranges,

timer ranges, capture sample rates, and most other instrument parameters have the same settings as the nonmerged leader channel.

For more information, see [Merged Mode](#) and [Assigning HexVS Channels for Merged DCVS Outputs](#) in the DIB design wiring guidelines section.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

hxprog.3.07

<	>
-----------------	-----------------

HexVS DCVS Programming : DCVS DataTool Programming

DCVS DataTool Programming

This section describes how to program several DataTool worksheets to handle basic DCVS parameters. It includes the following topics:

- Programming the Pin Map
- Programming the Channel Map
- Programming Pin Levels
- DCVS Test Instance Programming

<	>
-----------------	-----------------

hxprog.3.08

<	>
-----------------	-----------------

HexVS DCVS Programming : [DCVS DataTool Programming](#) : Programming the Pin Map

Programming the Pin Map

A pin map describes the device pins by assigning a pin type to each device pin name. Select the Power pin type for each device pin name connected to a DCVS instrument.
[For more information, see Pin Map Sheet](#) in the DataTool section.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxprog.3.09

<

>

HexVS DCVS Programming : DCVS DataTool Programming : Programming the Channel Map

Programming the Channel Map

A Channel Map sheet assigns individual device pin names to test system channels and power supplies. For the DCVS instrument, you must select one of the four channel types from the Type column menu for each pin or pin group. The following table shows the available DCVS instrument channel type options.

DCVS Instrument Channel Type Options	
Option	Explanation
DCVS	Standard connection to a single DCVS instrument on one HexVS board. Provides up to 15A of output current.
DCVSMerged2	Indicates a DIB wiring connection that merges two DCVS instruments on one HexVS board to provide up to 30A of output current.
DCVSMerged4	Indicates a DIB wiring connection that merges four DCVS instruments on one HexVS board to provide up to 60A of output current.
DCVSMerged6	Indicates a DIB wiring connection that merges six DCVS instruments on one HexVS board to provide up to 90A of output current.

When assigning channels for a specific site, use the following channel map signal name format:
<slot_number>.sense<leader_vs_number>
where:

<slot_number>	DIB slot where the HexVS board is installed
---------------	---

<leader_vs_number> DCVS leader channel number

For more information on merging DCVS instruments, see:

- [Channel Map Sheet](#)
- [Merged Mode](#)
- [Assigning HexVS Channels for Merged DCVS Outputs](#)

<

>

2015 Teradyne, Inc. All rights reserved.

hxprog.3.10

<

>

HexVS DCVS Programming : DCVS DataTool Programming : Programming Pin Levels

Programming Pin Levels

A Pin Levels sheet defines values for input and output voltages and current loads to be applied to the DUT. Once you insert this sheet into a test, you can program the Power Supply Test Instance Editor, which uses values assigned in this sheet.

The DCVS instrument supports incremental levels, which allows you to change levels for an individual pin or a pin group that has already been defined on the Pin Levels sheet. IG-XL applies the last set of specified levels for a pin or pin group when adding a new pin or pin group, applying and sequencing levels, characterizing pins, and determining specification dependencies. With this flexible programming model, you can reassign new pin levels to existing pins on the sheet while preserving previous pin level setups. You can always revert to previous levels settings and behavior by removing the most recent pin level change on the sheet. For more information, see Incremental Levels Mode.

For the DCVS instrument, you must assign a pin levels parameter for each pin name or group. The following table shows the HexVS DCVS instrument pin level parameter options.

HexVS DCVS Instrument Pin Levels Parameter Options

Option	Explanation
VMain	Main voltage power supply. Used to supply voltage to the DUT through the main DAC. Range: 0.0V to 5.5V. Replaces the Vps parameter in IG-XL V6.10.00 and later.
VAlt	Alternate voltage power supply. Used to supply voltage to the DUT through the alternate DAC. Range: 0.0V to 5.5V.
IFoldLevel	The fold current limit level. Ranges: N x (50mA to 15.5A), where N is the number of merged supplies. Default: N x 5.0A.

	Replaces the <code>Isc</code> parameter in IG-XL V6.10.00 and later. See Setting the Current Limit.
<code>IOverloadLevel</code>	The overload current limit level. Optional. Ranges: $N \times (0.3A \text{ to } 16.0A)$, where N is the number of merged supplies. Default: $N \times 16.5A$. Replaces the <code>Isc</code> parameter in IG-XL V6.10.00 and later. See Setting the Current Limit.
<code>Tdelay</code>	Time delay. Used to define the delay time after the application of the supply voltage. By programming <code>Tdelay</code>, you can wait some amount of time after applying device power. <code>Tdelay</code> is not used during the power-down sequence. It is only used after powering-up all power pins of the same sequence number. If you place more than one DCVS pin in the same sequence group on a Pin Levels sheet, then IG-XL applies power to all of those pins at once and uses the maximum of all <code>Tdelay</code> values for these pins. IG-XL automatically enforces a 1.0ms standard delay time on a HexVS channel for a Pin Levels sheet change. If this time is insufficient, you can program the <code>Tdelay</code> parameter to add to this standard delay time. For example, if you program <code>Tdelay</code> as 3ms, the actual delay time will be 4ms. Note also that IG-XL uses only the longest programmed delay time for channels of the same sequence number because it executes time delays from all instrument boards in parallel.
<code>Vps</code>	Voltage power supply. Used to supply voltage to the DUT. Range: 0.0V to 5.5V. This parameter is used in IG-XL V6.00.08 and earlier. Use <code>VMain</code> and <code>VAlt</code> to set voltage levels in V6.10.00 and later.
<code>Isc</code>	Current source clamp. Used to set the current limits (specifically, the fold limit level) for the DCVS. Two current limiting modes: single and dual. Ranges: $N \times (50mA \text{ to } 15.5A)$, where N is the number of merged supplies. Default: $N \times 15.5A$. This parameter is used in IG-XL V6.00.08 and earlier. Use <code>IFoldLimit</code> and <code>IOverloadLimit</code> to set current limit levels in V6.10.00 and later. See Setting the Current Limit.

For more information, see [Pin Levels Sheet](#) in the DataTool section.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxprog.3.11

<	>
-----------------	-----------------

[HexVS DCVS Programming](#) : [DCVS DataTool Programming](#) : [DCVS Test Instance Programming](#)

DCVS Test Instance Programming

You can use the instance editor to create and edit DCVSPowerSupply_T VBT test instances. A test instance defines a test that is applied to the DUT when the test program runs. You can create as many test instances as you want. The Test Instances sheet lists all test instances for your test program.

Before creating a DCVS test instance, you must declare a Power pin type in the pin map. You also need a Pin Levels sheet. Finally, you must connect the Power pin to a DCVS channel type.

To create or edit a DCVSPowerSupply_T test instance in the instance editor, see [Creating and Editing Test Instances](#). Once you have created a test instance, program the DCVSPowerSupply_T test instance by filling in the parameter fields. For a reference to the DCVSPowerSupply_T test parameters, see [DCVS Power Supply VBT Test](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxprog.3.12

<	>
-----------------	-----------------

HexVS DCVS Programming : DCVS VBT Programming

DCVS VBT Programming

Several basic DCVS programming tasks require that you use VBT syntax.

- | | |
|---|---|
| • | Setting the Current Limit |
| • | Setting the Output Compensation Bandwidth |
| • | Setting Alarms |
| • | Setting the DCVS Current Meter |

For more information, see [DCVS](#) in the VBT Syntax Reference.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxprog.3.13

<	>
---	---

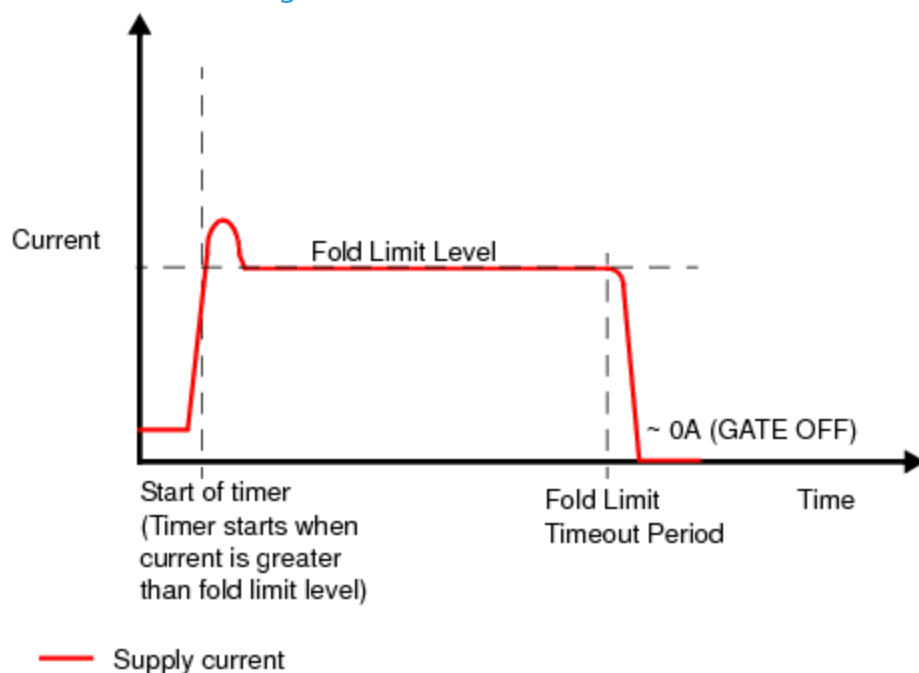
HexVS DCVS Programming : DCVS VBT Programming : Setting the Current Limit

Setting the Current Limit

The DCVS instrument has two current limiting modes: single-level and dual-level. The default setting is single-level current limiting where overload limiting is disabled. You must decide which one is best to use based on your device's power supply current limiting requirements. For an example of how to program current range and limits using DCVS "shorthand" functions, see [DCVS Current Range and Limit Programming](#).

Single-Level Current Limiting (Overload Limit Disabled)

The hardware limits the source or sink current to a programmed value until the user-defined fold time expires or the current drops below the limit level value. The following figure shows this single-level current limiting.



Single-Level Current Limiting

If the current is greater than the fold limit level, the fold limit timer starts, and the current is immediately limited to the fold limit level. The timer resets if the current drops below the fold limit level. If the timer is not reset and the fold limit timeout period expires, however, the hardware forces the DCVS output to gate off.

Optionally, you can program the output to stay gated on for as long as possible after the fold limit timeout period has elapsed. However, for safety reasons, the DCVS still gates off eventually (in approximately 10 seconds), even if programmed to remain on.

After determining the desired source and sink current limit types, you can program the overload and fold limit levels and timeout values. To activate single-level mode, you must disable overload limiting.

Use the DCVS Pin Levels sheet to set up the source current limit defaults:

•	Overload limiting: Disabled
•	Fold limit level: Isc or IFoldLevel level specified in the Pin Levels sheet
•	Fold limit timeout value: 0.010 seconds
•	Fold limit behavior: Gate Off

This VBT syntax example shows how to program the single-level source current limit:

With TheHdw.DCVS.Pins(PinList)

```
.Gate = False ' You MUST turn off gate to:
    ' a) Enable or Disable the Overload Limiting
    ' OR
    ' b) Change Fold or Overload Limit Timeout values
    ' otherwise a run-time error will occur.
```

With .CurrentLimit.Source

```
.OverloadLimit.Enabled = False
.FoldLimit.Level.Value = (0.65 * .FoldLimit.Level.Max)
.FoldLimit.Timeout.Value = 0.010
.FoldLimit.Behavior = tIDCVSCurrentLimitBehaviorGateOff
```

End With

End With

The sink current limit defaults cannot be set up in the Pin Levels sheet. These are the sink defaults programmed during system initialization:

- [Overload limiting: Disabled](#)
- [Fold limit level: 1.5A](#)
- [Fold limit timeout value: 0.010 seconds](#)
- [Fold limit behavior: Gate Off](#)

To change the sink current levels, you can use VBT syntax as shown in this example:

With TheHdw.DCVS.Pins(PinList)

```
.Gate = False ' You MUST turn off gate to:
    ' a) Enable or Disable the Overload Limiting
    ' OR
    ' b) Change Fold or Overload Limit Timeout values
    ' otherwise a run-time error will occur.
```

With .CurrentLimit.Sink

```
.OverloadLimit.Enabled = False
.FoldLimit.Level.Value = (0.65 * .FoldLimit.Level.Max)
.FoldLimit.Timeout.Value = 0.010
.FoldLimit.Behavior = tlDCVSCurrentLimitBehaviorGateOff
```

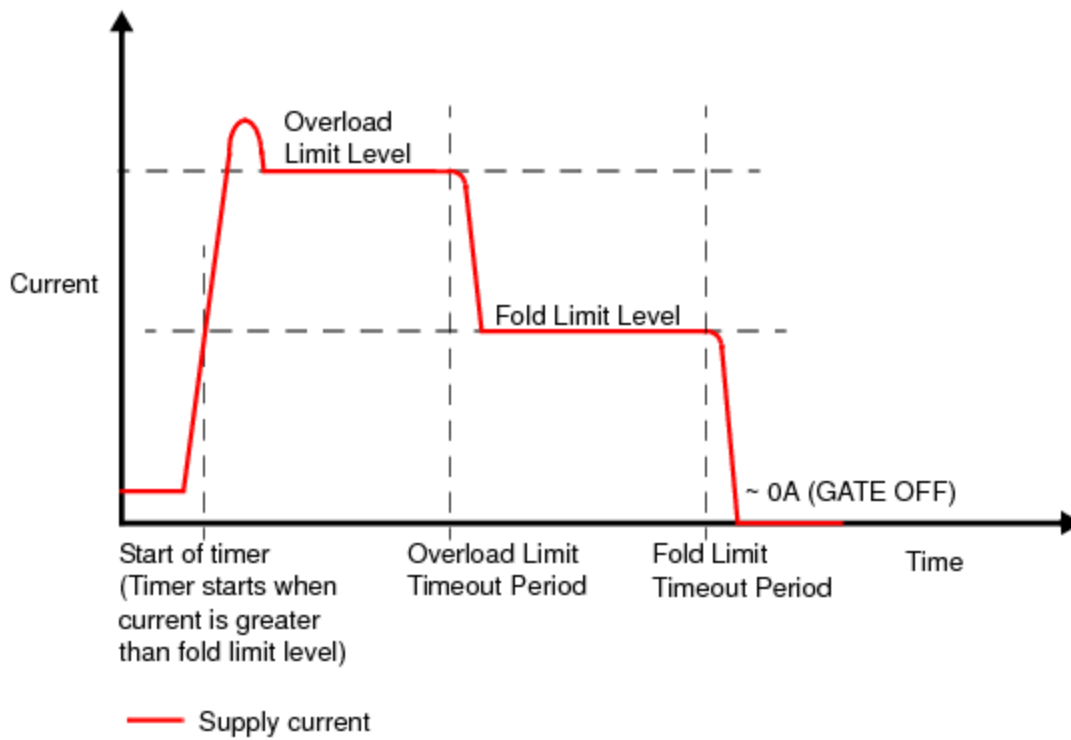
End With

End With

[DCVS Instrument: Programming the Source/Sink Current Limiting Subsystem](#) shows how the VBT code programs the hardware components.

[Dual-Level Current Limiting \(Overload Limit Enabled\)](#)

The hardware limits the current at two different programmable values with two different user-defined current limit timeout values. The following figure shows how the dual-level current limiting works.



Dual-Level Current Limiting

Initially, the operation of dual-level current limiting is similar to the single-level case. When the output current exceeds the fold limit level, a timer starts. To activate dual-level mode, you must enable overload limiting. In dual-level mode, however, the current is not immediately limited to the fold limit level. Instead, the output current is allowed to exceed the fold limit level until the overload limit timeout period expires. However, during this time the current is limited to the overload limit level. Once the overload timeout has elapsed, the output current is limited to the fold limit level.

As in the single-mode case, if the current drops below the fold limit level, the current limit timer is reset. If the timer is not reset and the fold limit timeout elapses, the hardware forces the DCVS output to gate off.

Optionally, you can program the output to stay gated on for as long as possible after the fold limit timeout period has elapsed. For safety reasons, however, the DCVS still gates off eventually (in approximately 10 seconds), even if programmed to remain on.

To change any of the DCVS overload or fold limit parameters, you must use VBT syntax.

This VBT syntax example shows how to program the dual-level source current limit:

With TheHdw.DCVS.Pins(PinList)

.Gate = False ' You MUST turn off gate to:

' a) Enable or Disable the Overload Limiting

' OR

' b) Change Fold or Overload Limit Timeout values

' otherwise a run-time error will occur.

With .CurrentLimit.Source

```
.OverloadLimit.Enabled = True  
.FoldLimit.Level.Value = (0.65 * .FoldLimit.Level.Max)  
.FoldLimit.Timeout.Value = 0.010  
.OverloadLimit.Level.Value = .OverloadLimit.Level.Max  
.OverloadLimit.Timeout.Value = 0.001
```

End With

End With

To change the sink current levels, you can use VBT syntax as shown in this example:

With TheHdw.DCVS.Pins(PinList)

```
.Gate = False ' You MUST turn off gate to:  
    ' a) Enable or Disable the Overload Limiting  
    ' OR  
    ' b) Change Fold or Overload Limit Timeout values  
    ' otherwise a run-time error will occur.
```

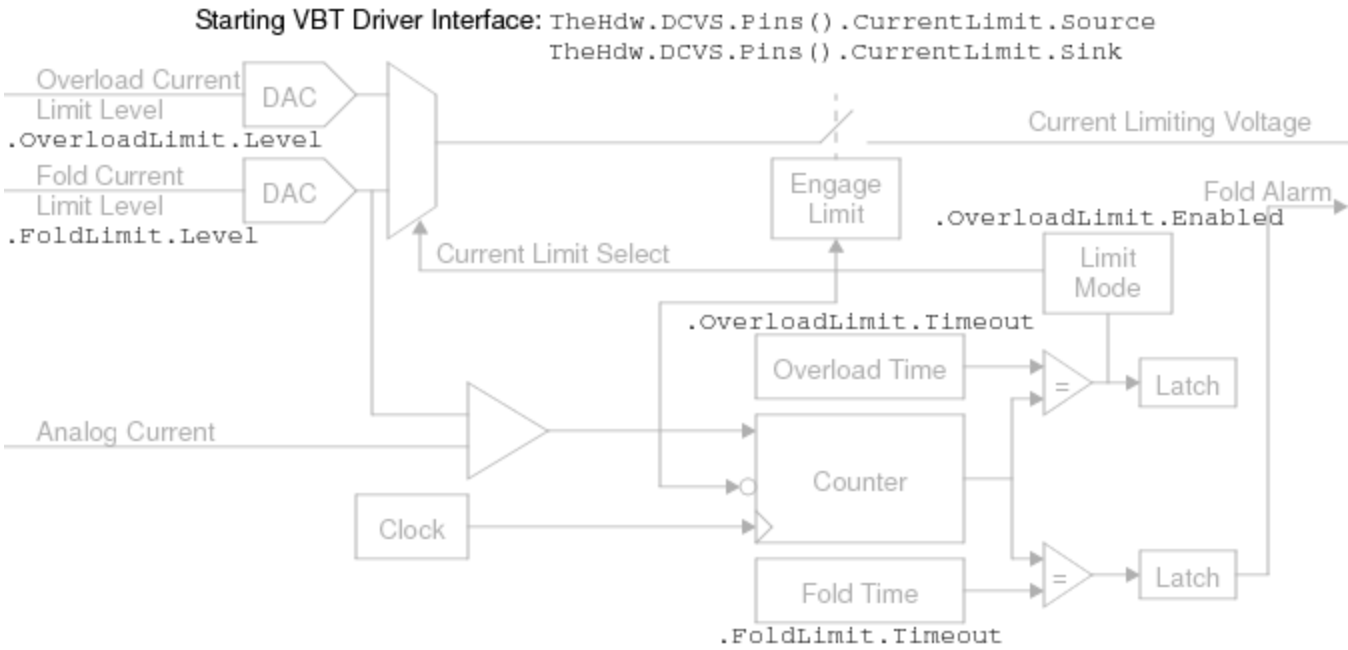
With .CurrentLimit.Sink

```
.OverloadLimit.Enabled = True  
.FoldLimit.Level.Value = (0.65 * .FoldLimit.Level.Max)  
.FoldLimit.Timeout.Value = 0.010  
.OverloadLimit.Level.Value = .OverloadLimit.Level.Max  
.OverloadLimit.Timeout.Value = 0.001
```

End With

End With

The following figure shows how the VBT code programs the hardware components.



- Notes:
1. Overload Limit Level > Fold Limit Level + Minimum Limit Level Difference
 2. Overload Time < Fold Time (Cannot change timeout value while Gate = True)
 3. Programmed Fold Limit Level is subject to output and adjacent pair power compliance restrictions.
 4. Minimum Limit Level Difference is specified in the instrument board specification.

DCVS Instrument: Programming the Source/Sink Current Limiting Subsystem

HexVS Default Current Limit Settings

The default current limit settings when IG-XL is started are as follows:

Source Overload Limit Enabled	False
Source Overload Limit Level	N*16.5A
Source Overload Limit Timeout	~100s (Minimum Overload Limit Timeout Value)
Source Fold Limit Level	N*5A
Source Fold Limit Timeout	100ms
Source Fold Limit Behavior	Gate Off
Sink Overload Limit Enabled	False
Sink Overload Limit Level	N*3A
Sink Overload Limit Timeout	~100s (Minimum Overload Limit Timeout Value)
Sink Fold Limit Level	N*1.5A
Sink Fold Limit Timeout	100ms
Sink Fold Limit Behavior	Gate Off

where N = Number of merged VS outputs

The [Pin Levels sheet parameter](#) `Isc` is used to set the Source Fold Limit Level when the levels worksheet levels are applied (for example, `TheHdw.PinLevels.ApplyPower`).

All other source current limit parameters (source overload level, timeout, and enable) and all sink parameters are not changed. These can only be changed using VBT.

You can use an `OnProgramStarted` executive interpose function to set up your own start-of-program current limiting “defaults” for an entire test program run.

HexVS Current Limit Programming Guidelines

Follow the guidelines listed below when programming current limits:

- Program the overload timeout value less than the fold time value. You will encounter run-time errors if you do not follow this rule.
- Keep the overload level greater than the fold level plus the minimum limit level difference. See the instrument board’s specification document for the minimum limit level difference specification. You will encounter run-time errors if you do not follow this rule.
- When is it “safe” to change current limiting values?

The fold limit and overload limit levels can be programmed with the output gate on (set to true). If any other current limit parameters are changed while the gate is set to true, you will encounter run-time errors.

- Why are my current limit level read-back values not what I programmed?

When programmed voltage exceeds 3.6V, the HexVS hardware does not allow the fold limit level to exceed the maximum power compliance level. So, if you program your value greater than the allowed compliance level, the read-back will not match your programmed values (see [Output Power Compliance](#)).

When adjacent outputs are programmed to different values, the hardware may change the fold limit level for the lower voltage output (see [Shared Tracking Supply Pair Considerations](#)).

- Whenever you program the output voltages, the HexVS hardware determines if the programmed source fold current limit level is programmed to a value that keeps the output power and the shared tracking supply power within specified limits.

If the hardware determines that the programmed current limit allows the delivery of too much power to the DUT or produces too much dissipated shared tracking supply power, then the HexVS hardware overrides the programmed current limit.

The hardware makes this determination at run time; therefore, this check applies to patgen-controlled voltage changes and VBT controlled levels.

For more information, see [Output Power Compliance](#) and [Shared Tracking Supply Pair Considerations](#).

You can use VBT to determine if the DCVS hardware has changed the programmed Current Limit Fold Level using the following language:

```
TheHdw.DCVS.Pins().CurrentLimit.Source.FoldLimit.LevelChanged
```

This function returns a True value if the hardware changed the fold limit; False, if it did not. This may be useful while debugging a test program.

<

>

2015 Teradyne, Inc. All rights reserved.

hxprog.3.14

<

>

HexVS DCVS Programming : DCVS VBT Programming : Setting the Output Compensation Bandwidth

Setting the Output Compensation Bandwidth

The bandwidth parameter allows the DCVS instrument to provide optimal performance with various output current range modes and DIB capacitance. These settings allow you to select the desired performance with your DIB and bypass capacitors.

Use the following syntax to specify the selection of compensation bandwidth.

TheHdw.DCVS.Pins.BandwidthSetting.Value = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7

To obtain a fast dynamic performance, you must adjust a DCVS instrument’s internal compensation bandwidth for the output current range and DIB capacitance. By changing the bandwidth, the response can be optimized for minimum droop or fast settle time, but not for both at the same time. By selecting the proper bandwidth, a DCVS can accommodate a wide range of DIB capacitance.

Note that changes to the output current range affect the performance of a given bandwidth. The DCVS meets or exceeds its dynamic performance specifications if the power supply’s sense lines are at the DIB capacitance and if you meet the minimum design specifications for the DIB and its capacitance. Note that you can compromise the power supply’s droop if you sense at the die or use AC sensing methods.

Bandwidth settings can be divided into two groups:

- Group 1: Compensation Settings 0 through 3

This group covers the same DIB capacitance range as group 2, but the options provide improved stability with less than adequate DIB layout and capacitor selection. The trade-off is more droop during dynamic conditions and much slower settle times than group 2. Compensation setting 3 is recommended for slower response, but minimal droop.

- Group 2: Compensation Settings 4 through 7

Given a proper DIB layout and quality capacitance, use this group of settings. Within this group, setting 4 provides the fastest settle times, and setting 7 minimizes droop during dynamic conditions. Note you may encounter instability with settings in this group.

✓ Note:

Not all bandwidth settings can be used effectively across all capacitance ranges. Generally, the lower bandwidth settings (0-3) accommodate lower capacitance values, and the higher settings (4-7) accommodate higher capacitance values.

General Compensation Guidelines

It is difficult to know which setting will work best for each application. For instance, if you are using the larger of the recommended capacitance for a given current mode range, you may find that compensation 6 works better than 7 for minimum settle. If you have a minimum capacitance for the recommended current mode range, compensation 7 may work the best.

Use the following guidelines to help you choose the compensation setting:

- If you encounter stability problems, use the following troubleshooting techniques in the order shown:

- Try compensation settings 0 through 3.

- Increase the DIB capacitance.

Whenever possible, try to accommodate 25% to 50% more DIB capacitance in your DIB design than what you know you need.

- Check your DUT sense wiring on the DIB.

DIB capacitance sensing is the most stable, and AC sensing is next. Die sensing may prove unstable to use.

- Check capacitor selection on DIB layout (ensure low ESL and ESR capacitors).

- Use the minimum capacitance necessary for the desired performance.

Even though the DCVS supports 1000F with a single supply, the trade-off is increasing current measurement noise with increasing DIB capacitance. This happens with all power supplies, not just

the DCVS.

- [Compensation does](#) not affect DC accuracy.

For more information, see [HexVS DIB Design Guidelines](#).

		
	2015 Teradyne, Inc. All rights reserved.	

hxprog.3.15

<

>

HexVS DCVS Programming : DCVS VBT Programming : Setting Alarms

Setting Alarms

An alarm indicates a hardware condition that may invalidate a measurement. These conditions can be caused by a bad device, an incorrectly programmed instrument, or a test system failure. By default, a test is forced to fail if an alarm occurs. You can set the action to be taken for each DCVS pin when an alarm occurs. The test can be forced to bin, or the alarm can be ignored.

To set up alarms for a test program, use the following VBT syntax:

TheHdw.DCVS.Pins(Pinlist).Alarm(AlarmType) = AlarmBehavior

The following table lists DCVS instrument alarm types for the HexVS.

DCVS Alarm Types for HexVS

Alarm Type	Description
All	Activates all DCVS alarms.
Capture Not Triggered	Indicates that the requested capture could not be started.
Memory Full	Indicates that the capture memory is full.
Open Kelvin DIB	Monitors the voltage drop between the DCVS and the DIB sense points. This alarm has a programmable Kelvin limit and is preset for 250mV of drop before the alarm.
Open Kelvin DUT	Monitors the voltage drop between the DIB sense points and the DUT sense points. It has a programmable Kelvin limit and is preset for 250mV of drop before the alarm.
Sink Fold Current Limit Timeout	Indicates that the sink fold current limit has timed out.
Source Fold Current Limit Timeout	Indicates that the source fold current limit has timed out.

This next table lists DCVS instrument alarm behaviors.

DCVS Alarm Behaviors

Alarm Behavior	Description
tlAlarmContinue	Report the alarm but do not change DUT status.
tlAlarmDefault	Restore behavior to default.
tlAlarmForceBin	Bin the current part.
tlAlarmForceFail	Current part fails.
tlAlarmOff	Disable alarms if possible.

For more information, see:

- Kelvin Sensing and Alarms Subsystem
- HexVS Kelvin Alarm Threshold Programming
- Programming Instrument Alarm Behavior

hxprog.3.16

<	>
---	---

HexVS DCVS Programming : DCVS VBT Programming : Setting the DCVS Current Meter

Setting the DCVS Current Meter

Each DCVS supports current measurement ranges of 10mA, 100mA, 1.0A, 15A, and an unfiltered High Speed 15A range. In merged mode, two additional current measure ranges are available: 15 A x N and High Speed 15 A x N, where N is the number of merged channels.

The unfiltered High Speed 15A range has a faster settle time (40s), compared to the standard 15A range (100s), which makes it better suited for current profiling. Moreover, the High Speed 15A range can measure negative current (-15A). The standard 15A range, however, is more accurate and has more hardware filtering.

To set the High Speed 15 A range, use this code:
TheHdw.DCVS.Pins.Meter.CurrentRange.Value = 15

TheHdw.DCVS.Pins.Meter.Filter.Bypass = True

Note that the output impedance of a channel depends on the meter current range selected. The higher the meter current range is, the lower is the output impedance. Use the highest meter current range to get the best dynamic performance.

For a VBT syntax example showing how to set up the DCVS current meter, see [Current Meter Setup, Strobe, and Readback Programming](#).

For a current profiling example, see [Current Profiling](#).

<	>
---	---

hxprog.3.17

	
---	---

HexVS DCVS Programming : DCVS Pattern Control

DCVS Pattern Control

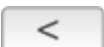
This section describes how to create DCVS test patterns by writing your own test patterns in an ASCII text file using the digital pattern format and DCVS microcodes. You can also use parameter sets (PSets) to program a DCVS instrument's state for testing.

Use the PatternTool to display and debug DCVS test patterns. To bring up PatternTool:

- [Click](#) Pattern Tool in the One-Click Launch group on the IG-XL Tools tab.
- [From the Windows](#) Start menu, select Programs>Teradyne IG-XL <version>>Tools>PatternTool.

This section includes the following topics:

- [HexVS DCVS Pattern Microcodes](#)
- [DCVS Pattern File Example](#)
- [PSets](#)

	
---	---

hxprog.3.18

<

>

HexVS DCVS Programming : DCVS Pattern Control : HexVS DCVS Pattern Microcodes

HexVS DCVS Pattern Microcodes

To create simple or complex test patterns, you can write your own ASCII text file using the digital pattern format and DCVS microcodes. DCVS microcodes can be used to control DCVS settings during pattern execution. This section describes pattern microcodes that are used only in DCVS patterns.

In DCVS patterns:

- All microcodes in a pattern vector are executed when the pattern vector is run. If a microcode in a pattern does not have all of its fields programmed, the unprogrammed fields are programmed to the default values when the pattern vector is run.
- You cannot have more than one DCVS microcode for each pin.
- Microcodes are not case sensitive.

Use the following DCVS microcodes in test pattern files:

HexVS DCVS Pattern Microcodes	
Microcode	Description
Nop	No operation (does nothing).
Pset <PSet Name>	Applies all parameter settings for the specified PSet to the hardware.
Gate_On	Gates the DCVS on.
Gate_Off	Gates the DCVS off.

Strobe	Strobes the DCVS meter once for that pin or channel. The reading is stored in measurement RAM. To retrieve use the call <code>TheHdw.DCVS.Pins("pinname").Meter.Read(tlNoStrobe, 1)</code> . By specifying no strobe, you read the measurement stored in memory without taking a new measurement.
Enable_Inst_Cond	Turns on reporting of the DCVS condition bits.
Disable_Inst_Cond	Turns off reporting of the DCVS condition bits.
Clear_Cond	Clears the state of the DCVS condition bits.
Enable_Alarm	Enables reporting of all the DCVS alarms.
Disable_Alarm	Disables reporting of all the DCVS alarms while the PatGen is running.
VS_Main_Sel	Selects the main voltage DAC. This value is the main voltage value you programmed before the pattern started, unless a PSet has been applied, in which case that voltage value is used.
VS_Alt_Sel	Selects the alternate voltage DAC. This value is the alternate voltage value you programmed before the pattern started, unless a PSet has been applied, in which case that voltage value is used.
Stop	Stops any capture that is currently happening on the pin.
Start_Cap_Timer	Reserved for future use.
Stop_Cap_Timer	Reserved for future use.
Trig <Signal Name>	Triggers a capture with the specified capture tag and the current hardware settings. During pattern execution, the <code>LoadSettings</code> method cannot be called. So, make sure to set up the hardware as desired before the pattern starts. You can set up capture hardware settings during a pattern burst using PSets.
Filter_On	Turns the meter filter on.
Filter_Off	Turns the meter filter off. This is only useful when you are in the 15A or $(15A * N)$ meter current range and you have the filter on. If that is the case, the filter is bypassed. Otherwise, the command is ignored.

For more information, see [Microcode \(Opcodes and Control Bits\)](#) in the Pattern Language section.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxprog.3.19

	
---	---

HexVS DCVS Programming : DCVS Pattern Control : DCVS Pattern File Example

DCVS Pattern File Example

The following DCVS pattern file shows the required pattern file statements. Note the following guidelines.

To make porting patterns between digital instruments easier, do not include a `digital_inst` or `opcode_mode` compiler options in your pattern file. Instead, include these options in the compile scripts or batch files. By following this programming practice, your pattern files are not locked into a particular instrument or timing mode and can be ported easily.

UltraFLEX HexVS pattern microcodes can be executed from both vector memory (VM) and subroutine memory (SRM).

For an example of a pattern file used in Vbump testing, see [Vbump Testing](#).

Settle Time Repeat Guidelines

The number of repeat cycles used in a settle time repeat depends on the following:

- | | |
|---|---|
| • | The timeset period defined in the active time sheet |
| • | The microcode or PSet settle time |
| • | The instrument's analog settling time (if applicable) |
| • | The minimum execution time required between microcodes or PSets |

The goal is to have enough repeats to ensure that no microcode is executed before the previous one is completed and/or the new instrument's analog parameters have time to settle (for example, output voltage level). The microcode and PSet execution times are specified in the instrument board specification document.

```
import tset ts1;

instruments = {
    vdd : DCVS;
    vio : DCVS;
}

srm_vector ($tset, dig1:S)
{
    > ts1 1;
    > ts1 1;
    // __ GATE ON __
    (vdd : DCVS = Gate_On)
    > ts1 1;
    // settle time repeat. See guidelines above.
    repeat 5000
    > ts1 1;
    > ts1 1;
    > ts1 1;
    > ts1 1;
    // __ GATE OFF __
    (vio : DCVS = Gate_Off)
    > ts1 1;
    // settle time repeat. See guidelines above.
    repeat 5000
    > ts1 1;
    > ts1 1;
    > ts1 1;
    > ts1 1;
    // __ TRIGGER CAPTURE: my_capture_signal_label defined in job thru VBT __
    (vdd : DCVS = Trig my_capture_signal_label)
    > ts1 1;
    // settle time repeat. See guidelines above.
    repeat 5000
    > ts1 1;
    > ts1 1;
    > ts1 1;
    > ts1 1;
    // __ MULTIPLE PINS SAME MICROCODE __
```

```
((vdd,vio) : DCVS = Gate_On)
> ts1 1;
// settle time repeat. See guidelines above.
repeat 5000
> ts1 1;
> ts1 1;
> ts1 1;
> ts1 1;
// __ MULTIPLE PINS DIFFERENT MICROCODES __
(vdd : DCVS = VS_Alt_Sel)
(vio : DCVS VS_Main_Sel)
> ts1 1;
// settle time repeat. See guidelines above.
repeat 5000
> ts1 1;
> ts1 1;
> ts1 1;
> ts1 1;
// __ SINGLE STROBE __
(vdd : DCVS = Strobe)
> ts_strobe 1;
> ts1 1;
// settle time repeat. See guidelines above.
repeat 5000
> ts1 1;
> ts1 1;
// __ PARAMETER SET (my_pset_1, my_pset_2 defined in job thru VBT __
> ts1 1;
(vdd : DCVS = PSet my_pset_1)
> ts1 1;
// settle time repeat. See guidelines above.
repeat 5000
> ts1 1;
> ts1 1;
> ts1 1;
> ts1 1;
(vdd : DCVS = PSet my_pset_2)
> ts1 1;
```

```
// settle time repeat. See guidelines above.  
repeat 5000  
  > ts1  1;  
  > ts1  1;  
  > ts1  1;  
  > ts1  1;  
  > ts1  1;  
}
```

hxprog.3.20

<

>

HexVS DCVS Programming : DCVS Pattern Control : PSets

PSets

Parameter sets (PSets) allow you to program the state of an instrument for testing, such as the force voltage value and force voltage range. You set up its characteristics using its parameters. You define a PSet in VBT language and invoke it using PSet microcode.

Once defined, PSets can be applied through VBT or through patterns using the DCVS PSet microcode. Assuming you have defined a parameter set named, "my_pset", the following example shows how to apply the PSet from VBT:

```
TheHdw.DCVS.Pins("vdd").Psets("my_pset").Apply
```

The statement above is equivalent to:

```
TheHdw.DCVS.Pins("vdd").Psets.Item("my_pset").Apply
```

PSet parameters are initialized with their default values, and all parameters are updated when the PSet is applied.

The following example shows an ASCII test pattern with a DCVS microcode entry to apply this parameter set:

```
(vdd:DCVS = Pset my_pset)
> tset0  H L 1 0 . . .;
```

Other PSet language properties and methods:

TheHdw.DCVS.Pins.PSets.Count	This property returns the number of PSets that have been defined for the pin.
TheHdw.DCVS.Pins.PSets.List	This property returns a string array that is a list of all PSet names defined for the pin.
TheHdw.DCVS.Pins.PSets.Add	This method creates a new PSet in PSet memory and sets all PSet parameters to the default settings. If your new PSet has

the same name as a PSet that already exists in memory, then this PSet is set to default settings.

TheHdw.DCVS.Pins.PSets.Item

This property accesses a PSet from the PSet memory, but it does not change that PSet's parameters to default settings. If the PSet does not exist in memory when called by Item, IG-XL returns a run-time error. You can program one or all PSet parameters using this property.

For a PSet example using VBT, see [DCVS PSet Example Function](#).

DCVS PSet Example Function

This [example function](#), AddPSet, creates a DCVS parameter set. To use the AddPSet function in Vbump testing, see [Vbump Testing](#).

```
Function AddPSet(PinName As String, _
    PSetName As String, _
    BandwidthSetting As Double, _
    CurrentRange As Double, _
    SampleRate As Double, _
    SampleSize As Long, _
    ConditionBitCaptureDone As Boolean, _
    ConditionBitLevelChanged As Boolean, _
    ConditionBitSourceTimeout As Boolean, _
    ConditionBitSinkTimeout As Boolean, _
    SourceFoldLimit As Double, _
    SourceOverloadLimit As Double, _
    MeterCurrentRange As Double, _
    MeterVoltageRange As Double, _
    MeterMode As tIDCVSMeterMode, _
    MainVoltage As Double, _
    AltVoltage As Double _
) As Long
```

```
Dim MinILimitLevelDelta As Double
```

```
On Error GoTo ErrorHandler
```

```
Call TheHdw.DCVS.Pins(PinName).PSets.Add(PSetName)
```

```
With TheHdw.DCVS.Pins(PinName).PSets.Item(PSetName)
```

```
.BandwidthSetting.Value = BandwidthSetting
```

```
' Need to set the output current range first because it determines the
' available Ilimit and Imeter ranges
```

```
.CurrentRange.Value = CurrentRange
```

```
.Capture.SampleRate.Value = SampleRate
```

```
.Capture.SampleSize.Value = SampleSize
```

```
.ConditionBit(tlDCVSConditionCaptureComplete) = ConditionBitCaptureDone
```

```
.ConditionBit(tlDCVSConditionSourceCurrentFoldLimitLevelChanged) =
```

```
ConditionBitLevelChanged
```

```
.ConditionBit(tlDCVSConditionSourceCurrentLimitTimeout) = ConditionBitSourceTimeout
```

```
.ConditionBit(tlDCVSConditionSinkCurrentLimitTimeout) = ConditionBitSinkTimeout
```

```
With .CurrentLimit.Source
```

```
MinILimitLevelDelta = GetMergeFactor(PinName) * HEXVS_MIN_ILIMIT_LEVEL_DELTA
```

```
' Note: GetMergeFactor() and HEXVS_MIN_ILIMIT_LEVEL_DELTA are defined elsewhere
```

```
If SourceOverloadLimit >= SourceFoldLimit + MinILimitLevelDelta Then
```

```
    If SourceOverloadLimit > .FoldLimit.Level.Value + MinILimitLevelDelta Then
```

```
        .OverloadLimit.Level.Value = SourceOverloadLimit
```

```
        .FoldLimit.Level.Value = SourceFoldLimit
```

```
    Else
```

```
        .FoldLimit.Level.Value = SourceFoldLimit
```

```
        .OverloadLimit.Level.Value = SourceOverloadLimit
```

```
    End If
```

```
Else
```

```
    ' This is an invalid setup
```

```
TheExec.Datalog.WriteComment ("AddPSet() Warning: Cannot set " & _
```

```
    "Source OverloadLimitLevel < FoldLimitLevel + " & _
```

```
    MinILimitLevelDelta & _
```

```
    " for pset named, " & _
```

```
    PSetName & _
```

```
    " (" & PinName & ")." & _
```

```
    " Attempt aborted.")
```

```
End If
```

```
End With
```

```
.Meter.CurrentRange.Value = MeterCurrentRange
```

```
.Meter.VoltageRange.Value = MeterVoltageRange
```

```
.Meter.Mode = MeterMode
.Voltage.Main.Value = MainVoltage
.Voltage.Alt.Value = AltVoltage
End With
Exit Sub
```

ErrorHandler:

```
Dim Reply As Integer

Reply = MsgBox("Error detected during AddPSet(). Stop to Debug?", vbYesNo, "HexVS PSet
Checkout")
If Reply = vbYes Then
    On Error GoTo ErrorHandler
    Stop
Else
    On Error GoTo 0
End If

Resume Next

End Function
```

hxprog.3.21

	
---	---

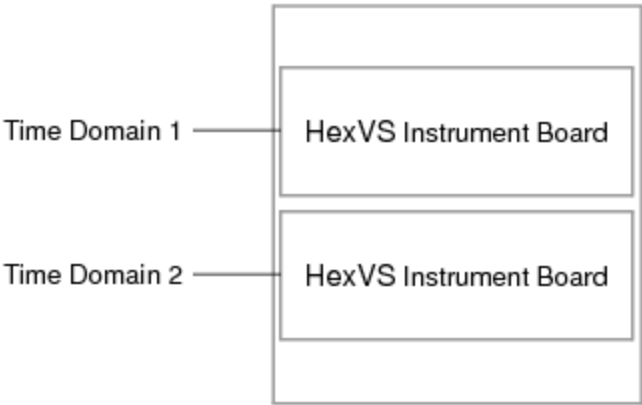
HexVS DCVS Programming : Using HexVS in Concurrent Testing

Using HexVS in Concurrent Testing

To support concurrent testing, you can assign each HexVS instrument to a unique time domain. Concurrent features for HexVS include:

- Each HexVS board can be configured for one time domain
- Each HexVS instrument can be controlled by a unique time domain using Sync-Link. Features include:
 - Independent access to Sync-Link
 - Independent sample clock programming, one sample rate per channel
 - Independent clock synchronization with other HexVS instruments or with digital clocks
 - Independent control by digital pattern
 - Independent capture memory and triggers
- Multiple instruments can be grouped into a common time domain
- HexVS is restricted to using the Global flow domain and synchronous pattern starts

The following figure shows a configuration with each HexVS instrument assigned to its own time domain.



HexVS Concurrent Testing Configuration Example, 2 Time Domains
For more information, see [About Concurrent Test](#).

<

>

2015 Teradyne, Inc. All rights reserved.

hxttheory.2.01

<

>

HexVS Theory of Operation

HexVS Theory of Operation

This section provides a theory of operation that includes a HexVS hardware summary, test head connection information, a functional description with block diagrams, and output power specifications.

The following topics are available:

- HexVS Board Hardware
- Test Head Connections
- Functional Description
- Output Power

For information on HexVS instrument performance specifications and standards, see [HexVS Specifications](#).

<

>

hxttheory.2.02

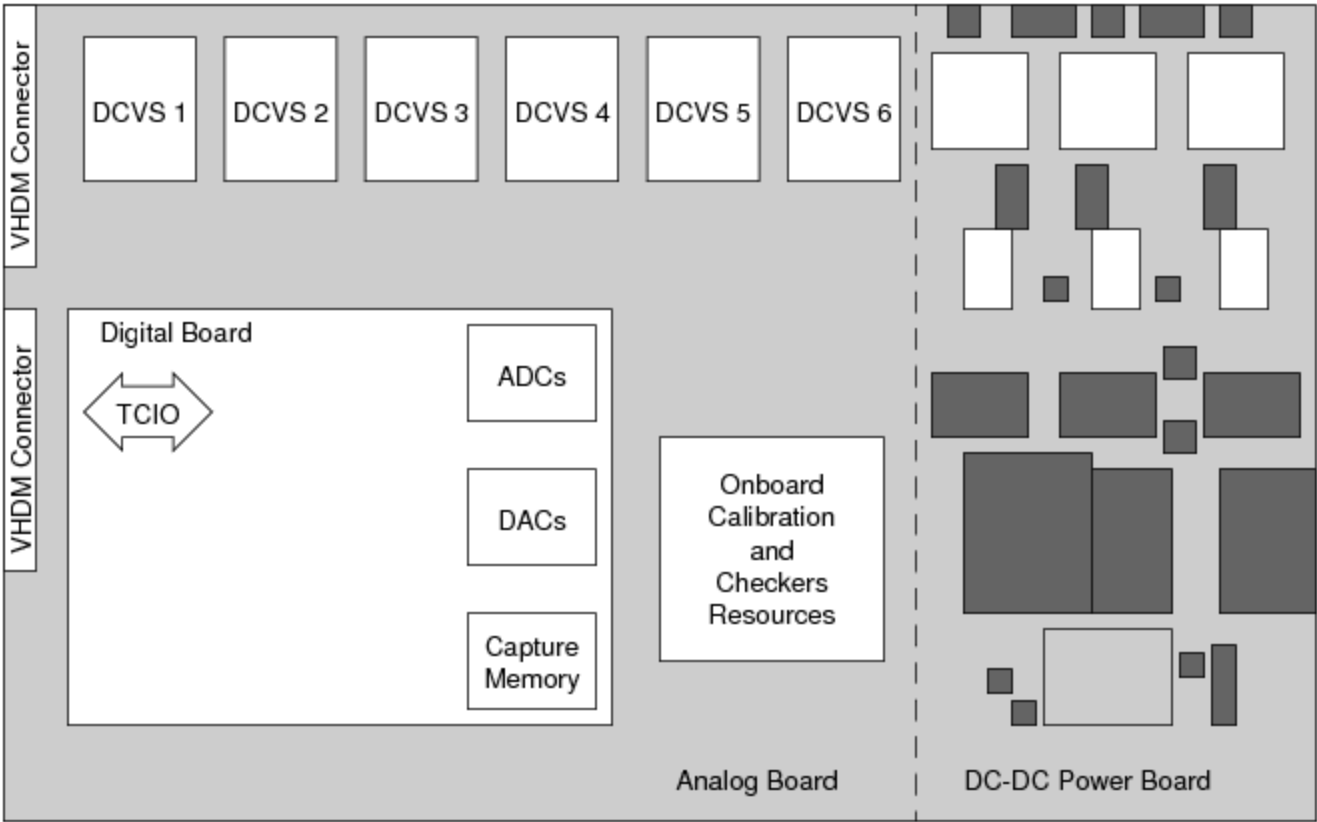
<

>

HexVS Theory of Operation : HexVS Board Hardware

HexVS Board Hardware

The HexVS instrument board assembly is located in the test head and consists of a set of three boards: analog, digital, and DC-DC power. The following figure shows the layout of an HexVS instrument board.



HexVS Instrument Board Hardware Block Diagram

From a programming perspective, the HexVS board can be viewed as a set of one or more independent DCVS instruments. However, implementation of most programmable DCVS features are distributed across the HexVS analog and digital boards. The power board also plays an important role in HexVS operation.

Analog Board

The analog board contains the output stages for the six single-quadrant DCVS power supplies. These supplies can be merged to increase the maximum output current and power delivered to the DUT. The analog board also contains the following components:

- Kelvin sense and current monitor circuits
- Current limiting and compensation bandwidth control hardware
- Power management hardware that controls DCVS output power compliance
- On-board calibration and checkers resources

Digital Board

The digital board contains the primary DCVS instrument control and tester communication components and has the following features:

- TCIO databus connection to communicate to the test system host computer
- Voltage output and current limit level DACs, one set per DCVS
- Voltage/current meter ADC and capture memory, one set per DCVS
- Parameter set memory, one memory per DCVS

Power Board

The power board’s primary function is to convert the +48V system power to the various on-board power voltages that the HexVS board requires. The power board hardware includes three DC-DC tracking supplies that supply power to the DCVS output stages. These tracking supplies are shared by DCVS channels in pairs: VS1 and VS2 share one tracking supply, VS3 and VS4 share another, and VS5 and VS6 share the third. This tracking supply sharing has important implications when designing a DIB that uses non-merged DCVS channels.

For more information, see HexVS DIB Design Guidelines.

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxttheory.2.03

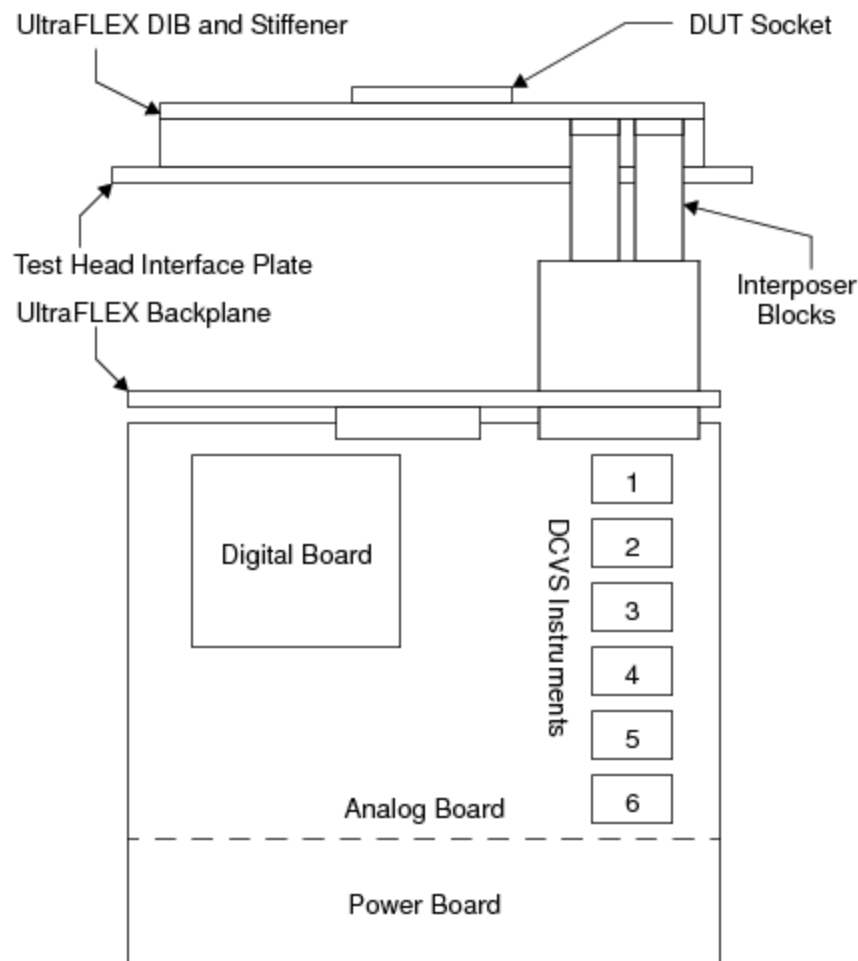


HexVS Theory of Operation : Test Head Connections

Test Head Connections

This section shows where the boards are connected to the test head backplane.

The following figure shows how the instrument board is connected with an interposer block to deliver power to the DUT.



HexVS Instrument Board Connection to the DUT

Several HexVS boards can be installed in the UltraFLEX test head. A HexVS can be installed in any available slot in the UltraFLEX test head. To ensure compatibility across a variety of UltraFLEX test system applications, however, Teradyne recommends the test head and DIB slot locations for up to two HexVS boards, as listed in the following table.

Note:

For applications requiring more than two HexVS instrument boards, contact Teradyne, whose test applications team can help you develop a solution that meets your needs.

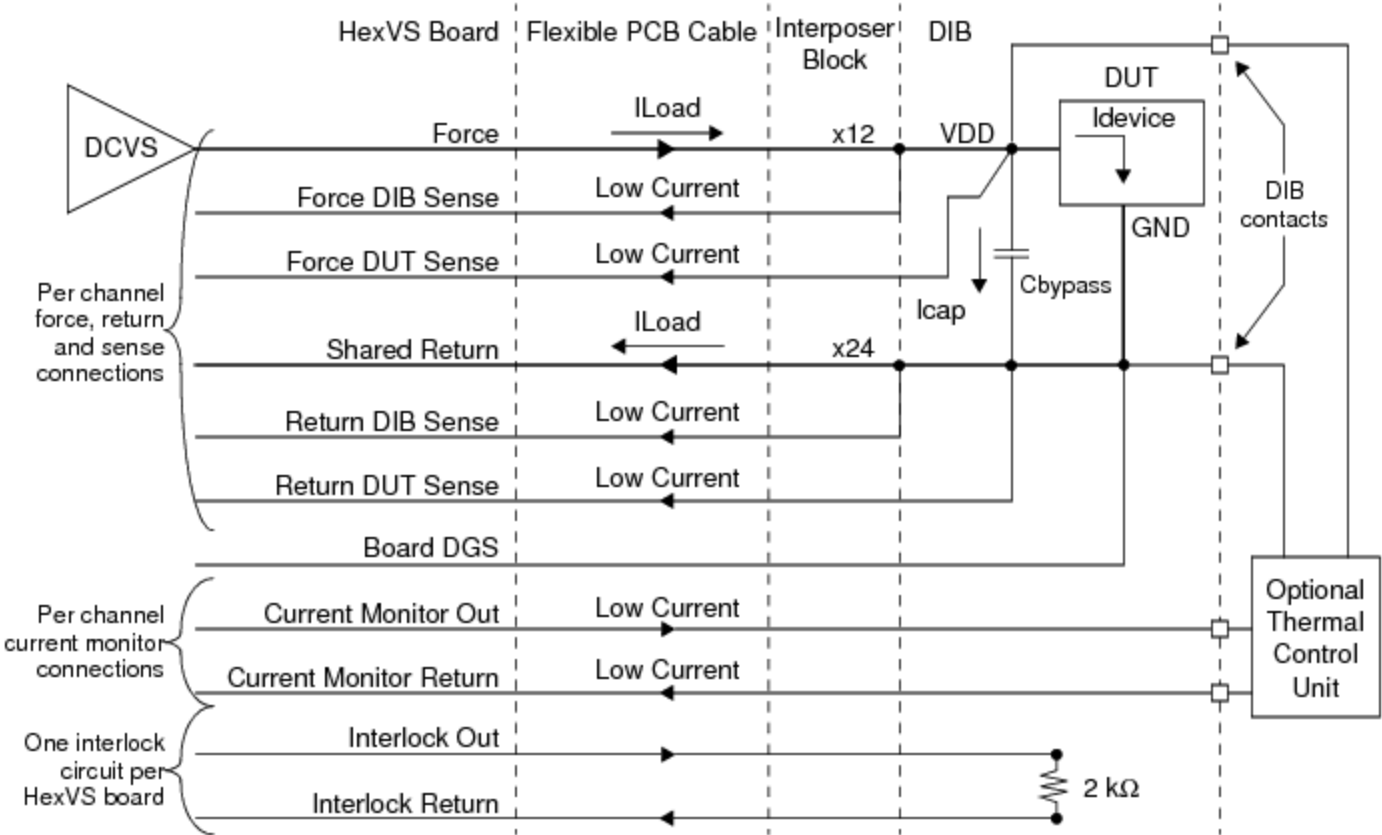
HexVS Slot Locations for Two Boards

Test Head Slot	DIB Slot
19	19
21	21

The HexVS test head interface features a high-performance connection to support high current to the DUT through a low impedance path. This interface supports the ability of each DCVS to respond rapidly to current transients, which helps reduce the need for large amounts of power supply bypass capacitance on the DIB. This saves space on the DIB for additional test sites, further enabling the testing of multiple devices at once.

For more information, see HexVS DIB Design Guidelines.

The following figure shows a more detailed connection model for the DCVS signals to the DUT.



DCVS Channel Connections to the DUT

☒ **Note:**

For proper operation, the load bypass capacitor for each DCVS must be installed at the load. Failure to do this causes oscillation at the output. Minimum capacitance is 300F per DCVS channel.

<		>	
		2015 Teradyne, Inc. All rights reserved.	

hxttheory.2.04

<	>
---	---

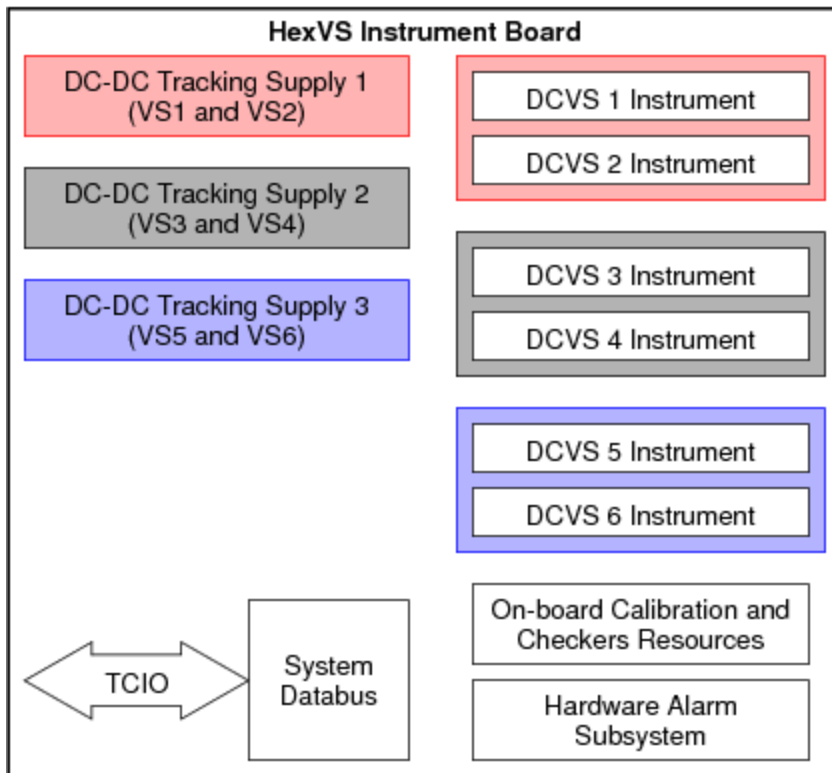
HexVS Theory of Operation : Functional Description

Functional Description

This section describes how signals move through the HexVS instrument board to the DUT from a programmer's point of view.

The HexVS board consists of several functional subsystems:

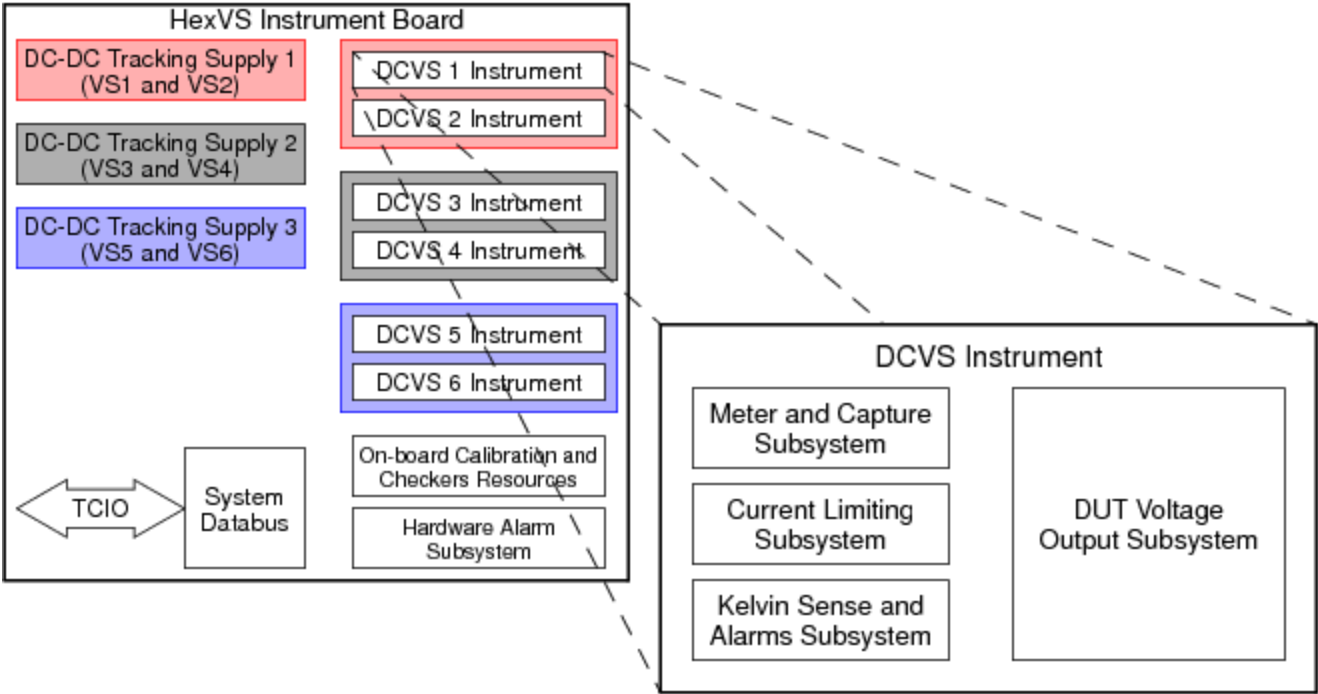
- Six DCVS instrument subsystems that can be merged to create fewer, more powerful DCVS instruments
- Hardware alarm detection and response subsystem
- Tester databus communication (TCIO) subsystem
- Calibration and checkers subsystem



HexVS Board Functional Overview

Each DCVS instrument consists the following functional subsystems (see the following figures):

- DUT Voltage Output Subsystem
- Current Limiting Hardware Subsystem
- Metering and Capture Subsystem
- Kelvin Sensing and Alarms Subsystem



DCVS Instrument: Logical/Functional Overview

< >		
	2015 Teradyne, Inc. All rights reserved.	

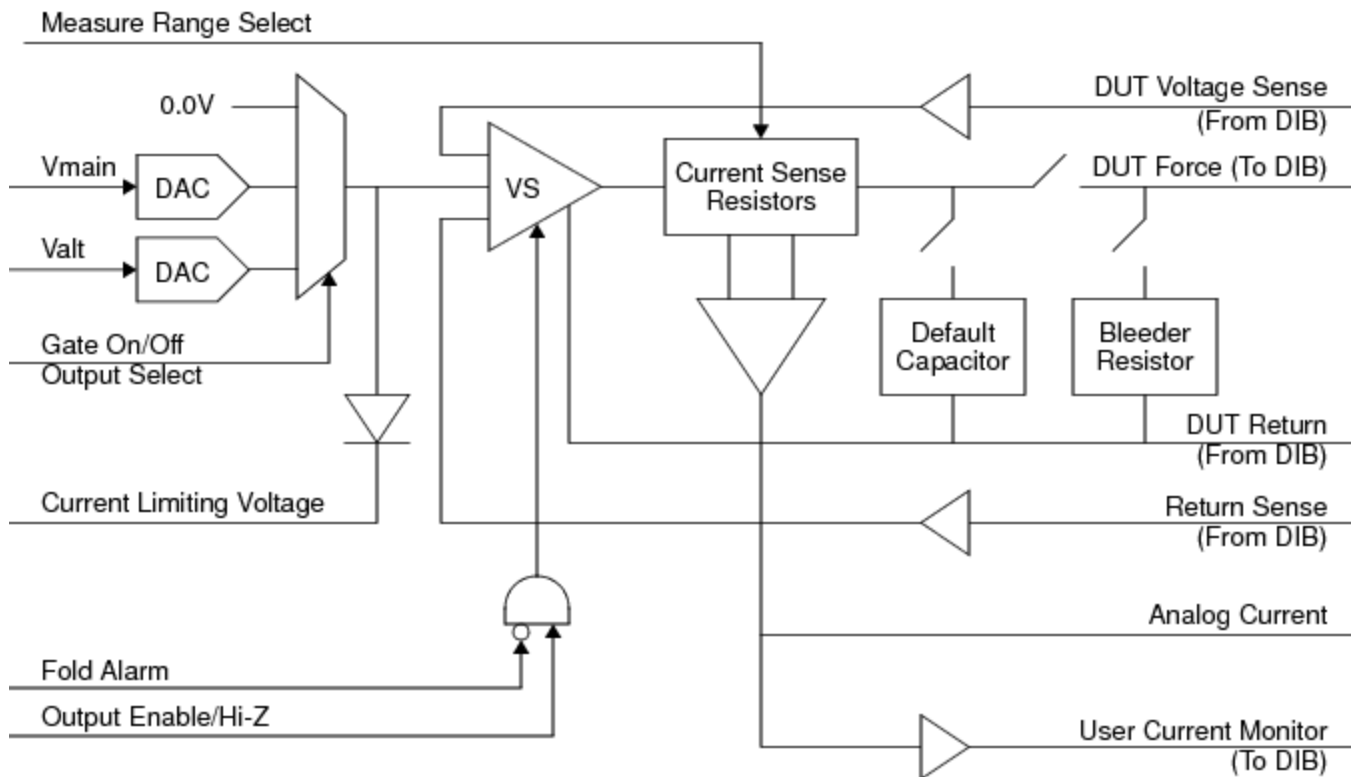
hxttheory.2.05



HexVS Theory of Operation : Functional Description : DUT Voltage Output Subsystem

DUT Voltage Output Subsystem

The following figure is a block diagram of the DCVS output subsystem.



DCVS Instrument: DUT Voltage Output Subsystem

Main and Alternate DACs

The programmed output voltage is set using the main and alternate DACs. Whether the main or alternate DAC is used, an on/off gate and an output selection multiplexer determines the value of the DCVS output stage. The selected DAC value controls the output driver delivering power to the DIB through the DUT force lines.

DUT Voltage Sense Signal

The DUT voltage sense signal monitors the voltage at the DUT. When the voltage at the DUT exceeds the selected programmed value, the output driver drops its voltage, thereby decreasing the current sourced to the DUT. Similarly, when the sensed voltage at the DUT drops below the programmed value, the output driver increases its output voltage and sources more current to compensate for the drop at the DUT pins. This is an automatic and continuous process that uses the DUT sense feedback to control the power delivered to the DUT.

When the DCVS output current changes rapidly from a low level to a high one, you expect a voltage drop. The DCVS senses this drop and compensates for it. The DCVS output system bandwidth controls how quickly the DCVS output responds to changes in the DUT capacitor voltage.

The DCVS also features programmable output bandwidth settings. By changing the bandwidth setting, you can program the DCVS output to respond faster or more slowly to changes in output current demand.

Current Limiting Signal

The current limiting signal and fold alarm help to control the maximum current that can be supplied to the DUT. For more information, see [Current Limiting Hardware Subsystem](#).

The DUT force and return lines make up the complete current loop bringing power to the device. They support the full current capability of the force lines. The return sense lines sense the power supply return path voltage at the DUT. So they are also part of the DUT voltage analog feedback control that monitors the voltage across the DUT power pins.

☑ **Note:**

The return paths are shared between DCVS pairs. This can affect your DIB design if you must separate the return planes.

Gating off the DCVS output sets the voltage output value to the voltage value sensed by the VSG DIB input signal (not shown). Ideally, this is the DUT ground voltage of 0.0V.

Embedded with the voltage output circuitry is the analog current and voltage meter circuitry. This includes the buffered DUT voltage sense input, current measure range select signal, analog current measurement hardware, and internal current signal shown in the figure above. For information on the functionality of these components, see [Metering and Capture Subsystem](#).

Current Monitor for Thermal Control

This current monitor signal is a low-current buffered output that is routed to the DIB interposer. With a properly designed DIB, the user current monitor signal can be used with the programmed DUT voltage to provide real time DUT power consumption information to an active DUT temperature control unit connected to the DUT side of the DIB.

Note that you cannot control the current monitor feature in your programming. This signal is directly connected to the maximum current range of the analog current meter signal. Its output is not affected by changes in the current measure range. The current monitor output voltage ranges between 0V and 4.32V. The relationship between output current and current monitor voltage is

$N \times 3.47A/V$, where N = the number of merged DCVS outputs. Thus, a voltage of 4.32V indicates that the maximum DCVS output current is being delivered to the DUT.

Local Output Capacitor and Bleeder Resistor

The output subsystem of each DCVS includes a default capacitor and bleeder resistor. When the DCVS is disconnected from the DIB, the default capacitor keeps the DCVS voltage output from oscillating, and the bleeder resistor helps quickly discharge DIB capacitance. You can program and control the connection state of this capacitor and resistor. The default states for these components is determined by hardware based on the state of the DCVS connection state. The default states are outlined in the table below.

Default Capacitor and Bleeder Resistor Connection States

Component	Default Connection State	
	DCVS Output Connected	DCVS Output Disconnected
Default Capacitor	Disconnected	Connected
Bleeder Resistor	Disconnected	Connected

 **Note:**
If you manually set the local capacitor to Off when you disconnect from the DUT, the local capacitor always reverts to the Auto mode to ensure the stable operation of the DCVS.





2015 Teradyne, Inc. All rights reserved.

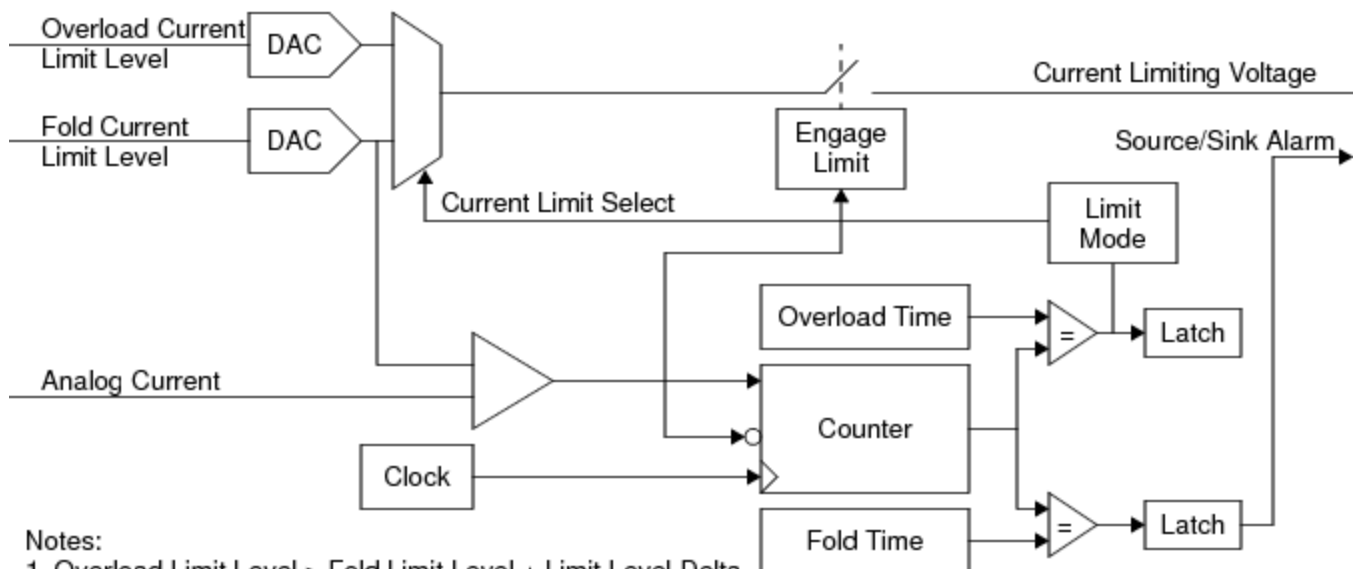
hxttheory.2.06



HexVS Theory of Operation : Functional Description : Current Limiting Hardware Subsystem

Current Limiting Hardware Subsystem

Current limit programming consists of setting the values of various DACs and counters. The following figure illustrates the concept behind a single DCVS instrument's source current limiting hardware.



Notes:

1. Overload Limit Level > Fold Limit Level + Limit Level Delta
2. Overload Time < Fold Time
3. Programmed Fold Limit Level is subject to output and adjacent pair power compliance restrictions.
4. Limit Level Delta is specified in the HexVS instrument board specification document.

DCVS Instrument: Source/Sink Current Limiting Subsystem

The functionality of each sink current limiting hardware is similar to that of the source limiting hardware. You can program the overload and fold current limit levels and timers. You can also program the current limit mode as part of the test setup. The valid modes are dual and single current limit modes. The default is single mode. Finally, you can program the fold limit timeout behavior to either gate off or not.

While the DCVS is gated on, the analog current output signal is compared against the fold limit level. If the analog current exceeds the programmed fold limit level, the current limiting voltage is engaged and a current limit timer is started.

Depending on the current limiting setting, the output current is limited to either the overload limit level (overload enabled) or the fold limit level (overload disabled). With the overload limit level enabled, if the timer value exceeds the programmed overload limit time value, the DCVS voltage output level is limited to the fold limit level. With the overload limit level disabled, the overload limit time is ignored. In both settings, however, if the timer value exceeds the fold limit time, a fold signal is set and the DCVS is gated off or it is left gated on until the maximum fold limit time value is exceeded.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

hxttheory.2.07

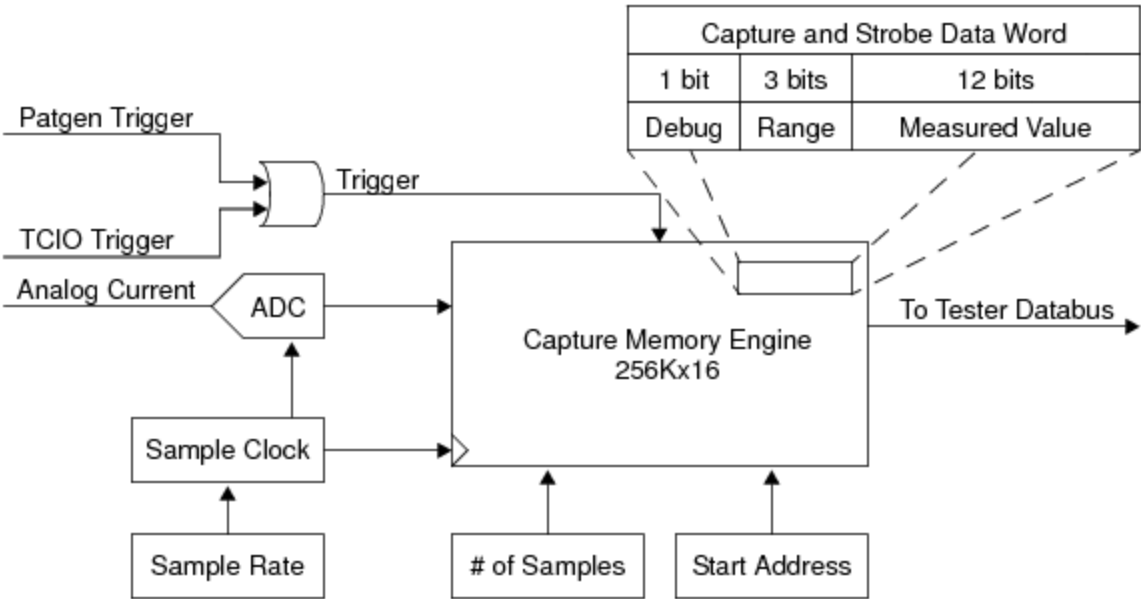
<

>

HexVS Theory of Operation : [Functional Description](#) : Metering and Capture Subsystem

Metering and Capture Subsystem

An ADC is at the heart of the DCVS meter. You can use the meter’s ADC to convert either analog current or buffered DUT voltage sense signals to digital values. The digital values can then be stored in the DCVS capture memory. From there, the values can be read back to the host computer through the TCIO databus using the meter and capture programming interfaces. These features are shown in the following figure.



DCVS Instrument: Metering and Capture Memory Subsystem

You must set the meter mode, either voltage or current, to make a meter measurement. The default meter mode is to measure current. Set the desired meter range before taking the measurement. For a single-strobe measurement this is all the setup that is required before reading. If you want to take multiple samples or capture a waveform, you must program the sample rate and number of samples. When capturing a waveform, you must also program a signal name.

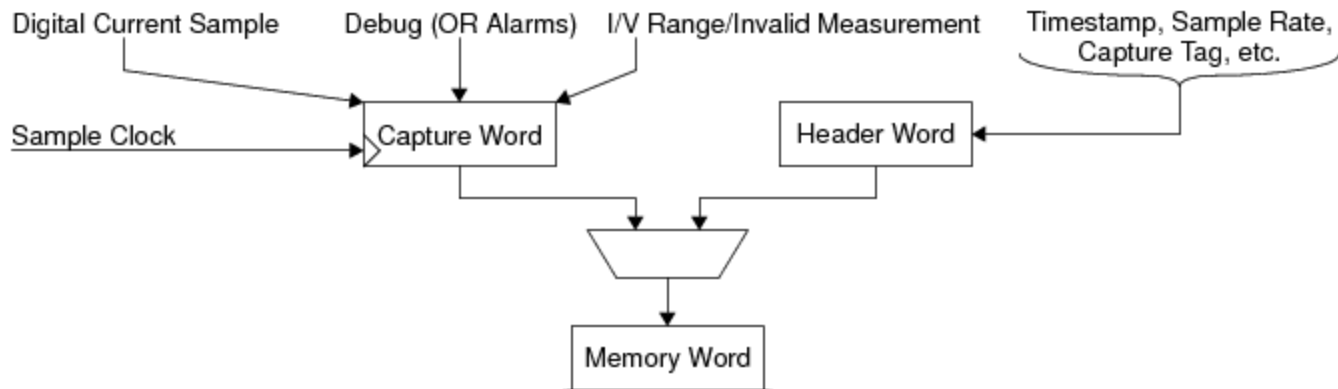
The DCVS meter also employs an “auto-ranging” feature. If the device current exceeds the programmed current range, then the meter automatically ranges up to a current range that

supports the device current. When the supply current drops below the programmed value, the meter ranges down to support the programmed value.

The ADC clock is a continuous 25MHz clock. The user-defined sample rate only changes the sample clock that samples the ADC input to the capture memory engine.

DCVS waveform capture can be triggered either through a program command or through a pattern generator microcode.

The following figure shows the details of the capture memory organization. The capture memory stores strobed values before read-back through TCIO. In terms of capture memory consumption and read-back speed, the strobe is more efficient for small numbers of samples (less than five). If many samples are required, however, captured waveforms make more efficient use of available capture memory.



16-bit Memory Word Definitions

Capture Header	Capture Header Word 1 of 5				
	1 bit	3 bits	4 bits	1 bit	7 bits
	Don't care	Start Flag	Capture Tag	I/V Flag	Real Time Counter [6:0]
	Capture Header Word 2 of 5				
	1 bit	3 bits	12 bits		
	Don't care	Start Flag	Real Time Counter [18:7]		
	Capture Header Word 3 of 5				
	1 bit	3 bits	12 bits		
	Don't care	Start Flag	Real Time Counter [30:19]		
	Capture Header Word 4 of 5				
	1 bit	3 bits	10 bits	1 bit	1 bit
	Don't care	Start Flag	Sample Rate	RTC[31]	Merge
	Capture Header Word 5 of 5				
	1 bit	3 bits	12 bits		
	Don't care	Start Flag	Reserved		
Strobe Header	Strobe Header Word				
	1 bit	3 bits	4 bits	1 bit	7 bits
	Don't care	Start Flag	Capture = 0000	I/V Flag	Don't Care
Strobe and Capture Data	Strobe/Capture Data Word				
	1 bit	3 bits	12 bits		
	Debug	Range	Measured Value		

DCVS Capture Memory Word Subsystem and Definitions

Using Capture Memory Wisely

Generally, do not use more memory locations in your capture memory than you have to when conducting strobe captures. The following figure shows the implications of strobing vs. capturing. You can only plot captured samples, not strobed samples, using the DSPWave object's plot method.

Single Strobe:
Two Memory Locations Used

Header
Data

More memory
used during
capture

Single Sample Capture:
Six Memory Locations Used

Header 1
Header 2
Header 3
Header 4
Header 5
Data

Five Strobes:
10 Memory Locations Used

Header
Data
Header
Data
Header
Data
Header
Data
Header
Data

Equal memory
used during
capture

Five Sample Capture:
10 Memory Locations Used

Header 1
Header 2
Header 3
Header 4
Header 5
Data
Data
Data
Data
Data

100 Strobes:
20 x 10 (5 Strobes Case) = 200
Memory Locations Used

Less memory
used during
capture

100 Sample Capture:
5 (Headers) + 100 (Data Locations) = 105
Memory Locations Used

Memory Usage: Strobe vs. Capture

<

>

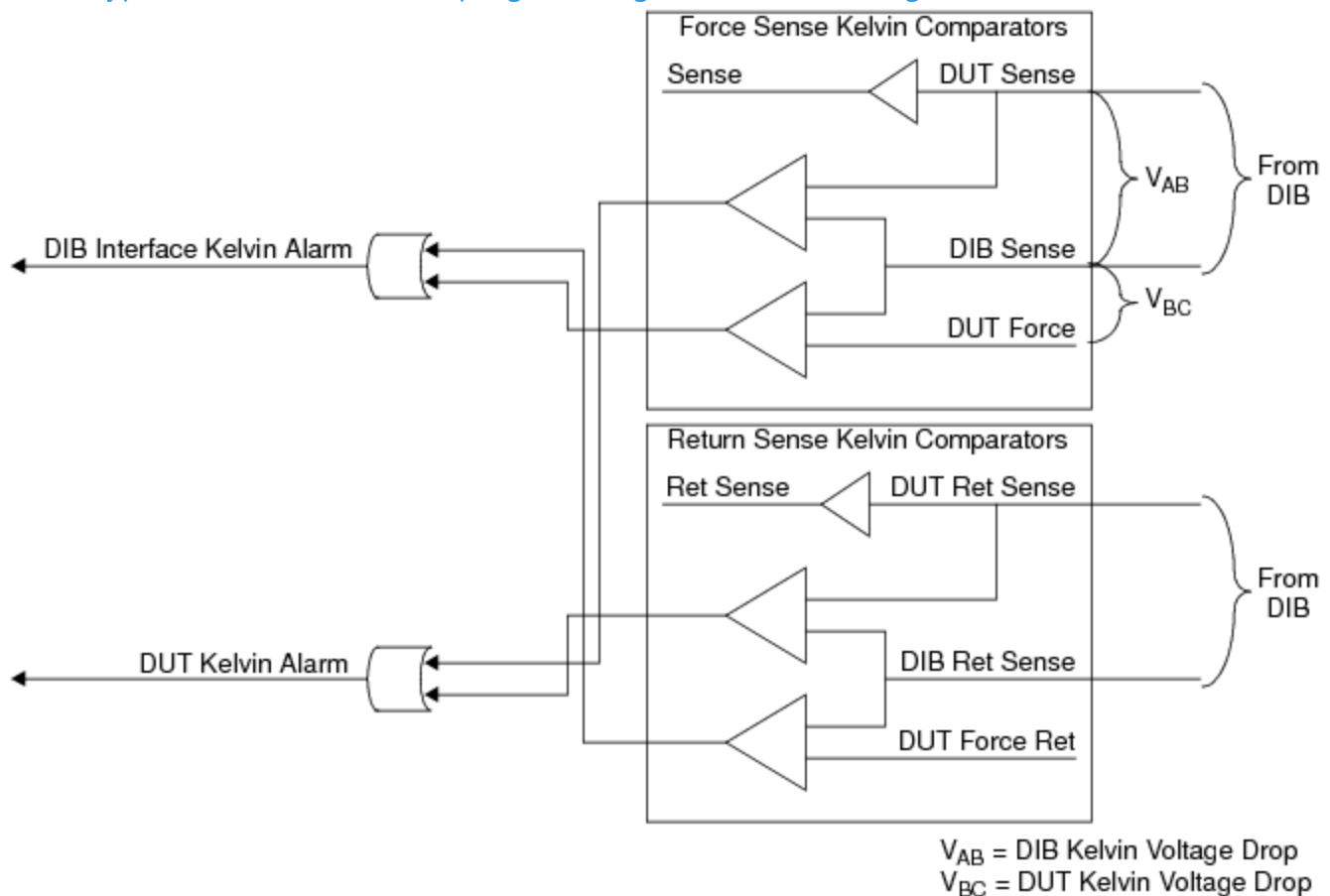
hxttheory.2.08



HexVS Theory of Operation : [Functional Description](#) : Kelvin Sensing and Alarms Subsystem

Kelvin Sensing and Alarms Subsystem

The following figure shows the DCVS Kelvin sensing and alarm subsystem hardware. For a list of alarm types and information on programming alarms, see [Setting Alarms](#).



DCVS Instrument: Kelvin Sense and Alarms Subsystem

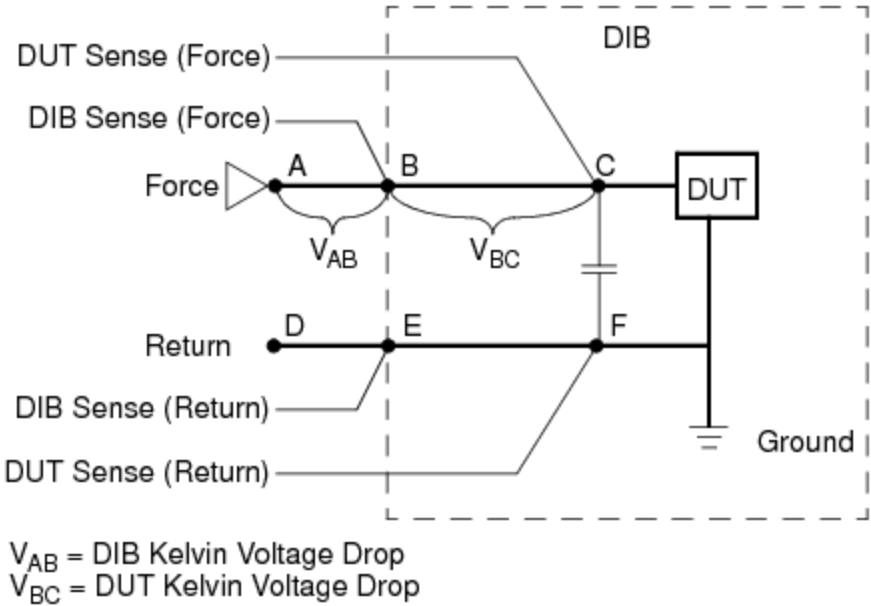
DIB Interface Kelvin Alarm

On a HexVS board, each voltage supply requires DIB sense Kelvin connections. These DIB sense pins assure good contact between the DCVS interface (interposer) and DIB signal pads. Two DIB interface sense pins are used for each DCVS output channel, one for the force path and one for the

return path. The DCVS triggers a DIB Kelvin alarm if the voltage difference between the VS output point and the corresponding DIB interface sense point is greater than the preset Kelvin threshold value. (Default is 250mV.) See the figure below.

DUT Kelvin Alarm

Each HexVS DCVS channel requires DUT sense Kelvin connections. Two DUT sense pins are used for each DCVS output, one for the force path and one for the return path. The DCVS triggers a DUT Kelvin alarm if the voltage difference between the DIB Kelvin point and corresponding DUT voltage sense point is greater than the preset Kelvin threshold value. (Default is 250mV.) See the figure below.



DIB Interface and DUT Kelvin Alarm Sense Points

<		>	
		2015 Teradyne, Inc. All rights reserved.	

hxttheory.2.09

<	>
-----------------	-----------------

HexVS Theory of Operation : Output Power

Output Power

This section discusses the output power capabilities and restrictions of the HexVS board’s DCVS instruments. It includes the following topics:

- HexVS DCVS Instrument Features
- Output Power Compliance
- Shared Tracking Supply Pair Considerations
- Merged Mode

<	>
-----------------	-----------------

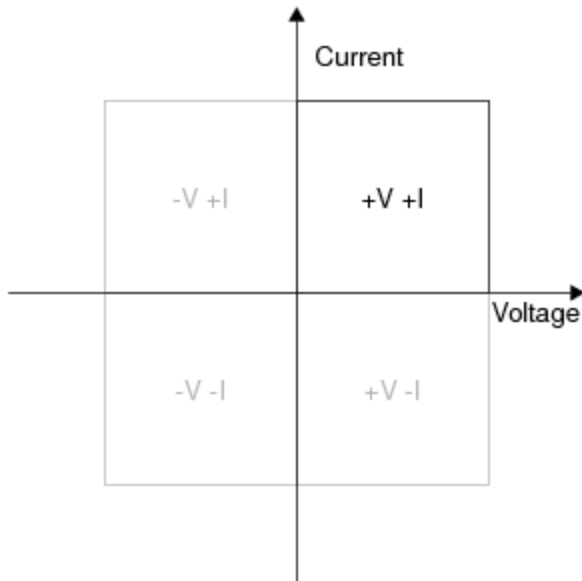
hxttheory.2.10

<	>
---	---

HexVS Theory of Operation : [Output Power](#) : HexVS DCVS Instrument Features

HexVS DCVS Instrument Features

Each DCVS instrument on the HexVS board operates in a single quadrant, allowing forcing of a positive voltage while sourcing high current. Each DCVS instrument also supports limited current sinking to ground. See the following figure.



Voltage and Current Operating Quadrants

The HexVS DCVS instruments are not floating voltage supplies. As a result, their outputs cannot be stacked to provide greater output voltages. Moreover, you must wire the ground reference (VSG) for each DCVS to the voltage supply return (VS_RTN_n) along with return sense (DUT_RTN_S_n) and at the same voltage as the device ground sense (DGS). For more information, see [HexVS DIB Design Guidelines](#).

Current Sourcing

Each DCVS can source up to 15A for force voltages up to 3.6V. The maximum available current decreases when force voltage exceeds 3.6V at a rate of 2.73A/V down to 9.8A at 5.5V. For more information, see [Output Power Compliance](#).

With merged configurations, the instrument provides up to 30A, 60A, or 90A of output current load capacity at force voltages up to 3.6V. At programmed voltages above 3.6V, the maximum output current is regulated in a similar fashion as the non-merged case. For more information, see Merged Mode.

Current Sinking

The HexVS DCVS instruments are designed primarily to source high device power supply currents. However, each DCVS has a limited current sinking capability for output voltages down to 0.0V. The primary purpose of the DCVS sink capability is to rapidly discharge DIB bypass capacitors when the DCVS output is gated off while connected to the DIB. Each DCVS can handle a short sink current transient of up to 15A and continuously sink up to 2.5A.

<		>	
		2015 Teradyne, Inc. All rights reserved.	

hxttheory.2.11

<

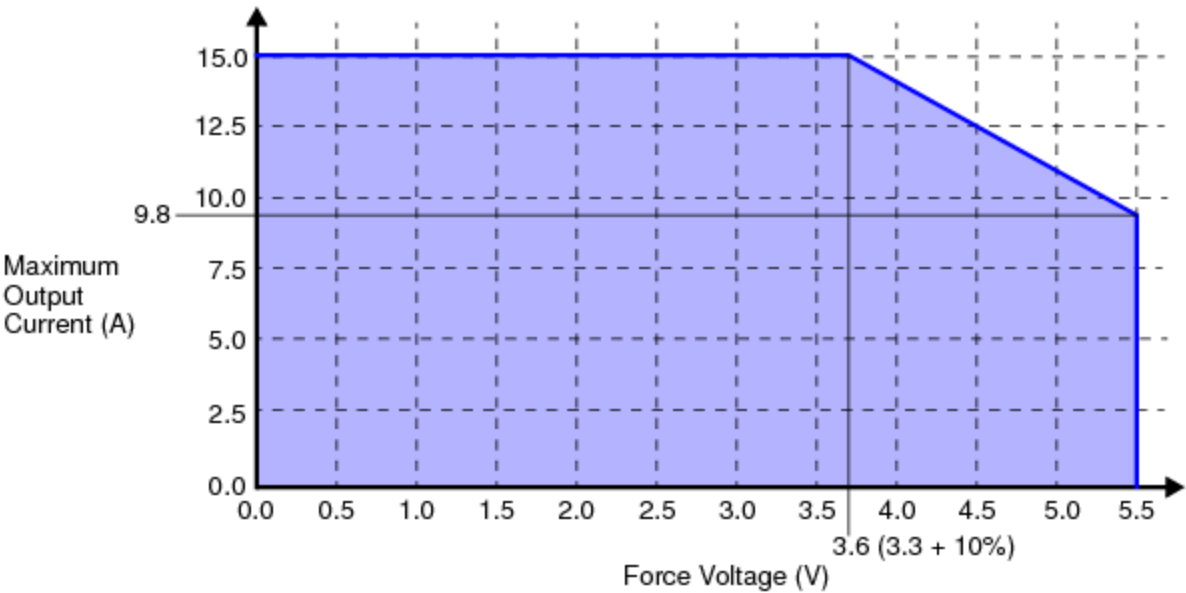
>

HexVS Theory of Operation : Output Power : Output Power Compliance

Output Power Compliance

Each HexVS voltage supply can deliver a maximum output power of 54W and dissipate a maximum of 25W. The following figure shows the output current versus force voltage profile for one voltage supply.

This profile scales up as you merge two, four, or six voltage supplies. This voltage to current curve is based on the fundamental electrical power relationship of total output power being the product of output voltage and current ($P = VI$). In merged mode, the maximum output power is 324W. Note that the HexVS cannot supply maximum current at maximum voltage, no matter what configuration you are using. The maximum current you can source at 3.6V is 15A. At 5.5V, you can force the maximum 9.8A.



HexVS Output Current Versus Force Voltage

The following table shows the power compliance merge configuration and the maximum DCVS output current.

Power Compliance and Maximum HexVS DCVS Output Current

Merge Configuration	Programmed DCVS Output Voltage (Vprog)		
	0V - 3.6V	3.6V < Vprog <= 5.5V	at 5.5V
None	15A	(24.83 - 2.73*Vprog)A	9.82A
Merged2	30A	(49.66 - 5.46*Vprog)A	19.63A
Merged4	60A	(99.31 - 10.92*Vprog)A	32.26A
Merged6	90A	(148.97 - 16.38*Vprog)A	58.89A

<

>

hxttheory.2.12

<

>

HexVS Theory of Operation : Output Power : Shared Tracking Supply Pair Considerations

Shared Tracking Supply Pair Considerations

Pairs of HexVS DCVS outputs share the same DC-DC tracking supply. The following table shows which DCVS output pairs share a tracking supply.

HexVS DCVS Output Shared Tracking Supply Pairs

Shared Tracking Supply	DCVS Linear Output Pair
1	VS1 and VS2
2	VS3 and VS4
3	VS5 and VS6

Note:

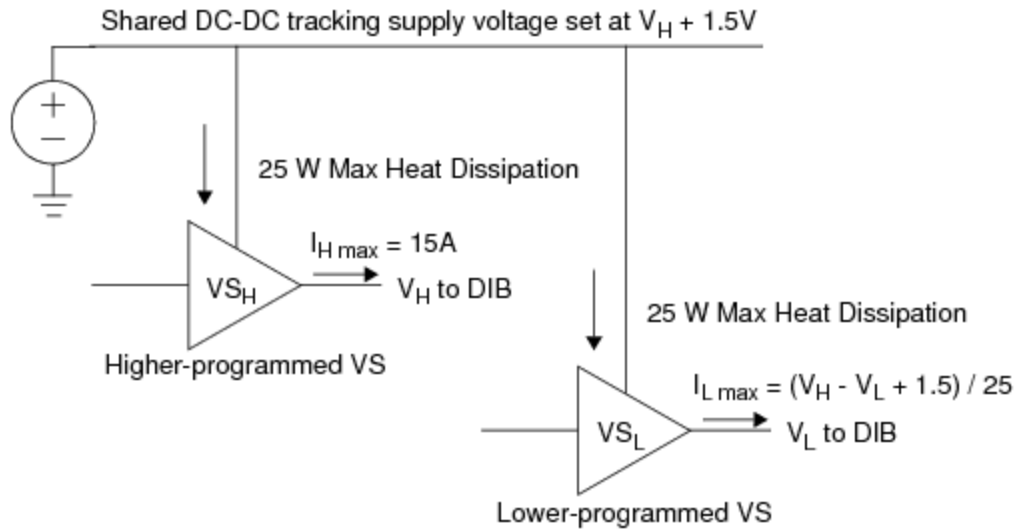
You must account for DCVS shared tracking supply pair characteristics in your DIB design. Consult with a test engineer who is familiar with the device test program or plan to ensure that the HexVS channel assignment supports the required power supply voltage and load currents.

Power Dissipation

The following figure shows power dissipation for a tracking voltage set at $V_H + V_O$, where V_H is the higher output voltage setting of the pair, V_L is the lower output voltage setting of the pair, and V_O is the tracking supply overhead. The shared DC-DC tracking supply voltage is set to 1.5V above the highest programmed output voltage of the pair ($V_O = 1.5V$). So, when the output voltages are set to different values, additional power is dissipated as heat in the lower-voltage VS linear output stage.

Note the following:

- Each supply can deliver a maximum of 54W to the DUT ($15A * 3.6V$; $9.81A * 5.5V$)
- Up to 25W of tracking supply power can be dissipated
- Pairs of supplies share the same DC-DC tracking voltage



DCVS Shared Tracking Supplies Maximum Power

A consequence of sharing the same tracking supply is that more heat is dissipated internally by the lower-voltage DCVS output. This is because the DCVS tracking supply voltage is always set to a value above the higher of the two attached DCVS voltage output stages. When the programmed output voltages for an adjacent DCVS pair are unequal, excess tracking power is dissipated as heat by the lower-voltage DCVS output. The heat that can be dissipated by a single DCVS is 25W. To prevent overheating, HexVS hardware reduces the maximum current of the lower-voltage DCVS to ensure that heat dissipation does not exceed this value.

When you program DCVS outputs sharing the same tracking supply to the same output voltage, no excess tracking power is dissipated. So, only the 54W DCVS output power compliance maximum current restriction is in effect. Shared tracking supply pair characteristics do not apply to merged DCVS supplies.

The following equations show the dissipated shared tracking supply power (heat):

- Power for Higher Voltage VS = $I_H * (1.5 + (V_H - V_H))$

For the higher-voltage VS, programmed to V_H , the tracking power increases only when output current to the DUT increases. (The maximum is $15.5 * 1.5A = 23.5W < 25W$.)

- Power for Lower Voltage VS = $I_L * (1.5 + (V_H - V_L))$

For the lower-voltage VS, programmed to V_L , the tracking power increases faster when the difference between programmed VS output values increases. If I_L is too high, then output power exceeds the 25W heat dissipation limit, and the board may overheat, triggering a HexVS board alarm.

HexVS hardware monitors output voltage settings and programmed current limit levels. When the difference in voltage between a shared tracking supply pair increases, the hardware checks the programmed current limit levels. If the current limit level on the lower-voltage VS is too high, the hardware overrides the current limit value by setting it to a safe value to protect the HexVS from overheating.

Maximum Output Current

The following table describes the following maximum power equation:

Maximum Current ($I_{L\ max}$) = $25W / (1.5V + (V_H - V_L))$

where:

V_H = Programmed output value of the higher voltage DCVS

V_L = Programmed output value of the lower voltage DCVS

To prevent overheating, HexVS hardware adjusts the maximum output current on the lower voltage supply to approximate the power curve defined by the maximum current equation. The lowest value for $I_{L\ max}$ is 3.5A.

 Note:

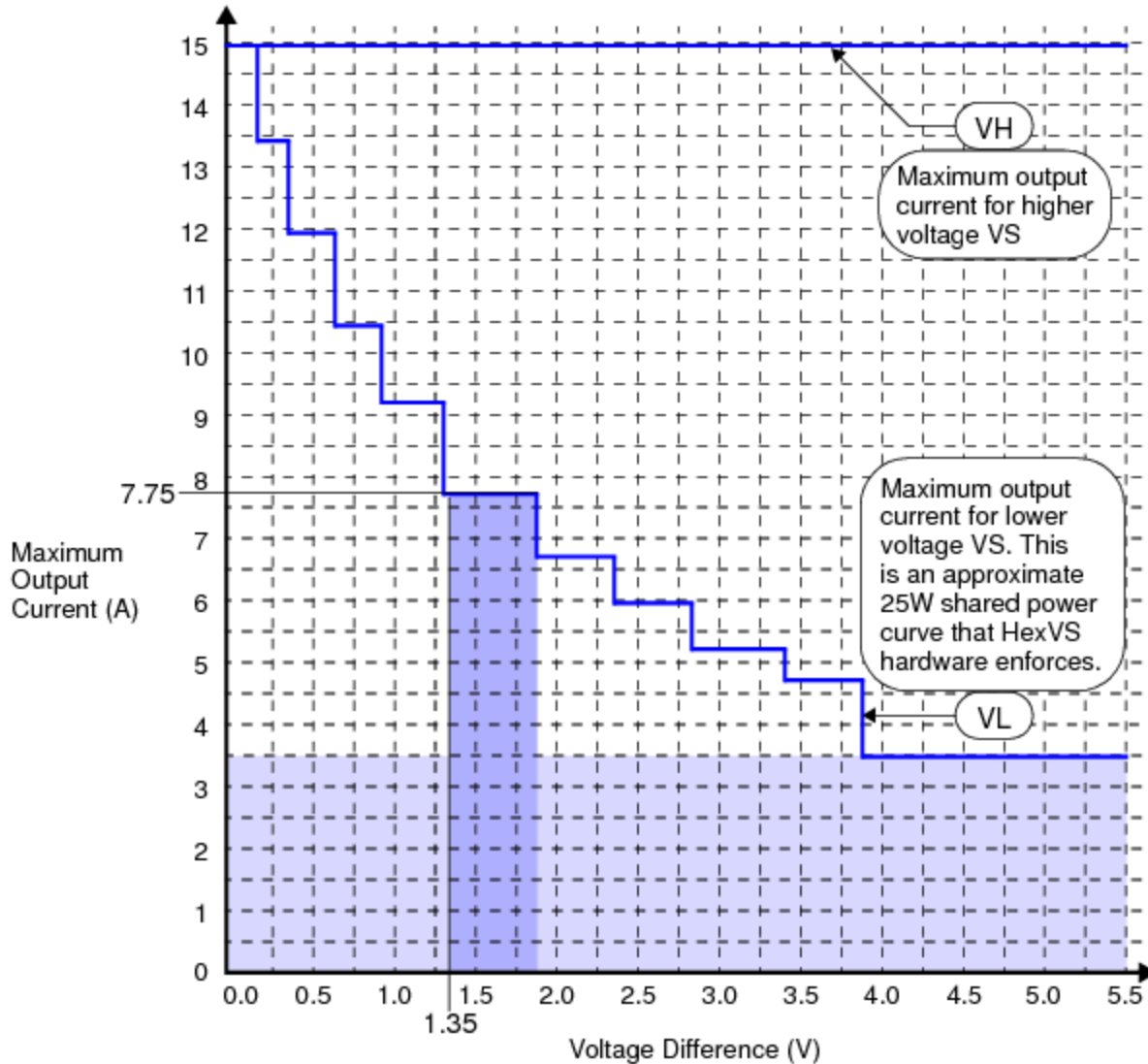
If a pair of device pins has a pin that requires a lower voltage and less than 3.5A of current, then the lower-voltage pin can be assigned to the same DCVS pair as a higher-voltage pin.

Independent DCVS Pair Voltage Difference and Maximum Current Limit

Voltage Difference ($V_H - V_L$)	Maximum Current at V_H ($V_H \leq 3.6V$)	Maximum Current at V_L
0.0V to 0.20V	15.00A	15.00A
>0.20V to 0.40V	15.00A	13.50A
>0.40V to 0.65V	15.00A	12.00A
>0.65V to 0.90V	15.00A	10.50A
>0.90V to 1.30V	15.00A	9.25A
>1.30V to 1.80V	15.00A	7.75A
>1.80V to 2.30V	15.00A	6.75A
>2.30V to 2.80V	15.00A	6.00A

>2.80V to 3.40V	15.00A	5.25A
>3.40V to 3.80V	15.00A	4.75A
>3.90V	15.00A	3.50A

The following figure shows the maximum output current for a shared tracking supply pair. In this example, the pair consists of VS3 and VS4, where VS3 equals 2.25V and VS4 equals 3.6V. Since VS4 is the higher voltage VS or (VH) and VS3 is the lower voltage VS (VL), the voltage difference between VH and VL equals 1.35V. From this voltage difference, the table above indicates that the maximum output current for VS3 is 7.75A and for VS4 is 15.0A.



Maximum Output Current for Shared Tracking Supply Pairs

Channel Allocation Guidelines

Tracking voltage sharing has important implications for VS power pin-to-channel allocation during DIB design. Since HexVS tracking supplies are shared, carefully consider DCVS pin assignments for nonmerged (independent) supplies.

✓ **Note:**

Due to the HexVS shared tracking supply characteristics, test programmers must help assign HexVS channels before a DIB is designed and fabricated.

Follow these guidelines when assigning HexVS channels:

- Carefully consider per pin voltage ranges and maximum current requirements when pairing up pin assignments. In general, these guidelines apply:

- Core logic power pins (often named VDD) require both the lowest voltage (less than 2V) and the highest current during test. Consequently, be especially careful when assigning these pins.

- I/O power pins often require higher voltages (2.5 to 5.0V), but less current than core logic.

- Internal clock (PLL) power pins often require core voltage levels, but little current (less than 2A).

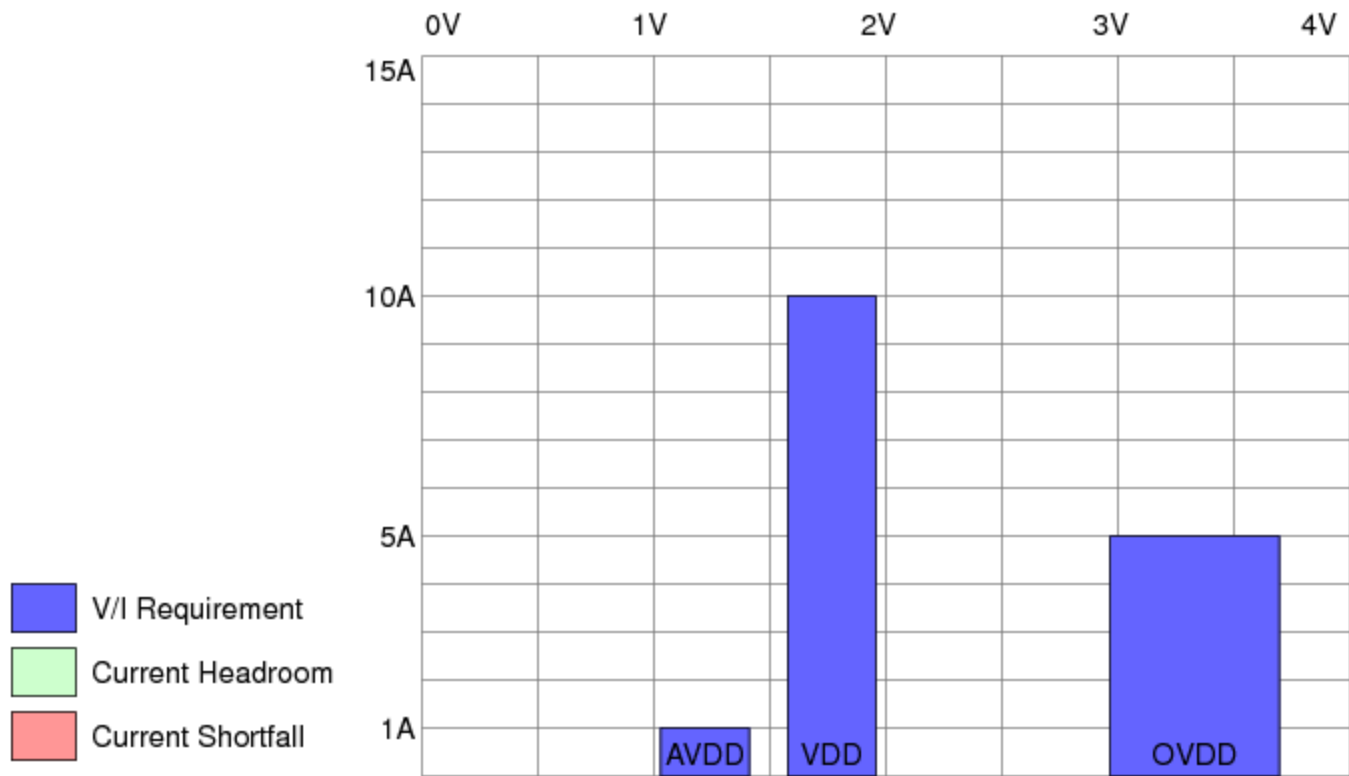
To assess test program voltages, use the specified minimum and maximum values, not only the nominal or typical value. Also consider other test plan voltages and currents that you expect during testing and device characterization.

The following examples illustrate how to decide which pins can be shared on the same tracking supply. The first figure shows a single site and three power pins with the following voltage and current tester requirements:

- VDD: 1.8V 10% (1.62 to 1.98V) with maximum current = 10A

- OVDD: 3.3V 10% (2.97 to 3.63V) with maximum current = 5A

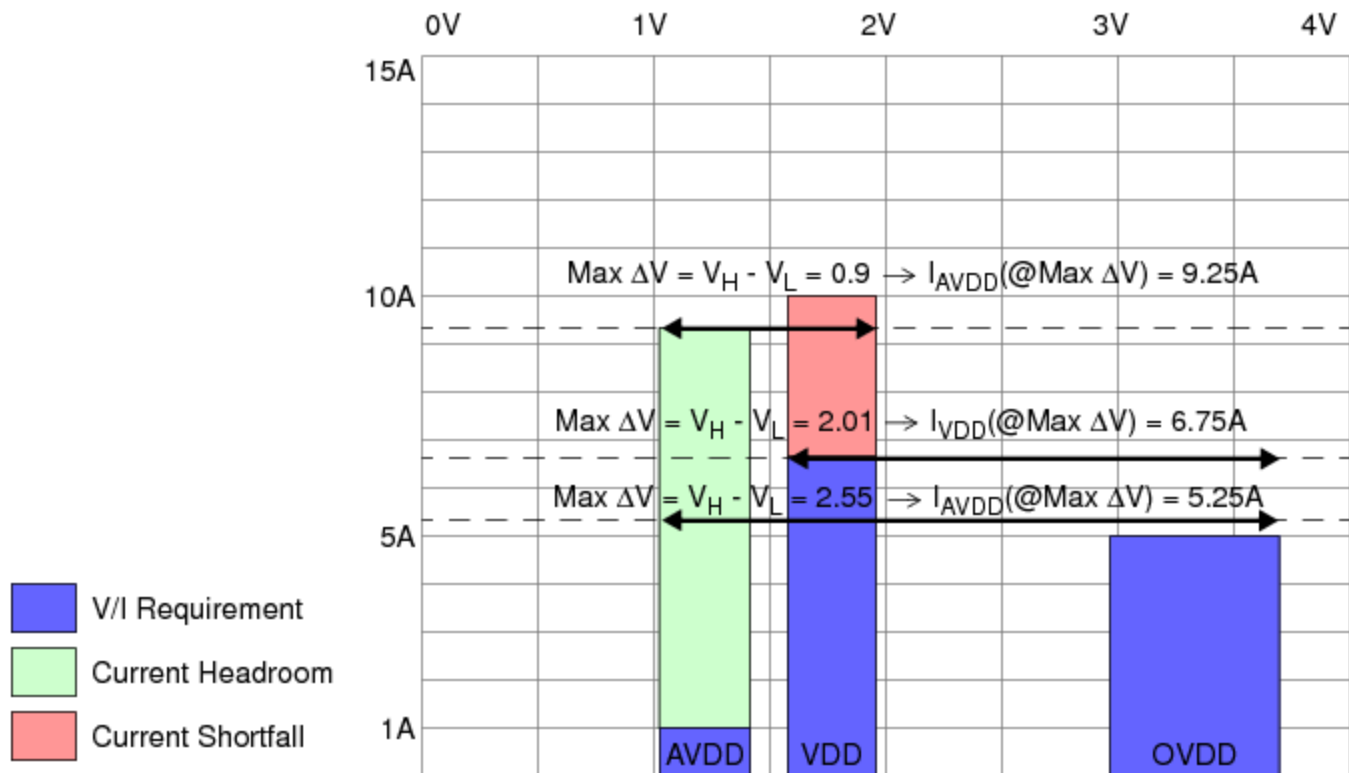
- AVDD: 1.2V 10% (1.08 to 1.32V) with maximum current = 1A



Shared Tracking Supply Pair First Example Pin Specification Requirements

On analysis of pin requirements and shared tracking supply pair considerations, you can draw the following conclusions, which are illustrated in the figure below:

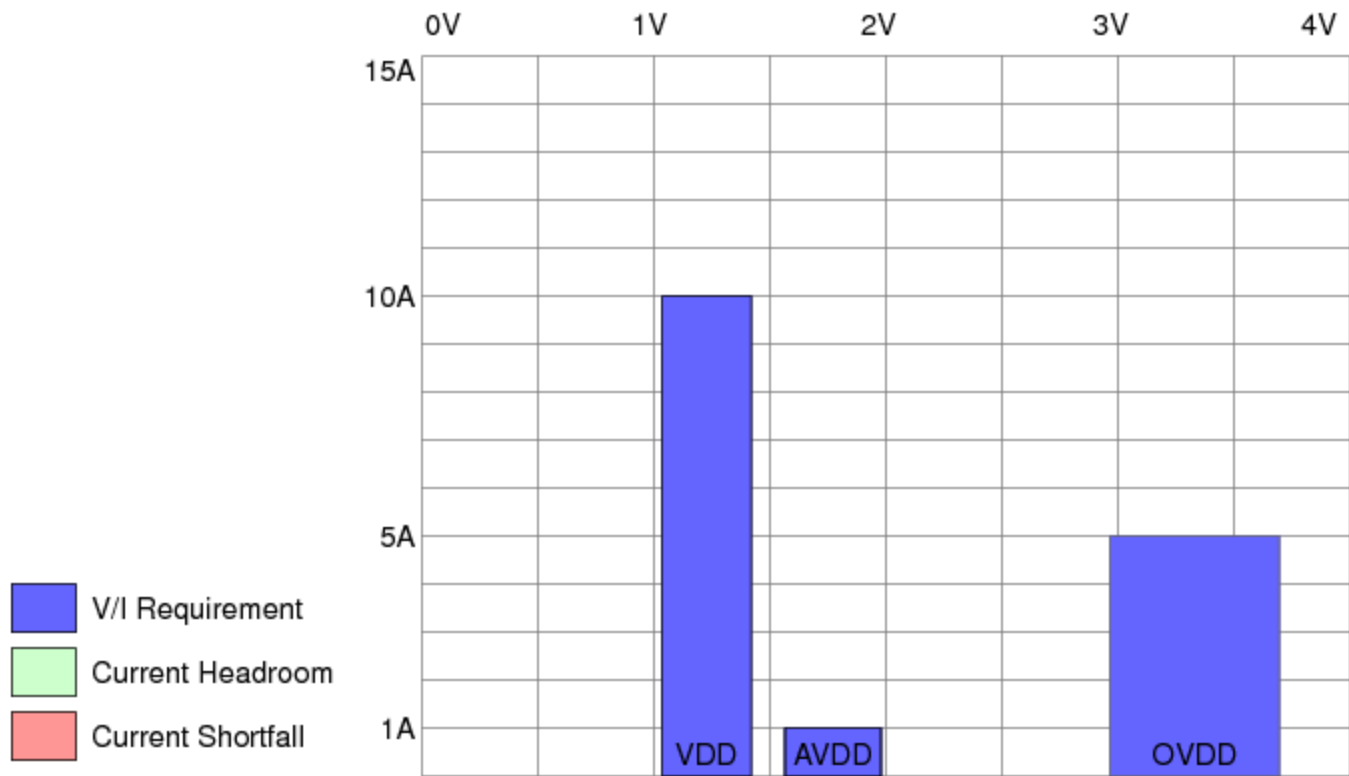
- AVDD can be assigned to the same shared tracking supply pair as VDD or OVDD.
- VDD cannot be assigned to the same shared tracking supply pair as OVDD.



Shared Tracking Supply Pair First Example Conclusions

The second example also shows a single site and three power pins with the same current requirements as in the first example. In this example, however, the voltage requirements have changed, where the VDD and AVDD specifications from the first example have been swapped:

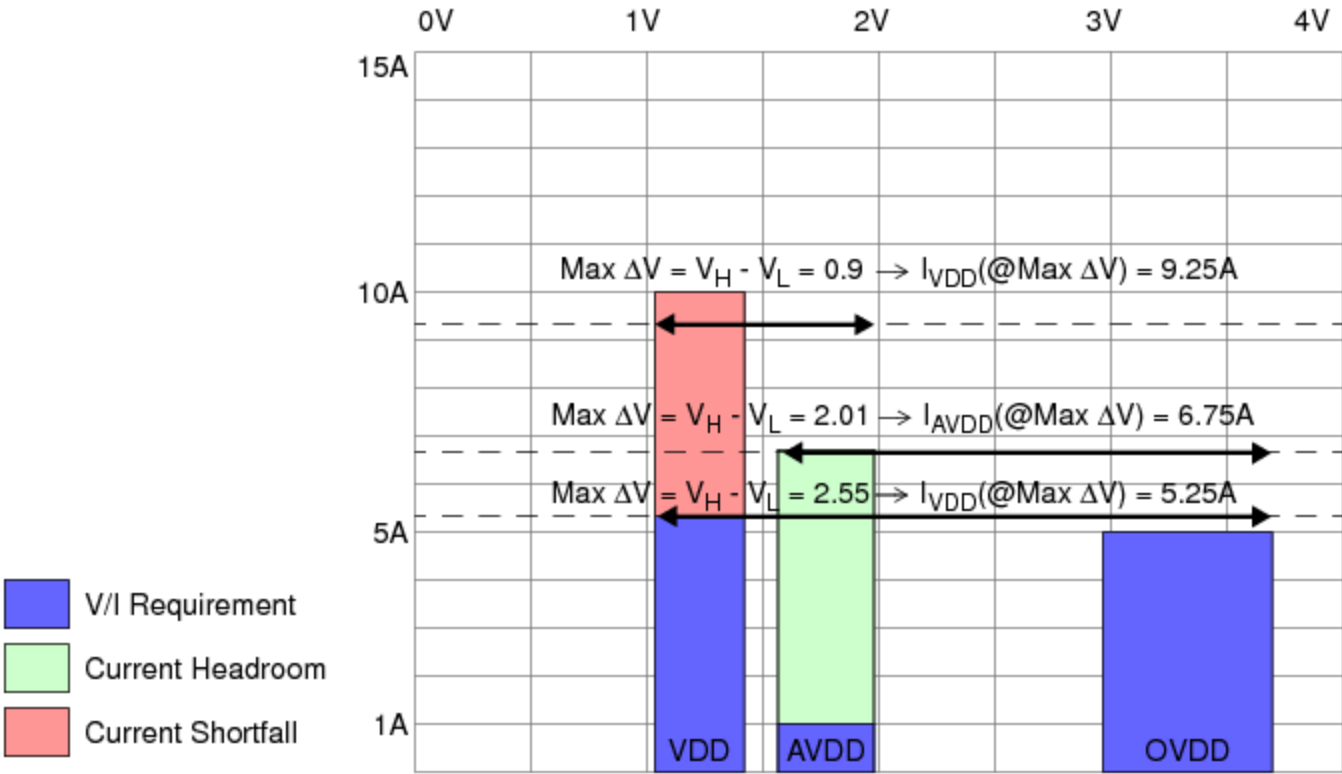
- VDD: 1.2V 10% (1.08 to 1.32V) with maximum current = 10A
- OVDD: 3.3V 10% (2.97 to 3.63V) with maximum current = 5A
- AVDD: 1.8V 10% (1.62 to 1.98V) with maximum current = 1A



Shared Tracking Supply Pair Second Example Pin Specification Requirements

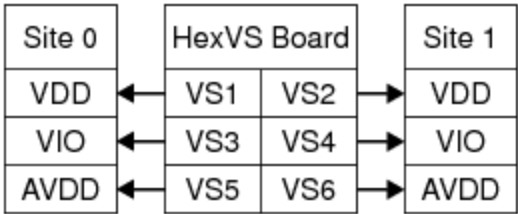
On analysis of the pin requirements and shared tracking supply pair considerations, you can draw the following conclusions, which are illustrated in the figure below:

- VDD cannot be assigned to the same shared tracking supply pair as AVDD or OVDD.
- AVDD and OVDD can be assigned to the same shared tracking supply pair.



Shared Tracking Supply Pair Second Example Conclusions

- Break up shared DC-DC tracking supply pairs between sites. This ensures that shared tracking supply pairs are always programmed to the same output voltage value.



Shared Tracking Supply Pair Multisite Example

- If lower-voltage pins do not require more than 3.5A, then they can safely share a tracking supply voltage with a higher voltage pin.
- The shared DC-DC tracking supply voltage characteristic does not affect merged channels. If a device power pin requires more than 15A, these guidelines do not apply.

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

hxttheory.2.13

<

>

HexVS Theory of Operation : Output Power : Merged Mode

Merged Mode

The HexVS allows you to merge its DCVS outputs to provide up to 90A of current to a device pin. Although multiple HexVS boards can be installed in a test system, board-to-board merging is not supported. The following table shows available merged mode configurations.

☒ **Note:**

Not all supplies must be merged for use. Adjacent pair characteristics do not apply to merged supplies.

Merged Mode Configurations			
Merged Supplies	Channel Type	Leader Channel	Maximum Current
None	DCVS	Not applicable	15A
VS1 and VS2	DCVSMerged2	VS1	30A
VS3 and VS4	DCVSMerged2	VS3	30A
VS5 and VS6	DCVSMerged2	VS5	30A
VS1, VS2, VS3, and VS4	DCVSMerged4	VS1	60A
VS3, VS4, VS5, and VS6	DCVSMerged4	VS3	60A
VS1, VS2, VS3, VS4, VS5, and VS6	DCVSMerged6	VS1	90A

HexVS supplies can only be merged in pairs. However, not all outputs need to be merged across the board.

DIB power plane wiring must be designed to support DCVS merging. For more information, see [HexVS DIB Design Guidelines](#).

In IG-XL, you specify the merge configuration in a Channel Map sheet using the DCVS channel types: DCVS, DCVSMerged2, DCVSMerged4, and DCVSMerged6. The merge configuration for HexVS supplies cannot be changed during test program run time. So, the configuration specified in the channel map can be considered static.

⚠ Caution:

Confirm that the DCVS merge configuration specified in the channel map matches the merge configuration of the DIB. If the merge configurations do not match, you can damage the DIB and the tester hardware during test program run time.

Adjacent pair power characteristics do not apply to merged DCVS outputs. Maximum power output compliance still applies with merged supplies. However, it is scaled up by the applicable merge factor (two, four, or six). For more information, see [Output Power Compliance](#).